

bmpracproject (/github/shoez-sweg/bmpracproject/tree/1982fc5180323c29c4eec3f156cf2ea82eb47b2f)
/
BMPRAC_Project (3).ipynb (/github/shoez-sweg/bmpracproject/tree/1982fc5180323c29c4eec3f156cf2ea82eb47b2f/BMPRAC_Project (3).ipynb)

Analysis of Relationship between Sleep Duration and Alcohol-related Liver Dysfunction

1.1 Group members

1. Chan Guan Zi, Violet S10267086G
2. Koh Shu Yun S10240960J
3. Lim Xinying S10266685E

1.2 Class: 2MO2

2. Declaration of Contribution

We confirm that all members of the group contributed equally to the design, implementation, analysis, and writing of this research project.

3. Summary

Alcohol-related liver dysfunction (ALD) is a growing concern in the U.S., particularly among Hispanic males. Sleep duration has been identified as a potential modifier of metabolic and liver health, but its relationship to ALD remains unclear.

This study investigates the relationship between sleep duration and liver enzyme levels (ALT, AST, GGT), and the combined impact of lifestyle factors (alcohol intake, BMI, physical activity) on ALD risk in Hispanic males aged 40–60 using NHANES 2007–2018 data.

Data from six NHANES cycles were merged and filtered for the target demographic. ALD risk was defined as AST/ALT ratio > 2 and GGT > 50 U/L. Median trends, boxplots, heatmaps, and nonparametric tests (Spearman, Kruskal–Wallis) were used to explore associations.

Median levels of all liver enzymes and independent variables were tracked across cycles, where ALT and AST levels remained generally stable, while GGT showed more variability. Results from data analysis show that no significant direct association was found between sleep duration and individual liver enzymes. However, higher ALD risk was observed among participants with shorter sleep duration in combination with heavy alcohol use and elevated BMI. Annual ALD risk trends remained low ($< 5\%$) but showed slight increases post-2013.

In conclusion, while sleep duration alone was not a strong predictor of liver enzyme elevation, its interaction with alcohol and BMI may contribute to increased ALD risk.

This project builds on existing clinical knowledge by applying validated ALD criteria to a large national dataset, investigating the novel role of sleep duration in ALD risk, and focusing on an underserved but vulnerable demographic.

4. Background and Rationale

4.1 Health Issue

Alcohol-related liver dysfunction is a major public health issue in the United States. Alcohol-associated liver disease happens over years of continuous alcohol abuse or alcohol use disorder. It causes toxic fat to accumulate in your liver, which leads to inflammation and cirrhosis. It encompasses a spectrum of liver conditions, including alcoholic fatty liver, steatosis, fibrosis, and cirrhosis, caused by chronic excessive alcohol consumption.

436 814 ALD deaths were recorded (308 923 men [70.7%]), with ALD mortality increasing from 6.71 to 12.53 deaths per 100 000 between 1999 and 2022. The sharp increase in mortality shows that targeted interventions among high-risk populations are needed.

These conditions often progress silently as there are no symptoms in the early stages. However, there are elevated liver enzymes (ALT, AST, GGT) serving as early biomarkers of liver injury. Despite its severity, alcohol-related liver disease remains underdiagnosed and underrepresented in targeted research, especially among high-risk subgroups, making early detection and lifestyle-based prevention strategies critically important.

Biomarker	Full Name	What It Indicates	How It Relates to the Health Issue
ALT (LBXSAT)	Alanine Aminotransferase	Damage to liver cells (hepatocellular injury)	Elevated in early NAFLD, hepatitis, and alcohol-related liver inflammation
AST (LBXAST)	Aspartate Aminotransferase	Liver and muscle damage	Elevated in alcohol-related liver injury , especially when AST > ALT
GGT (LBXSGT)	Gamma-Glutamyl Transferase	Bile duct function and oxidative stress	Sensitive to alcohol intake and used as a marker for ALD and cholestasis

4.2 Importance of Research Question in Public Health or Biomedical Science

This research question is highly significant in public health and biomedical science because it addresses the early detection and prevention of alcohol-related liver dysfunction in a high-risk and underrepresented population, Hispanic males aged 40–60. Prior studies have shown that chronic liver disease is the leading cause of death in the Hispanic population. On top of that, for Hispanic males aged 45–54, it was the fourth leading cause of death. Seeing that this demographic is more susceptible to liver diseases, tailoring interventions to them can reduce health disparities and improve outcomes.

By examining the combined effects of modifiable lifestyle factors such as sleep duration, alcohol consumption, BMI, and physical activity on liver enzyme levels (ALT, AST, GGT), the study offers a comprehensive approach to identifying early markers of liver damage. Leveraging six cycles of NHANES data (2007–2018) enhances the robustness and generalizability of the findings, enabling trend analysis and subgroup insights. This work fills a critical gap in the literature by integrating multiple risk factors and focusing on a specific demographic, thereby informing targeted interventions and contributing to the broader understanding of liver disease pathophysiology and prevention.

4.3 PubMed Article Citations

1. Yao, L. and Chen, T. (2024) 'A combined association of alanine aminotransferase, aspartate transaminase and bilirubin with sleep duration in aged 16–85 years (2005–2010),' *Medicine*, 103(49), p. e40915.
<https://doi.org/10.1097/md.00000000000040915> (<https://doi.org/10.1097/md.00000000000040915>).

2. Yeung, C.H.C., Bauer, C. and Xiao, Q. (2023) 'Associations between Rest–Activity Rhythms and Liver Function Tests: The US National Health and Nutrition Examination Survey, 2011–2014,' *Clocks & Sleep*, 5(4), pp. 667–685.
<https://doi.org/10.3390/clockssleep5040045> (<https://doi.org/10.3390/clockssleep5040045>).

3. Zong, G. et al. (2024) 'Association of sleep patterns and disorders with metabolic dysfunction-associated steatotic liver disease and liver fibrosis in contemporary American adults,' *Annals of Hepatology*, p. 101583.
<https://doi.org/10.1016/j.aohep.2024.101583> (<https://doi.org/10.1016/j.aohep.2024.101583>).

4.4 NHANES Usage, Extension of Existing Work

This study utilizes data from six cycles of the National Health and Nutrition Examination Survey (NHANES), spanning 2007–2018, which provides a nationally representative dataset of U.S. adults. NHANES includes detailed information on demographics, lifestyle behaviors (e.g., alcohol use, physical activity, sleep), and laboratory biomarkers, including liver enzymes (ALT, AST, GGT). While previous studies have used NHANES to explore individual risk factors, such as alcohol intake or sleep duration, in relation to liver health, they often focus on the general population or specific conditions like MASLD.

This project extends existing work by focusing on a specific high-risk demographic, Hispanic males aged 40–60. Integrating multiple modifiable lifestyle factors (alcohol consumption, BMI, sleep duration, physical activity) in relation to early liver dysfunction, as indicated by liver enzyme levels. Using six NHANES cycles to enhance statistical power, allow for trend analysis, and improve the generalizability of findings. Bridging the gap between metabolic and alcohol-related liver disease research, offering insights into early detection and prevention strategies tailored to a vulnerable population.

5. Research Questions and Objectives

5.1 Research Question

What is the relationship between sleep duration and liver enzyme levels (ALT(BM1), AST(BM2), GGT(BM3)) among Hispanic males in the USA aged 40–60, using NHANES data from 2007–2018, in the context of identifying risk factors for alcohol-related liver dysfunction?

5.2 Literature Review

Alcohol-related liver dysfunction is a growing public health concern, particularly among Hispanic males aged 40–60 in the United States. This demographic faces elevated risks due to a combination of lifestyle and metabolic factors. While liver enzyme biomarkers such as ALT, AST, and GGT are widely used to detect early liver dysfunction, the role of modifiable behaviors, like sleep duration, remains underexplored in this population. This literature review presents current knowledge, identifies gaps, and positions the present study within the broader research landscape.

Recent research has begun to explore the relationship between sleep and liver health, but key gaps remain, particularly in relation to liver enzyme biomarkers and specific demographic groups.

Yao and Chen (2024) examined the association between sleep duration and liver markers (ALT, AST, and bilirubin) in U.S. adults aged 16–85 using NHANES 2005–2010 data. While they found that both short and long sleep durations were associated with abnormal liver markers, their study did not include GGT, nor did it stratify by ethnicity or gender, limiting its relevance to high-risk subgroups such as Hispanic males aged 40–60.

Yeung, Bauer, and Xiao (2023) investigated rest and activity rhythms and liver function tests using NHANES 2011–2014, identifying associations with GGT and ALP. However, their focus was on circadian rhythmicity rather than sleep duration, and they did not analyze ALT or AST in depth or target specific demographic groups.

Zong et al. (2024) explored the relationship between sleep patterns and metabolic dysfunction-associated steatotic liver disease (MASLD) and liver fibrosis in a general adult population. Although they found that short and late sleep were risk factors, the study used diagnostic criteria rather than enzyme biomarkers and did not include GGT or integrate other lifestyle factors such as alcohol intake, BMI, or physical activity. None of these studies focused on Hispanic males aged 40–60, a group with elevated risk for alcohol-related liver dysfunction.

Our study addresses this gap by analyzing ALT, AST, and GGT in this underrepresented population using six NHANES cycles (2007–2018), while integrating sleep duration, alcohol use, BMI, and physical activity to better understand early indicators of liver dysfunction.

Hispanic individuals have significantly higher age-adjusted mortality from chronic liver disease than non-Hispanic Whites (≈ 13.7 vs. 9.2 per 100,000).

This disparity is partially attributed to NAFLD, alcoholic liver disease, and viral hepatitis, with Hispanics showing higher prevalence and worse outcomes.

The age range of 40 to 60 years is a critical period for the development and detection of chronic liver diseases, including non-alcoholic fatty liver disease (NAFLD) and alcohol-related liver disease (ALD). Epidemiological studies show that liver enzyme elevations (ALT, AST, GGT) become more common and clinically significant in this age group due to the accumulation of metabolic risk factors such as obesity, insulin resistance, alcohol use, and sedentary lifestyle (Younossi et al., 2016; Tapper & Parikh, 2018).

According to data from the CDC and NHANES-based analyses, individuals aged 40–60 experience a notable rise in the prevalence of elevated ALT and AST, which often reflects early or undiagnosed liver dysfunction. Moreover, Hispanic males in this age group are disproportionately affected by NAFLD and ALD, with higher rates of both liver enzyme abnormalities and liver-related mortality compared to other ethnic and gender groups (Rodriguez-Torres, 2011).

Focusing on this age group enables earlier identification of liver dysfunction during a potentially reversible phase, allowing for meaningful lifestyle interventions. It also reduces the confounding effects of age-related comorbidities common in older populations (e.g., cardiovascular or renal disease), improving the specificity of findings.

So far, a study focusing on high risk groups and taking in multiple variables like modifiable lifestyle factors and biomarkers has yet to be done. Thus, emphasising our study's originality and novelty.

5.3 Gaps in Existing Studies

**1. Yao, L. and Chen, T. (2024) 'A combined association of alanine aminotransferase, aspartate transaminase and bilirubin with sleep duration in aged 16–85 years (2005–2010),' Medicine, 103(49), p. e40915.
<https://doi.org/10.1097/md.00000000000040915>
(<https://doi.org/10.1097/md.00000000000040915>).**

Existing Knowledge:

This prior study had found associations between short/long sleep duration and liver enzyme levels. On top of that sleep disruptions may affect metabolic and liver health.

Gaps:

There is no ethnic group analysis (eg. Hispanic Males). The study also did not include GGT or integrate alcohol or physical activity. Furthermore, they used only older NHANES cycles (2005–2010), so the datasets may be outdated

**2. Zong, G. et al. (2024) 'Association of sleep patterns and disorders with metabolic dysfunction-associated steatotic liver disease and liver fibrosis in contemporary American adults,' Annals of Hepatology, p. 101583.
<https://doi.org/10.1016/j.aohep.2024.101583>
(<https://doi.org/10.1016/j.aohep.2024.101583>).**

Existing knowledge:

Sleep duration and disorders are independently associated with metabolic dysfunction-associated steatotic liver disease and liver fibrosis. As sleeping later or for a shorter duration is a shared risk factor. They concluded that the effects are stronger in women and certain age groups.

Gaps:

Like in the previous study there was no ethnic subgroup analysis, it only focuses on contemporary American adults where our chosen demographic (Male Hispanics aged 40–60) will be underrepresented in. Additionally, they did not specify 'certain age groups'. Rather, they mainly focus on sleep and does not include other variables like alcohol consumption, BMI, physical activity, and sleep. Furthermore this study uses diagnostic criteria for MASLD or fibrosis, not enzyme levels as early indicators. This study also only takes a snapshot of the data as they only use 2 NHANES cycles (2017–2020). With only two cycles the statistical power to detect meaningful associations in stratified analyses is lower. This study does not differentiate between alcohol-related and metabolic-related liver dysfunction.

How our project proposal fills a gap in the current knowledge:

We are conducting the research on Hispanic males aged 40–60, which is a demographic underrepresented in liver enzyme research. We analyse ALT, AST, and GGT levels, which can provide further insight into alcohol-related liver dysfunction. We will also consider alcohol intake, BMI and physical activity along with sleep duration, using multiple variables to assess how

lifestyle factors influence liver function. In order to improve precision of estimates and to enhance the representativeness of subgroups, we will use 6 newer and more comprehensive NHANES cycles from 2007-2018. On top of that we will focus on alcohol-related liver dysfunction

**3. Yeung, C.H.C., Bauer, C. and Xiao, Q. (2023) 'Associations between Rest–Activity Rhythms and Liver Function Tests: The US National Health and Nutrition Examination Survey, 2011–2014,' *Clocks & Sleep*, 5(4), pp. 667–685.
<https://doi.org/10.3390/clockssleep5040045>
(<https://doi.org/10.3390/clockssleep5040045>).**

Existing knowledge:

This prior study shows rest–activity rhythm measures (e.g., F statistic, amplitude, interdaily stability) from accelerometer data were associated with liver function markers lower overall rhythmicity correlated with higher odds of abnormal ALP and GGT, and lower albumin. The associations with liver enzymes remained significant even after controlling for: Age, gender, ethnicity, socioeconomic status, along with lifestyle factors, including sleep and physical activity, as well as BMI, diabetes, and hepatitis status.

Gaps:

The study measures overall circadian rhythmicity, not simple sleep duration. While adjusted for ethnicity, it doesn't specifically stratify or analyze Hispanic males aged 40–60. It includes ALP and albumin but doesn't focus on ALT, AST in depth | ALT/AST associations are included but circadian-focused. The study is also limited to 2011–2014 only. Additionally, the objective was circadian rhythms, with alcohol/BMI included only as confounders, not as primary variables of interest.

How our research question fills this gap:

Our research question focused directly on sleep duration instead of overall circadian rhythmicity. We also focus more on ALP, ALT and AST, and target the specific demographic of hispanic males aged 40-60. We also utilised a broader time frame of 2007 - 2018.

5.4 Objectives

1. To examine how sleep duration, alcohol consumption, BMI and physical activity affect liver enzyme levels among Hispanic males aged 40–60 in the US from 2007–2018.
2. Statistical analysis and visualisation of six merged datasets from 2007-2018 to provide sufficient sample size of Hispanic males aged 40-60 in the US.
3. Dependent variables are liver enzyme levels (ALT, AST, GGT) which are related to alcohol-related liver dysfunction.

6. Variables and Data Sources

Variables

Dependent variables (Biomarkers, Continuous)

Variable	Unit	Source Dataset	Description	Clinical Significance
ALT (LBXSAT)	U/L	BIOPRO_E to BIOPRO_J	Alanine aminotransferase level in blood	Specific marker for liver cell injury; elevated in NAFLD, hepatitis, and alcohol-related liver disease
AST (LBXAST)	U/L	BIOPRO_E to BIOPRO_J	Aspartate aminotransferase level in blood	Enzyme found in liver and muscle; elevated in liver injury but less specific than ALT
GGT (LBXSGT)	U/L	BIOPRO_E to BIOPRO_J	Gamma-glutamyl transferase level in blood	Associated with alcohol-related liver dysfunction and bile duct issues

Independent variables (Continuous)

Variable	Unit	Source Dataset	Description	Clinical Significance
BMXBMI	kg/m ²	BMX_E to BMX_J	Body Mass Index calculated from height and weight	High BMI is linked to NAFLD and elevated liver enzymes
SLD010H	Hours	SLQ_E to SLQ_J	Average sleep duration on weekday nights	Abnormal sleep duration is associated with metabolic and hepatic dysfunction
PAD615	Minutes/week	PAQ_E to PAQ_J	Time spent in moderate/vigorous leisure-time physical activity	Low activity is a risk factor for fatty liver and metabolic syndrome
ALQ130	Drinks per day	ALQ_E to ALQ_J	Self-reported average daily alcohol intake	Chronic alcohol intake raises ALT, AST, and GGT levels

Independent variables (categorical)

Variable	Codes	Source Dataset	Description	Clinical Significance
RIAGENDR	1 = Male, 2 = Female	DEMO_E to DEMO_J	Biological sex of participant	Used to filter for males only
RIDRETH1	1 = Mexican American, 2 = Other Hispanic	DEMO_E to DEMO_J	Race/ethnicity categories	Used to filter for Hispanic participants
RIDAGEYR	Integer	DEMO_E to DEMO_J	Participant age in years	Used to filter for age group 40–60

The code below imports the python libraries needed for data analysis and visualisation. It also changes the display settings for easier viewing of data. It is also used for definition of the function used for cleaning the data.

```

In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
def safe_cleaning(df_input):
    import numpy as np
    import pandas as pd

    df = df_input.copy() # keep original safe

    # Ensure columns are uppercase
    df.columns = df.columns.str.upper()

    # List of variables we care about
    vars_to_check = ['SLD010H', 'ALQ130', 'PAD615']

    # Missing value summary
    total_rows = len(df)
    missing_report = {}

    for var in vars_to_check:
        if var in df.columns:
            missing_count = df[var].isna().sum()
            percent_missing = missing_count / total_rows * 100
            missing_report[var] = round(percent_missing, 2)

            # Handle sleep duration (impute if <30% missing)
            if var == 'SLD010H' and percent_missing < 30:
                median_val = df[var].median()
                df[var] = df[var].fillna(median_val)

            # Alcohol intake categorical
            if var == 'ALQ130':
                def alcohol_cat(x):
                    if pd.isna(x):
                        return 'Missing'
                    elif x == 0:
                        return 'Non-drinker'
                    elif x <= 2:
                        return 'Light (1-2)'
                    elif x <= 4:
                        return 'Moderate (3-4)'
                    elif x > 4:
                        return 'Heavy (5+)'
                    else:
                        return 'Other'
                df['ALQ130_CAT'] = df['ALQ130'].apply(alcohol_cat)

            # Physical activity categorical
            if var == 'PAD615':
                def activity_cat(x):
                    if pd.isna(x):
                        return 'Missing'
                    elif x == 0:
                        return 'Inactive (0)'
                    elif x < 150:
                        return 'Low (1-149)'
                    elif x >= 150:
                        return 'Active (150+)'
                    else:
                        return 'Other'
                df['PAD615_CAT'] = df['PAD615'].apply(activity_cat)

    # Display missingness report
    print("Safe Cleaning Summary:")
    for var, pct in missing_report.items():
        print(f" - {var}: {pct:.2f}% missing")

    return df

#set display settings for easier viewing
pd.set_option('display.max_columns', 100)

```

The code below imports the files to be analysed.

```
In [ ]: from google.colab import files
        uploaded = files.upload()
```

Choose Files No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving SLQ_J.xpt to SLQ_J.xpt
Saving SLQ_I.xpt to SLQ_I.xpt
Saving SLQ_H.xpt to SLQ_H.xpt
Saving SLQ_G.xpt to SLQ_G.xpt
Saving SLQ_F.xpt to SLQ_F.xpt
Saving SLQ_E.xpt to SLQ_E.xpt
Saving PAQ_J.xpt to PAQ_J.xpt
Saving PAQ_I.xpt to PAQ_I.xpt
Saving PAQ_H.xpt to PAQ_H.xpt
Saving PAQ_G.xpt to PAQ_G.xpt
Saving PAQ_F.xpt to PAQ_F.xpt
Saving PAQ_E.xpt to PAQ_E.xpt
Saving DEMO_J.xpt to DEMO_J.xpt
Saving DEMO_I.xpt to DEMO_I.xpt
Saving DEMO_H.xpt to DEMO_H.xpt
Saving DEMO_G.xpt to DEMO_G.xpt
Saving DEMO_F.xpt to DEMO_F.xpt
Saving DEMO_E.xpt to DEMO_E.xpt
Saving BMX_J.xpt to BMX_J.xpt
Saving BMX_I.xpt to BMX_I.xpt
Saving BMX_H.xpt to BMX_H.xpt
Saving BMX_G.xpt to BMX_G.xpt
Saving BMX_F.xpt to BMX_F.xpt
Saving BMX_E.xpt to BMX_E.xpt
Saving BIOPRO_J.xpt to BIOPRO_J.xpt
Saving BIOPRO_I.xpt to BIOPRO_I.xpt
Saving BIOPRO_H.xpt to BIOPRO_H.xpt
Saving BIOPRO_G.xpt to BIOPRO_G.xpt
Saving BIOPRO_F.xpt to BIOPRO_F.xpt
Saving BIOPRO_E.xpt to BIOPRO_E.xpt
Saving ALQ_J.xpt to ALQ_J.xpt
Saving ALQ_I.xpt to ALQ_I.xpt
Saving ALQ_H.xpt to ALQ_H.xpt
Saving ALQ_G.xpt to ALQ_G.xpt
Saving ALQ_F.xpt to ALQ_F.xpt
Saving ALQ_E.xpt to ALQ_E.xpt
```

The code below loads the files to be analysed. (alcohol intake, standard biochemical profile, body measure, demographic, physical activity, sleep disorders data from 2007-2018)

Loading demographic data

```
In [ ]: # Load 2007-2008 demographic data
        demo_e = pd.read_sas("DEMO_E.xpt", format='xport')
        demo_e["cycle"] = "2007-2008"
        # Load 2009-2010 demographic data
        demo_f = pd.read_sas("DEMO_F.xpt", format='xport')
        demo_f["cycle"] = "2009-2010"
        # Load 2011-2012 demographic data
        demo_g = pd.read_sas("DEMO_G.xpt", format='xport')
        demo_g["cycle"] = "2011-2012"
        # Load 2013-2014 demographic data
        demo_h = pd.read_sas("DEMO_H.xpt", format='xport')
        demo_h["cycle"] = "2013-2014"
        # Load 2015-2016 demographic data
        demo_i = pd.read_sas("DEMO_I.xpt", format='xport')
        demo_i["cycle"] = "2015-2016"
        # Load 2017-2018 demographic data
        demo_j = pd.read_sas("DEMO_J.xpt", format='xport')
        demo_j["cycle"] = "2017-2018"
```


Loading standard biochemistry profile data

```
In [ ]: # Load 2007-2008 standard biochemistry profile data
biopro_e = pd.read_sas("BIOPRO_E.xpt", format='xport')
biopro_e["cycle"] = "2007-2008"
# Load 2009-2010 standard biochemistry profile data
biopro_f = pd.read_sas("BIOPRO_F.xpt", format='xport')
biopro_f["cycle"] = "2009-2010"
# Load 2011-2012 standard biochemistry profile data
biopro_g = pd.read_sas("BIOPRO_G.xpt", format='xport')
biopro_g["cycle"] = "2011-2012"
# Load 2013-2014 standard biochemistry profile data
biopro_h = pd.read_sas("BIOPRO_H.xpt", format='xport')
biopro_h["cycle"] = "2013-2014"
# Load 2015-2016 standard biochemistry profile data
biopro_i = pd.read_sas("BIOPRO_I.xpt", format='xport')
biopro_i["cycle"] = "2015-2016"
# Load 2017-2018 standard biochemistry profile data
biopro_j = pd.read_sas("BIOPRO_J.xpt", format='xport')
biopro_j["cycle"] = "2007-2008"
```

Loading body measure data

```
In [ ]: # Load 2007-2008 body measure data
bmx_e = pd.read_sas("BMX_E.xpt", format='xport')
bmx_e["cycle"] = "2007-2008"
# Load 2009-2010 body measure data
bmx_f = pd.read_sas("BMX_F.xpt", format='xport')
bmx_f["cycle"] = "2009-2010"
# Load 2011-2012 body measure data
bmx_g = pd.read_sas("BMX_G.xpt", format='xport')
bmx_g["cycle"] = "2011-2012"
# Load 2013-2014 body measure data
bmx_h = pd.read_sas("BMX_H.xpt", format='xport')
bmx_h["cycle"] = "2013-2014"
# Load 2015-2016 body measure data
bmx_i = pd.read_sas("BMX_I.xpt", format='xport')
bmx_i["cycle"] = "2015-2016"
# Load 2017-2018 body measure data
bmx_j = pd.read_sas("BMX_J.xpt", format='xport')
bmx_j["cycle"] = "2017-2018"
```

Loading alcohol consumption data

```
In [ ]: # Load 2007-2008 alcohol consumption data
alq_e = pd.read_sas("ALQ_E.xpt", format='xport')
alq_e["cycle"] = "2007-2008"
# Load 2009-2010 alcohol consumption data
alq_f = pd.read_sas("ALQ_F.xpt", format='xport')
alq_f["cycle"] = "2009-2010"
# Load 2011-2012 alcohol consumption data
alq_g = pd.read_sas("ALQ_G.xpt", format='xport')
alq_g["cycle"] = "2011-2012"
# Load 2013-2014 alcohol consumption data
alq_h = pd.read_sas("ALQ_H.xpt", format='xport')
alq_h["cycle"] = "2013-2014"
# Load 2015-2016 alcohol consumption data
alq_i = pd.read_sas("ALQ_I.xpt", format='xport')
alq_i["cycle"] = "2015-2016"
# Load 2017-2018 alcohol consumption data
alq_j = pd.read_sas("ALQ_J.xpt", format='xport')
alq_j["cycle"] = "2017-2018"
```

Loading physical activity data

```
In [ ]: # Load 2007-2008 physical activity data
paq_e = pd.read_sas("PAQ_E.xpt", format='xport')
paq_e["cycle"] = "2007-2008"
# Load 2009-2010 physical activity data
paq_f = pd.read_sas("PAQ_F.xpt", format='xport')
paq_f["cycle"] = "2009-2010"
# Load 2011-2012 physical activity data
paq_g = pd.read_sas("PAQ_G.xpt", format='xport')
paq_g["cycle"] = "2011-2012"
# Load 2013-2014 physical activity data
paq_h = pd.read_sas("PAQ_H.xpt", format='xport')
paq_h["cycle"] = "2013-2014"
# Load 2015-2016 physical activity data
paq_i = pd.read_sas("PAQ_I.xpt", format='xport')
paq_i["cycle"] = "2015-2016"
# Load 2017-2018 physical activity data
paq_j = pd.read_sas("PAQ_J.xpt", format='xport')
paq_j["cycle"] = "2017-2018"
```

Loading sleeping disorders data

```
In [ ]: # Load 2007-2008 sleeping disorders data
slq_e = pd.read_sas("SLQ_E.xpt", format='xport')
slq_e["cycle"] = "2007-2008"
# Load 2009-2010 sleeping disorders data
slq_f = pd.read_sas("SLQ_F.xpt", format='xport')
slq_f["cycle"] = "2009-2010"
# Load 2011-2012 sleeping disorders data
slq_g = pd.read_sas("SLQ_G.xpt", format='xport')
slq_g["cycle"] = "2011-2012"
# Load 2013-2014 sleeping disorders data
slq_h = pd.read_sas("SLQ_H.xpt", format='xport')
slq_h["cycle"] = "2013-2014"
# Load 2015-2016 sleeping disorders data
slq_i = pd.read_sas("SLQ_I.xpt", format='xport')
slq_i["cycle"] = "2015-2016"
# Load 2017-2018 sleeping disorders data
slq_j = pd.read_sas("SLQ_J.xpt", format='xport')
slq_j["cycle"] = "2017-2018"
```

Merging of datasets

The code below merges the data from the separate data files into one dataframe for each cycle.

```
In [ ]: # Merge data for 2007 - 2008
demo_e = pd.read_sas("DEMO_E.xpt")
biopro_e = pd.read_sas("BIOPRO_E.xpt")
bmx_e = pd.read_sas("BMX_E.xpt")
alq_e = pd.read_sas("ALQ_E.xpt")
paq_e = pd.read_sas("PAQ_E.xpt")
slq_e = pd.read_sas("SLQ_E.xpt")

merged_e = demo_e.merge(biopro_e, on="SEQN", how="inner") \
    .merge(bmx_e, on="SEQN", how="inner") \
    .merge(alq_e, on="SEQN", how="inner") \
    .merge(slq_e, on="SEQN", how="inner") \
    .merge(paq_e, on="SEQN", how="inner")

# Add column to track cycle
merged_e["cycle"] = "2007-2008"
```

```
In [ ]: # Merge data for 2009-2010
demo_f = pd.read_sas("DEMO_F.xpt")
biopro_f = pd.read_sas("BIOPRO_F.xpt")
bmx_f = pd.read_sas("BMX_F.xpt")
alq_f = pd.read_sas("ALQ_F.xpt")
paq_f = pd.read_sas("PAQ_F.xpt")
slq_f = pd.read_sas("SLQ_F.xpt")

merged_f = demo_f.merge(biopro_f, on="SEQN", how="inner") \
    .merge(bmx_f, on="SEQN", how="inner") \
    .merge(alq_f, on="SEQN", how="inner") \
    .merge(slq_f, on="SEQN", how="inner") \
    .merge(paq_f, on="SEQN", how="inner")

# Add column to track cycle
merged_f["cycle"] = "2009-2010"
```

```
In [ ]: # Merge data for 2011-2012
demo_g = pd.read_sas("DEMO_G.xpt")
biopro_g = pd.read_sas("BIOPRO_G.xpt")
bmx_g = pd.read_sas("BMX_G.xpt")
alq_g = pd.read_sas("ALQ_G.xpt")
paq_g = pd.read_sas("PAQ_G.xpt")
slq_g = pd.read_sas("SLQ_G.xpt")

merged_g = demo_g.merge(biopro_g, on="SEQN", how="inner") \
    .merge(bmx_g, on="SEQN", how="inner") \
    .merge(alq_g, on="SEQN", how="inner") \
    .merge(slq_g, on="SEQN", how="inner") \
    .merge(paq_g, on="SEQN", how="inner")

# Add column to track cycle
merged_g["cycle"] = "2011-2012"
```

```
In [ ]: # Merge data for 2013-2014
demo_h = pd.read_sas("DEMO_H.xpt")
biopro_h = pd.read_sas("BIOPRO_H.xpt")
bmx_h = pd.read_sas("BMX_H.xpt")
alq_h = pd.read_sas("ALQ_H.xpt")
paq_h = pd.read_sas("PAQ_H.xpt")
slq_h = pd.read_sas("SLQ_H.xpt")

merged_h = demo_h.merge(biopro_h, on="SEQN", how="inner") \
    .merge(bmx_h, on="SEQN", how="inner") \
    .merge(alq_h, on="SEQN", how="inner") \
    .merge(slq_h, on="SEQN", how="inner") \
    .merge(paq_h, on="SEQN", how="inner")

# Add column to track cycle
merged_h["cycle"] = "2013-2014"
```

```
In [ ]: # Merge data for 2015-2016
demo_i = pd.read_sas("DEMO_I.xpt")
biopro_i = pd.read_sas("BIOPRO_I.xpt")
bmx_i = pd.read_sas("BMX_I.xpt")
alq_i = pd.read_sas("ALQ_I.xpt")
paq_i = pd.read_sas("PAQ_I.xpt")
slq_i = pd.read_sas("SLQ_I.xpt")

merged_i = demo_i.merge(biopro_i, on="SEQN", how="inner") \
    .merge(bmx_i, on="SEQN", how="inner") \
    .merge(alq_i, on="SEQN", how="inner") \
    .merge(slq_i, on="SEQN", how="inner") \
    .merge(paq_i, on="SEQN", how="inner")

# Add column to track cycle
merged_i["cycle"] = "2015-2016"
```

```
In [ ]: # Merge data for 2017-2018
demo_j = pd.read_sas("DEMO_J.xpt")
biopro_j = pd.read_sas("BIOPRO_J.xpt")
bmx_j = pd.read_sas("BMX_J.xpt")
alq_j = pd.read_sas("ALQ_J.xpt")
paq_j = pd.read_sas("PAQ_J.xpt")
slq_j = pd.read_sas("SLQ_J.xpt")

merged_j = demo_j.merge(biopro_j, on="SEQN", how="inner") \
               .merge(bmx_j, on="SEQN", how="inner") \
               .merge(alq_j, on="SEQN", how="inner") \
               .merge(slq_j, on="SEQN", how="inner") \
               .merge(paq_j, on="SEQN", how="inner")

# Add column to track cycle
merged_j["cycle"] = "2017-2018"
```

The code below appends all cycles into one dataframe to make up the full data.

```
In [ ]: # Append all cycles into one DataFrame
full_data = pd.concat([merged_e, merged_f, merged_g, merged_h, merged_i, merged_j], ignore_index=True)

# Ensure the 'cycle' column is included in the concatenated dataframe
full_data['cycle'] = pd.concat([merged_e['cycle'], merged_f['cycle'], merged_g['cycle'], merged_h['cycle'], me
```

Dealing with missing values, cleaning up code

```
In [ ]: #cleaning data to manage missing values
df_cleaned = safe_cleaning(full_data)
```

Safe Cleaning Summary:

- SLD010H: 32.64% missing
- ALQ130: 39.86% missing
- PAD615: 80.95% missing

The code below inspects the dataframe to check whether there is excessive loss of data.

```
In [ ]: # Check the shape (rows, columns)
print(df_cleaned.shape)
```

(34573, 283)

The code below checks whether any columns have been lost and that the columns to be analysed are in the dataframe.

```
In [ ]: # See columns
print(df_cleaned.columns)

Index(['SEQN', 'SDDSRVYR', 'RIDSTATR', 'RIDEXMON', 'RIAGENDR', 'RIDAGEYR',
       'RIDAGEMN', 'RIDAGEEX', 'RIDRETH1', 'DMQMILIT',
       ...,
       'ALQ142', 'ALQ270', 'ALQ280', 'ALQ290', 'ALQ170', 'SLQ320', 'SLQ330',
       'SLD013', 'ALQ130_CAT', 'PAD615_CAT'],
      dtype='object', length=283)
```

The code below checks for the data sample size before filtering, which is 34573.

```
In [ ]: df_cleaned.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 34573 entries, 0 to 34572
Columns: 283 entries, SEQN to PAD615_CAT
dtypes: float64(276), object(7)
memory usage: 74.6+ MB
```

Inclusion criteria

- Hispanic
- Male
- Age is between 40 - 60

Filtering

The code below filters the full dataframe by gender (Male), ethnicity (Mexican American and other Hispanic) and age (40-60).

```
In [ ]: # Filter by gender (RIAGENDR), ethnicity (RIDRETH1), and age (RIDAGEYR)
filteredfd = df_cleaned[
    (df_cleaned['RIAGENDR'] == 1) & # 1 = Male
    (df_cleaned['RIDRETH1'].isin([1, 2])) & # 1 = Mexican American, 2 = Other Hispanic
    (df_cleaned['RIDAGEYR'].between(40, 60))
]
```

The code below checks for the filtered data sample size, which is 1468.

```
In [ ]: filteredfd.info()

<class 'pandas.core.frame.DataFrame'>
Index: 1468 entries, 2 to 34570
Columns: 283 entries, SEQN to PAD615_CAT
dtypes: float64(276), object(7)
memory usage: 3.2+ MB
```

The code below filters the data again to contain only the variables focused on for this analysis.

```
In [ ]: #filter data to only contain independent and dependent variables we are focusing on
df = filteredfd[['LBXSATSI', # ALT
                 'LBXSASSI', # AST
                 'LBXSGTSI', # GGT
                 'BMXBMI', # BMI
                 'ALQ130', # Alcohol drinks/day
                 'SLD010H', # Sleep duration (hours)
                 'PAD615', 'RIAGENDR', 'RIDRETH1', 'RIDAGEYR'
                ]] # Physical activity (minutes/week)
```

The code below checks for the percentage of blank data for each variable.

```
In [ ]: #percentage of data that is missing
df.isnull().mean()
```

```
Out[ ]:

```

LBXSATSI	0.042484
LBXSASSI	0.042484
LBXSGTSI	0.042484
BMXBMI	0.008987
ALQ130	0.315359
SLD010H	0.000000
PAD615	0.693627
RIAGENDR	0.000000
RIDRETH1	0.000000
RIDAGEYR	0.000000

dtype: float64

The code below checks for the number of blank data for each variable.

```
In [ ]: #number of data that is missing
df.isnull().sum()
```

```
Out[ ]:
```

LBXSATSI	57
LBXSASSI	57
LBXSGTSI	57
BMXBMI	12
ALQ130	470
SLD010H	430
PAD615	1009
RIAGENDR	0
RIDRETH1	0
RIDAGEYR	0
GENDER_LABEL	0
ETHNICITY_LABEL	0
AGE_GROUP	0
AGE_GROUP_CUSTOM	0

dtype: int64

The code below generates a summary table indicating the number of valid and invalid data points, as well as the percentage of missing data for each variable.

```
In [ ]: #Sample size for each variable, indicating the number of valid and invalid data points.
# List of key variables
variables = ['LBXSATSI', # ALT
            'LBXSASSI', # AST
            'LBXSGTSI', # GGT
            'BMXBMI', # BMI
            'ALQ130', # Alcohol intake
            'SLD010H', # Sleep duration
            'PAD615', # Physical activity
            'RIAGENDR', # Gender
            'RIDRETH1', # Ethnicity
            'RIDAGEYR'] # Age

# Create a summary table
summary = []

for var in variables:
    if var in df.columns:
        total = len(df)
        valid = df[var].notnull().sum()
        missing = df[var].isnull().sum()
        percent_missing = round((missing / total) * 100, 2)
        summary.append({
            'Variable': var,
            'Valid (non-missing)': valid,
            'Missing (NaN)': missing,
            'Missing (%)': percent_missing
        })
    else:
        summary.append({
            'Variable': var,
            'Valid (non-missing)': '✖ Not Found',
            'Missing (NaN)': '✖',
            'Missing (%)': '✖'
        })

# Convert to a DataFrame and display
sample_size_df = pd.DataFrame(summary)
sample_size_df
```

Out[]:

	Variable	Valid (non-missing)	Missing (NaN)	Missing (%)
0	LBXSATSI	1411	57	3.88
1	LBXSASSI	1411	57	3.88
2	LBXSGTSI	1411	57	3.88
3	BMXBMI	1456	12	0.82
4	ALQ130	998	470	32.02
5	SLD010H	1038	430	29.29
6	PAD615	459	1009	68.73
7	RIAGENDR	1468	0	0.00
8	RIDRETH1	1468	0	0.00
9	RIDAGEYR	1468	0	0.00

How we dealt with missing values

For dependent variables (ALT, AST, GGT) it is not advised to impute missing values with mean or median. Hence, as the missing percentage is <5%, the missing values are dropped.

As for alcohol consumption and physical activity and sleep duration, the percentage of missing values is >30%, thus it the values cannot be imputed as it would risk biasness. Thus, it is treated as categorical data, and missing values are assigned "missing" and sorted into categories by the cleaning code defined at the top.

The categories are shown below.

```
In [ ]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Ensure column names are uppercase
df_cleaned.columns = df_cleaned.columns.str.upper()

# Include sleep duration category in the plot
cat_vars = {
    'ALQ130_CAT': 'Alcohol Intake Category',
    'PAD615_CAT': 'Physical Activity Category',
    'SLEEP_CAT': 'Sleep Duration Category'
}

# Loop through each categorical variable
for var, title in cat_vars.items():
    if var in df_cleaned.columns:
        # Create summary table
        summary = df_cleaned[var].value_counts(dropna=False).reset_index()
        summary.columns = ['Category', 'Count']
        summary['Percent'] = round((summary['Count'] / len(df_cleaned)) * 100, 2)

        print(f"\n Summary Table: {title}")
        display(summary)

        # Plot bar chart
        plt.figure(figsize=(8, 5))
        ax = sns.barplot(data=summary, x='Category', y='Count', palette='Set2')

        # Add Labels
        for container in ax.containers:
            ax.bar_label(container, label_type='edge', fontsize=10)

        plt.title(f'{title} Distribution')
        plt.xlabel('')
        plt.ylabel('Number of Participants')
        plt.xticks(rotation=30)
        plt.tight_layout()
        plt.show()
    else:
        print(f"Variable '{var}' not found in df_cleaned.")
```

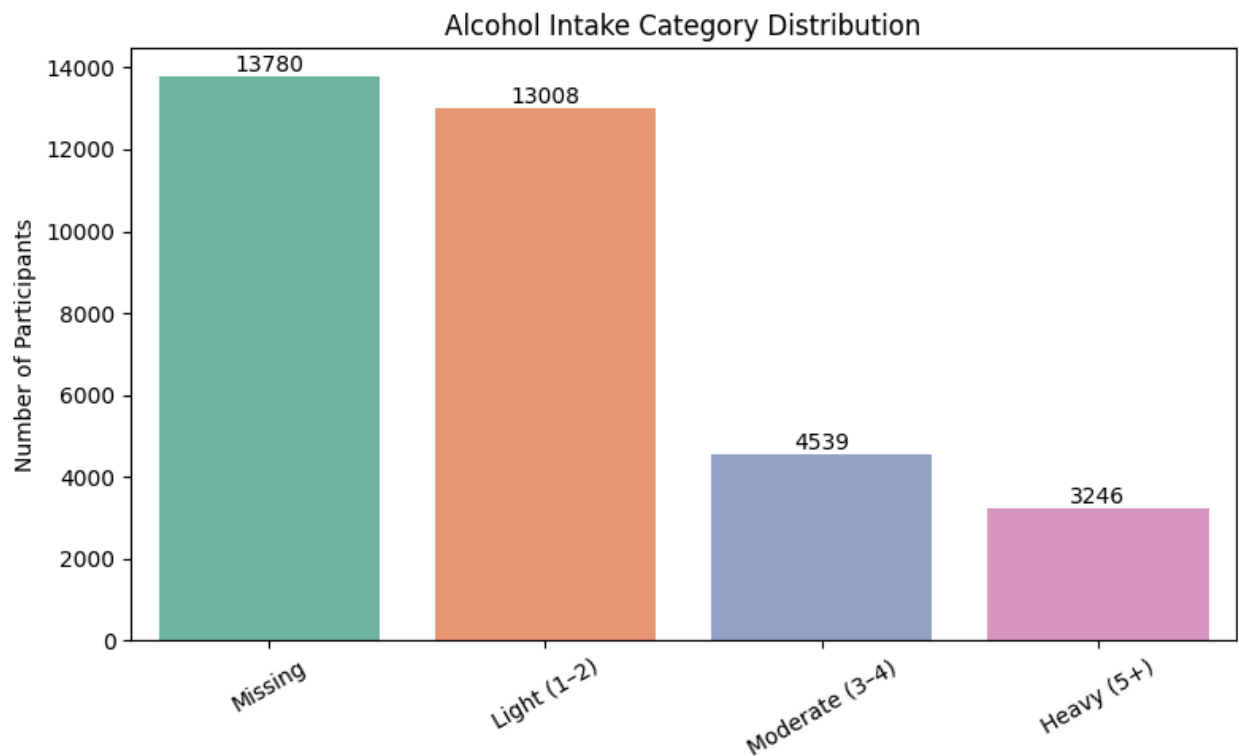
Summary Table: Alcohol Intake Category

	Category	Count	Percent
0	Missing	13780	39.86
1	Light (1-2)	13008	37.62
2	Moderate (3-4)	4539	13.13
3	Heavy (5+)	3246	9.39

/tmp/ipython-input-54-3064145113.py:28: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
ax = sns.barplot(data=summary, x='Category', y='Count', palette='Set2')
```



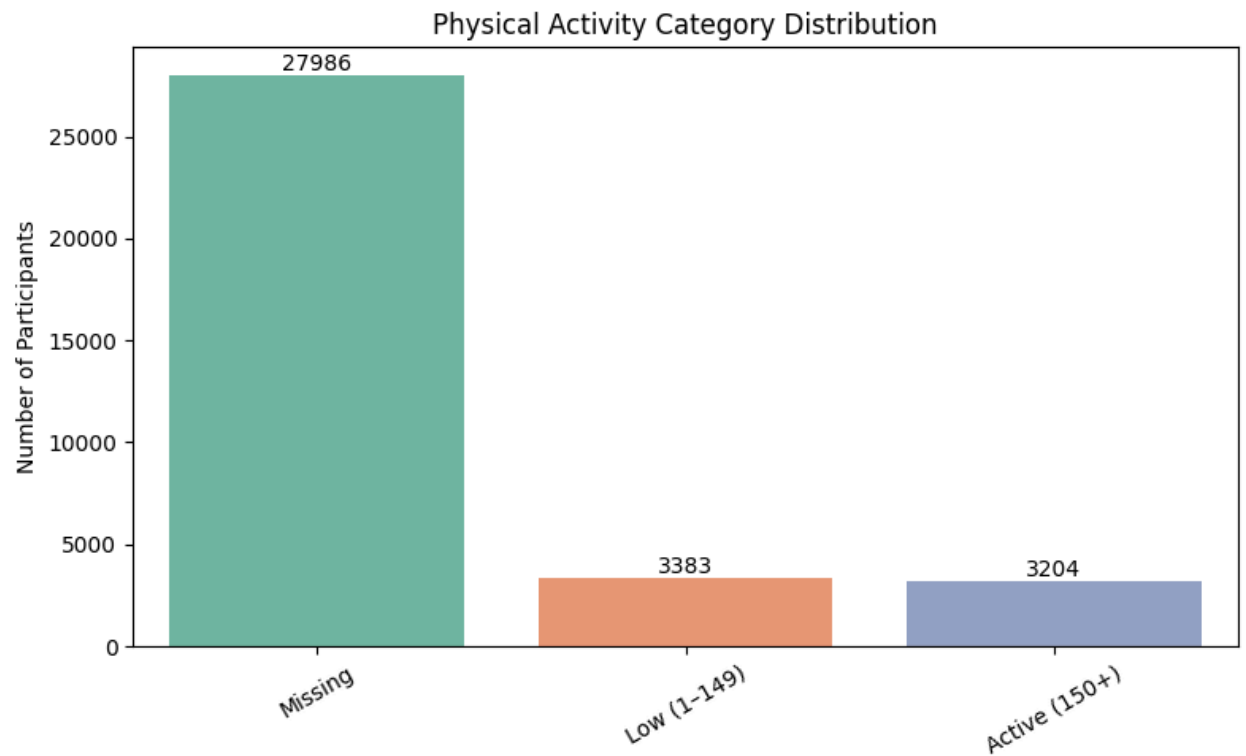
Summary Table: Physical Activity Category

	Category	Count	Percent
0	Missing	27986	80.95
1	Low (1-149)	3383	9.79
2	Active (150+)	3204	9.27

/tmp/ipython-input-54-3064145113.py:28: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
ax = sns.barplot(data=summary, x='Category', y='Count', palette='Set2')
```

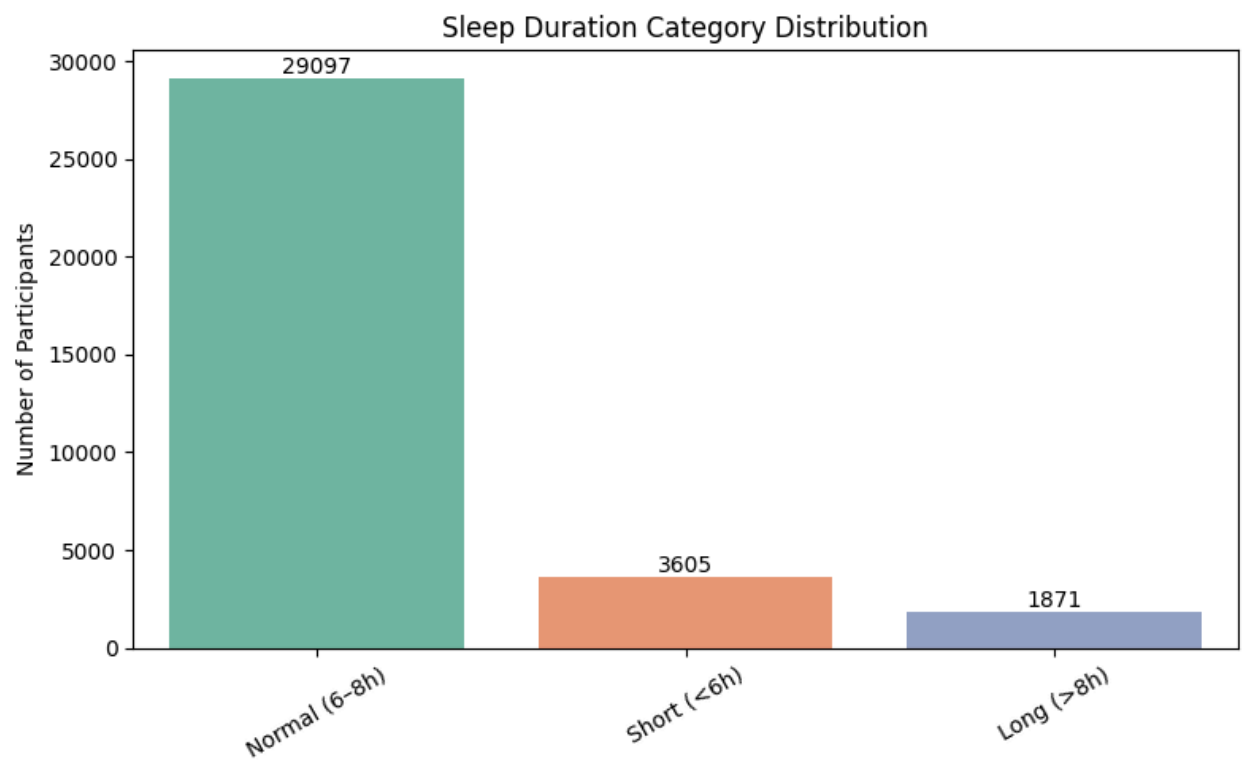
Summary Table: Sleep Duration Category

	Category	Count	Percent
0	Normal (6-8h)	29097	84.16
1	Short (<6h)	3605	10.43
2	Long (>8h)	1871	5.41

```
/tmp/ipython-input-54-3064145113.py:28: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.
```

```
ax = sns.barplot(data=summary, x='Category', y='Count', palette='Set2')
```



```
In [ ]: import seaborn as sns
import matplotlib.pyplot as plt

# Count number of participants in each sleep category
sleep_counts = df_cleaned['SLEEP_CAT'].value_counts().sort_index().reset_index()
sleep_counts.columns = ['Sleep Category', 'Count']

# Plot
plt.figure(figsize=(7, 5))
sns.barplot(data=sleep_counts, x='Sleep Category', y='Count', palette='Set2')

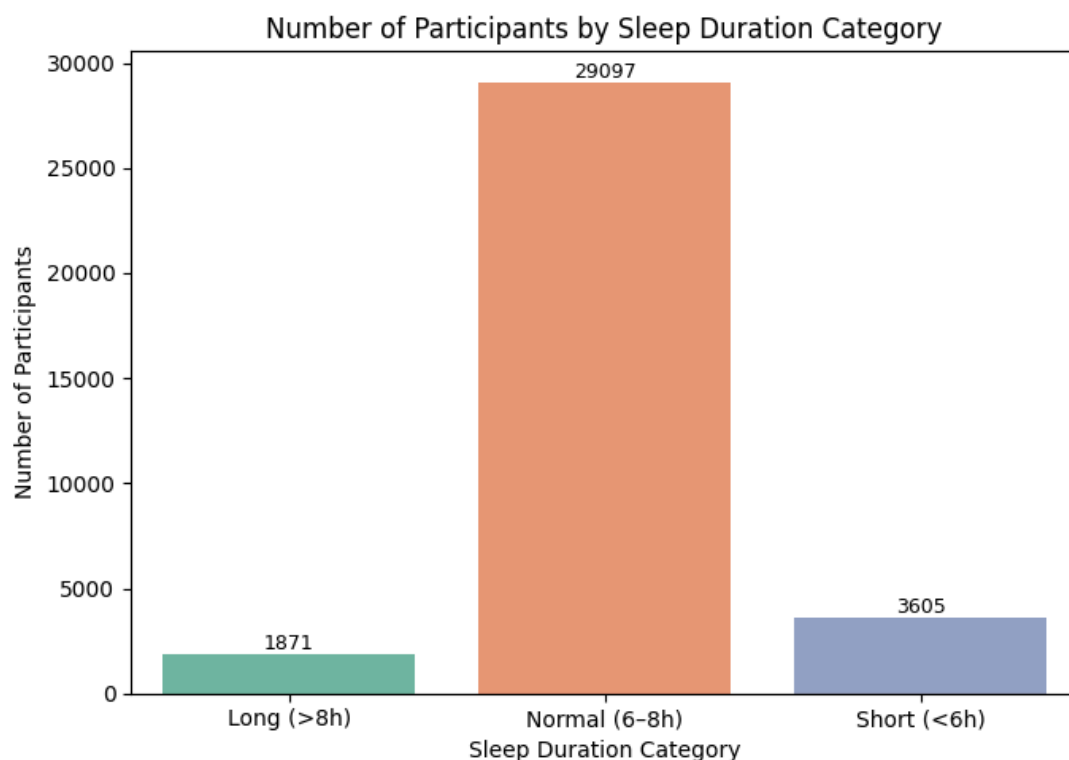
# Annotate each bar with the count
for i, row in sleep_counts.iterrows():
    plt.text(i, row['Count'] + 2, f'{row["Count"]}', ha='center', va='bottom', fontsize=9)

plt.title('Number of Participants by Sleep Duration Category')
plt.xlabel('Sleep Duration Category')
plt.ylabel('Number of Participants')
plt.tight_layout()
plt.show()
```

/tmp/ipython-input-53-1967427925.py:10: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=sleep_counts, x='Sleep Category', y='Count', palette='Set2')
```



7. Dataset Exploration

The code below generates a summary table for the descriptive statistics for each variable (mean, median, standard deviation, first and third quartile, and skewness).

Descriptive statistics

The code below generates a summary table for the descriptive statistics for each variable (mean, median, standard deviation, first and third quartile, and skewness).

Continuous variable

```
In [ ]: #Descriptive statistics for each variable
#continuous variables

import pandas as pd
import numpy as np
from scipy.stats import skew

# List of continuous variables to analyse
continuous_vars = ['LBXSATSI', # ALT
                  'LBXSASSI', # AST
                  'LBXSGTSI', # GGT
                  'BMXBMI', # BMI
                  'ALQ130', # Alcohol drinks/day
                  'SLD010H', # Sleep duration (hours)
                  'PAD615'] # Physical activity (minutes/week)

# Initialize results dictionary
descriptive_stats = {}

# Loop through each variable and calculate stats
for var in continuous_vars:
    data = full_data[var].dropna()
    descriptive_stats[var] = {
        'mean': round(data.mean(), 2),
        'median': round(data.median(), 2),
        'std_dev': round(data.std(), 2),
        'Q1 (25%)': round(data.quantile(0.25), 2),
        'Q3 (75%)': round(data.quantile(0.75), 2),
        'skewness': round(skew(data), 2)
    }

# Convert to DataFrame for display
desc_df = pd.DataFrame(descriptive_stats).T
desc_df.index.name = "Variable"
desc_df.columns = ['Mean', 'Median', 'Standard Deviation', 'Q1 (25%)', 'Q3 (75%)', 'Skewness']

# Display the table
desc_df
```

Out[]:

	Mean	Median	Standard Deviation	Q1 (25%)	Q3 (75%)	Skewness
--	------	--------	--------------------	----------	----------	----------

Variable						
LBXSATSI	24.85	20.0	20.74	16.0	28.0	16.52
LBXSASSI	25.33	22.0	17.35	19.0	27.0	20.36
LBXSGTSI	29.36	20.0	42.54	14.0	30.0	12.10
BMXBMI	29.12	28.0	7.04	24.2	32.6	1.27
ALQ130	4.26	2.0	37.79	1.0	3.0	25.79
SLD010H	6.93	7.0	3.12	6.0	8.0	23.14
PAD615	210.02	120.0	492.53	60.0	240.0	17.90

The code below generates a table showing the numbers and percentages of participants in each category for the categorical variables.

Categorical variable

```
In [ ]: # visualisation of catergorical data: including numbers and frequency of participants in each category, and nu

# Ensure column names are uppercase
df.columns = df.columns.str.upper()

# 1. Define readable mappings
gender_map = {
    1: "Male",
    2: "Female",
    7: "Don't Know",
    9: "Refused"
}

ethnicity_map = {
    1: "Mexican American",
    2: "Other Hispanic",
    3: "Non-Hispanic White",
    4: "Non-Hispanic Black",
    5: "Other/Multi-Racial"
}

def age_group(age):
    if pd.isna(age):
        return 'Missing'
    elif age < 40:
        return '<40'
    elif 40 <= age <= 60:
        return '40-60'
    elif age > 60:
        return '>60'
    else:
        return 'Other'

# 2. Apply Label transformations
df['GENDER_LABEL'] = df['RIAGENDR'].map(gender_map)
df['ETHNICITY_LABEL'] = df['RIDRETH1'].map(ethnicity_map)
df['AGE_GROUP'] = df['RIDAGEYR'].apply(age_group)

# 3. Define variables to summarize
label_vars = {
    'GENDER_LABEL': 'Gender',
    'ETHNICITY_LABEL': 'Ethnicity',
    'AGE_GROUP': 'Age Group'
}

# 4. Build summary table
summary_list = []

for var, var_name in label_vars.items():
    if var in df.columns:
        value_counts = df[var].value_counts(dropna=False).reset_index()
        value_counts.columns = ['Category', 'Count']
        value_counts['Percent'] = round((value_counts['Count'] / len(df)) * 100, 2)
        value_counts['Variable'] = var_name
        summary_list.append(value_counts)
    else:
        print(f"⚠️ Column {var} not found.")

# 5. Combine all into one table
summary_table = pd.concat(summary_list, ignore_index=True)

# 6. Reorder columns
summary_table = summary_table[['Variable', 'Category', 'Count', 'Percent']]

# 7. Display final combined table
print("📄 Combined Summary Table for Gender, Ethnicity, and Age Group")
display(summary_table)
```

📄 Combined Summary Table for Gender, Ethnicity, and Age Group

```
/tmp/ipython-input-32-3599210253.py:35: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df['GENDER_LABEL'] = df['RIAGENDR'].map(gender_map)
/tmp/ipython-input-32-3599210253.py:36: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df['ETHNICITY_LABEL'] = df['RIDRETH1'].map(ethnicity_map)
/tmp/ipython-input-32-3599210253.py:37: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df['AGE_GROUP'] = df['RIDAGEYR'].apply(age_group)
```

	Variable	Category	Count	Percent
0	Gender	Male	1468	100.00
1	Ethnicity	Mexican American	891	60.69
2	Ethnicity	Other Hispanic	577	39.31
3	Age Group	40–60	1468	100.00

Assessing normality of data via Sharpiro-Wilk test, plotting histograms and QQ plots

The code below assesses the normality of the data via the Shapiro-Wilk test and visualises the data through plotting of histograms and Q-Q plots

Visualizations:

Numerical variables: Histograms (adjust the number of bins to improve visualization) and QQ plots (remember to relate to the results of the Shapiro Wilk test)

Categorical variables: Labelled bar charts showing the number of persons in each categories

Grouped bar charts or heat maps, showing the effect of variables on the dependent variables.

```

In [ ]: #Numerical values: run Shapiro Wilk test + plot Histograms and QQ plots
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import scipy.stats as stats

# Make sure column names are uppercase
df.columns = df.columns.str.upper()

# Define the continuous variables and labels
continuous_vars = {
    'LBXSATSI': 'ALT (U/L)',
    'LBXSASSI': 'AST (U/L)',
    'LBXSGTSI': 'GGT (U/L)',
    'ALQ130': 'Alcohol Intake (Drinks/Day)',
    'SLD010H': 'Sleep Duration (Hours)',
    'PAD615': 'Physical Activity (Minutes/Week)',
    'BMXBMI': 'BMI'
}

# Loop through each variable
for var, label in continuous_vars.items():
    if var in df.columns:
        # Remove NaNs
        data = df[var].dropna()

        # Custom filter for ALQ130 and SLD010H to remove dont know/ refused codes
        if var == 'ALQ130':
            data = data[~data.isin([777, 999])] # remove Don't Know / Refused
        elif var == 'SLD010H':
            data = data[~data.isin([77, 99])]

        # Shapiro-Wilk test
        if len(data) >= 3 and len(data) <= 5000:
            stat, p = stats.shapiro(data)
            result = f"✅ p = {p:.4f} {'(normal)' if p > 0.05 else '(not normal)'}"
        else:
            result = "⚠️ Sample size too large for Shapiro-Wilk (limited to ≤5000 in SciPy)"

        print(f"\n📊 Shapiro-Wilk Test for {label} [{var}]: {result}")

        # Plot histogram and QQ plot
        fig, axs = plt.subplots(1, 2, figsize=(12, 4))

        # Histogram
        sns.histplot(data, kde=True, ax=axs[0], color='skyblue')
        axs[0].set_title(f'{label} - Histogram')
        axs[0].set_xlabel(label)

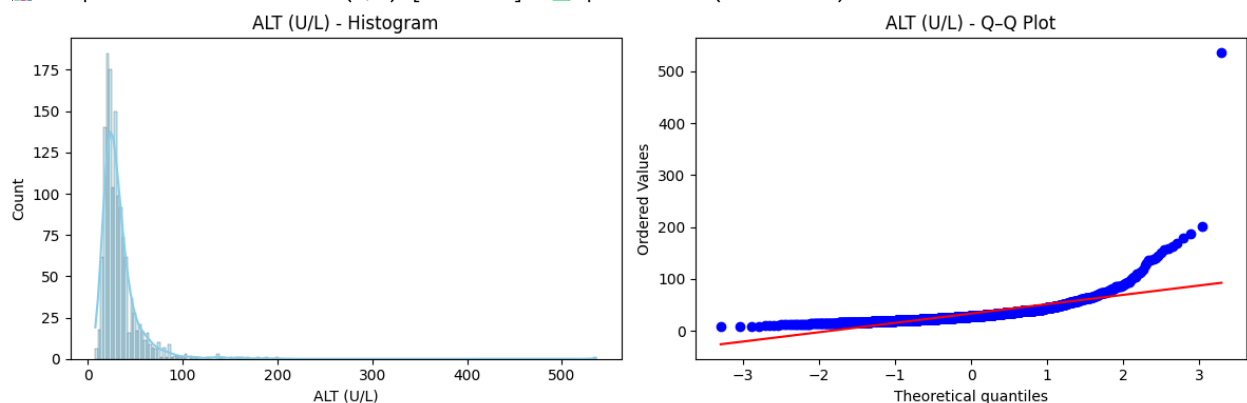
        # Q-Q Plot
        stats.probplot(data, dist="norm", plot=axs[1])
        axs[1].set_title(f'{label} - Q-Q Plot')

        plt.tight_layout()
        plt.show()

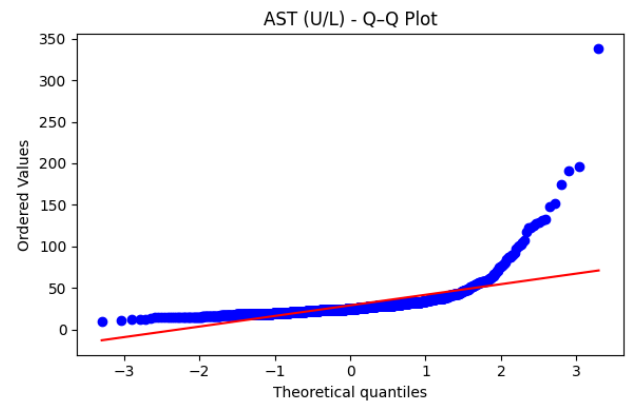
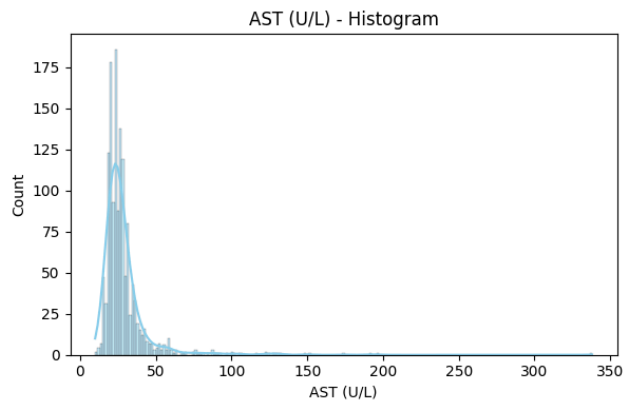
    else:
        print(f"⚠️ Variable {var} not found in the dataset.")

```

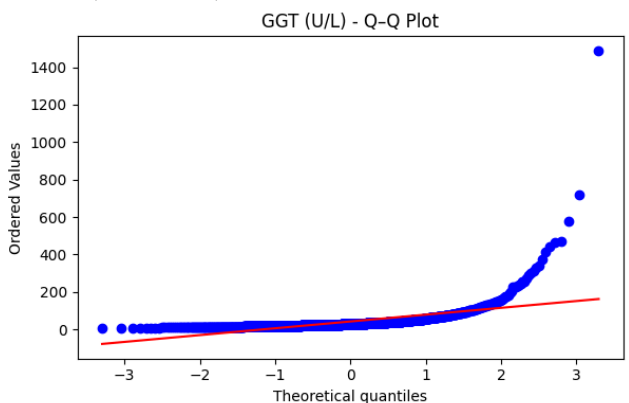
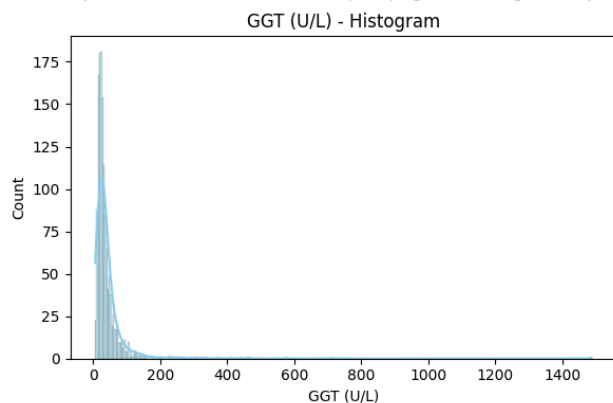
📊 Shapiro-Wilk Test for ALT (U/L) [LBXSATSI]: ✅ p = 0.0000 (not normal)



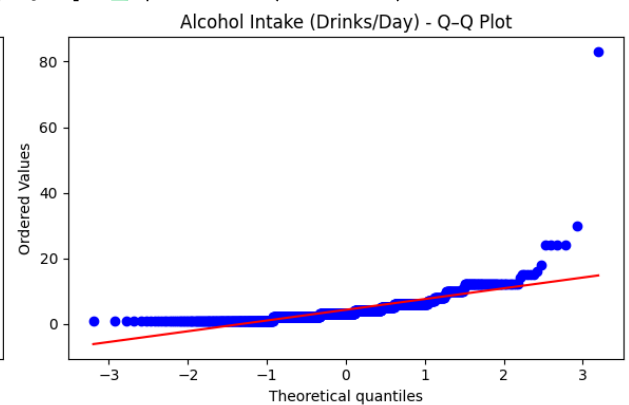
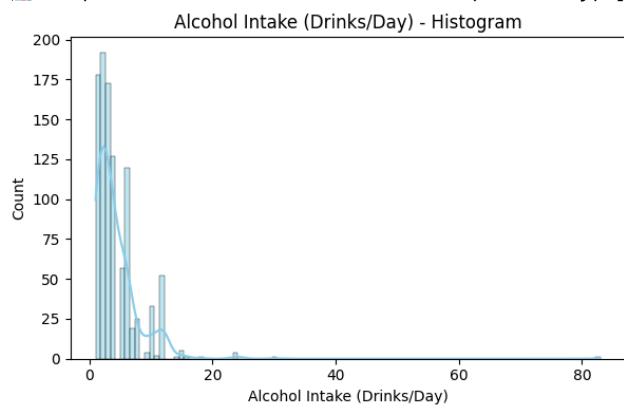
📊 Shapiro-Wilk Test for AST (U/L) [LBXSASSI]: ✅ p = 0.0000 (not normal)



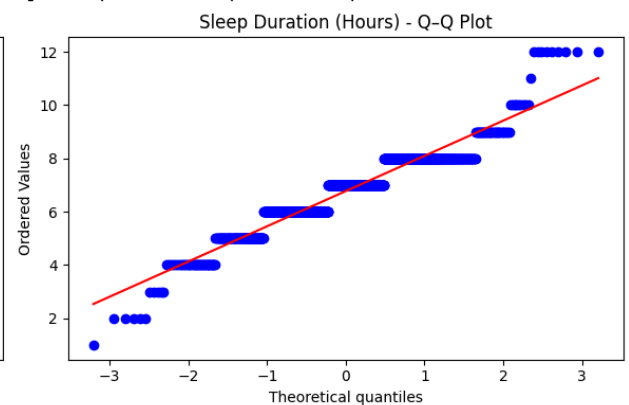
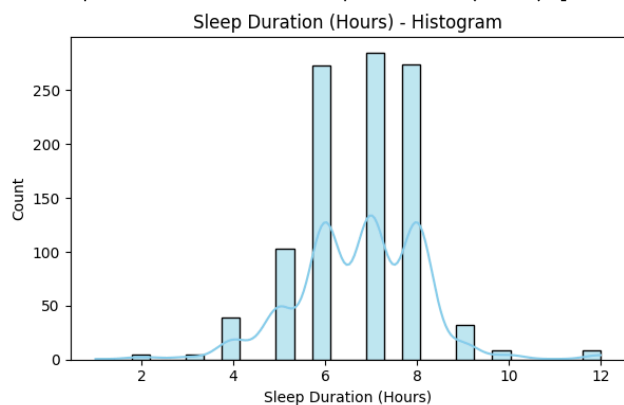
Shapiro-Wilk Test for GGT (U/L) [LBXSGTSI]: $\checkmark$ $p = 0.0000$ (not normal)



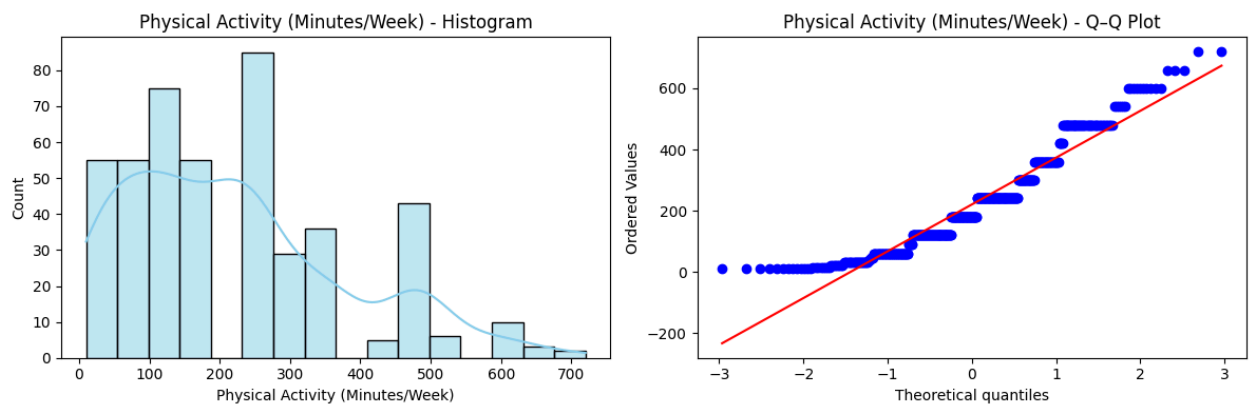
Shapiro-Wilk Test for Alcohol Intake (Drinks/Day) [ALQ130]: $\checkmark$ $p = 0.0000$ (not normal)



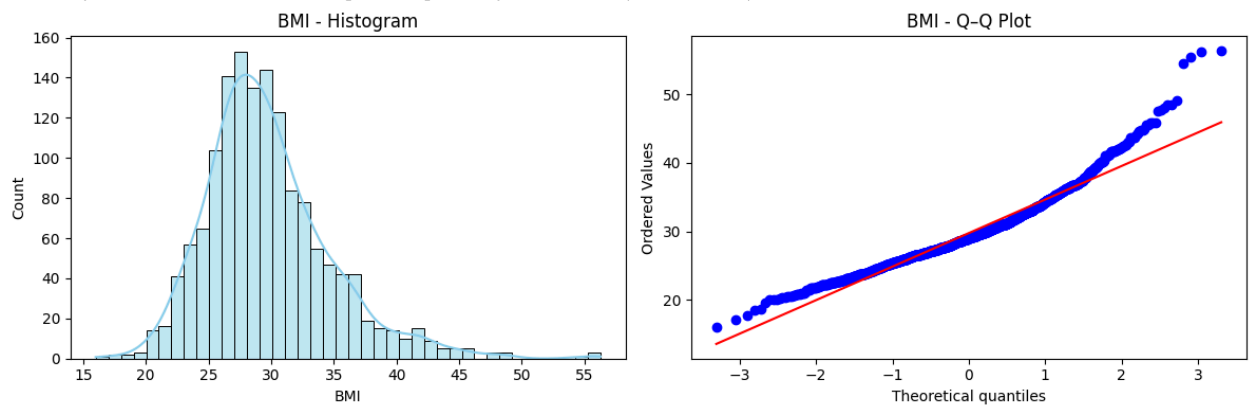
Shapiro-Wilk Test for Sleep Duration (Hours) [SLD010H]: $\checkmark$ $p = 0.0000$ (not normal)



Shapiro-Wilk Test for Physical Activity (Minutes/Week) [PAD615]: $\checkmark$ $p = 0.0000$ (not normal)



Shapiro-Wilk Test for BMI [BMXBMI]: $\checkmark$ $p = 0.0000$ (not normal)



Variable	Shapiro-Wilk Test	Histogram	Q-Q Plot	Normality
ALT (LBXSAT)	$p = 0.0000$	Asymmetrical, skews to the left	S-curve	Not normal
AST (LBXAST)	$p = 0.0000$	Asymmetrical, skews to the left	S-curve	Not normal
GGT (LBXSGT)	$p = 0.0000$	Asymmetrical, skews to the left	S-curve	Not normal
BMI	$p = 0.0000$	Asymmetrical, skews to the left	S-curve	Not normal
Alcohol Intake	$p = 0.0000$	Asymmetrical, skews to the left	S-curve	Not normal
Sleep Duration	$p = 0.0000$	Asymmetrical, skews to the right	S-curve	Not normal
Physical Activity	$p = 0.0000$	Asymmetrical, skews to the left	S-curve	Not normal

Bar chart plotting for categorical variables

The code below plots bar charts according to the number of people in each category for the categorical variables (gender, ethnicity, age).

In []: *#Categorical variables: Labelled bar charts showing the number of persons in each categories*

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Ensure column names are uppercase
df.columns = df.columns.str.upper()

# 1. Label maps
gender_map = {
    1: "Male",
    2: "Female",
    7: "Don't Know",
    9: "Refused"
}

ethnicity_map = {
    1: "Mexican American",
    2: "Other Hispanic",
    3: "Non-Hispanic White",
    4: "Non-Hispanic Black",
    5: "Other/Multi-Racial"
}

# 2. Apply maps
df['GENDER_LABEL'] = df['RIAGENDR'].map(gender_map)
df['ETHNICITY_LABEL'] = df['RIDRETH1'].map(ethnicity_map)

# 3. Custom Age Grouping (you specified these bins)
age_bins = [-1, 40, 42, 44, 46, 48, 50, 52, 54, 56, 58, 60]
age_labels = ['40', '42', '44', '46', '48', '50', '52', '54', '56', '58', '60']
df['AGE_GROUP_CUSTOM'] = pd.cut(df['RIDAGEYR'], bins=age_bins, labels=age_labels)

# 4. Variables to plot
categorical_vars = {
    'GENDER_LABEL': 'Gender',
    'ETHNICITY_LABEL': 'Ethnicity',
    'AGE_GROUP_CUSTOM': 'Age Group (Custom)'
}

# 5. Plot bar charts with labels
for var, title in categorical_vars.items():
    if var in df.columns:
        counts = df[var].value_counts(dropna=False).sort_index().reset_index()
        counts.columns = ['Category', 'Count']

        # Plot
        plt.figure(figsize=(8, 5))
        ax = sns.barplot(data=counts, x='Category', y='Count', palette='Set2')

        # Add value labels on top of bars
        for i in ax.containers:
            ax.bar_label(i, label_type='edge', fontsize=10)

        plt.title(f'{title} Distribution with Counts')
        plt.ylabel('Number of Participants')
        plt.xlabel(title)
        plt.xticks(rotation=30)
        plt.tight_layout()
        plt.show()
    else:
        print(f"🚩 Column '{var}' not found.")
```

```
/tmp/ipython-input-34-3545338463.py:26: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df['GENDER_LABEL'] = df['RIAGENDR'].map(gender_map)
/tmp/ipython-input-34-3545338463.py:27: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

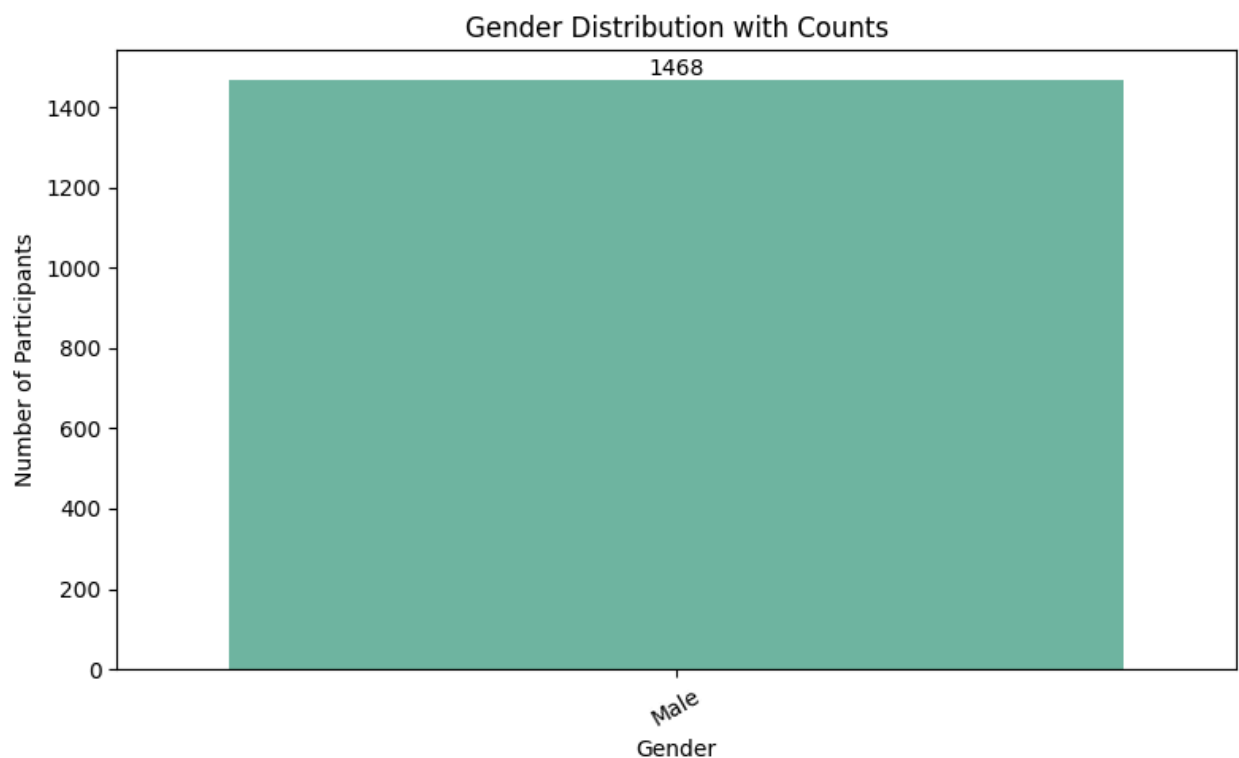
```
df['ETHNICITY_LABEL'] = df['RIDRETH1'].map(ethnicity_map)
/tmp/ipython-input-34-3545338463.py:32: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df['AGE_GROUP_CUSTOM'] = pd.cut(df['RIDAGEYR'], bins=age_bins, labels=age_labels)
/tmp/ipython-input-34-3545338463.py:49: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

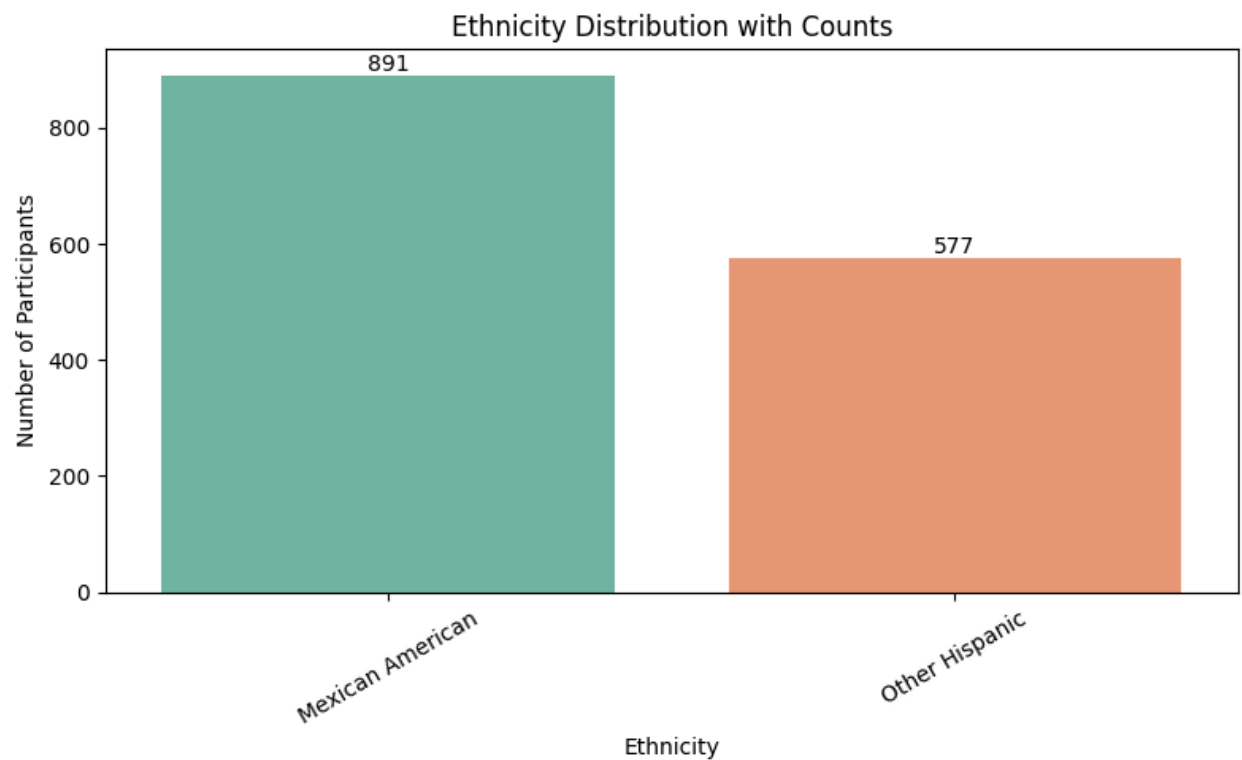
```
ax = sns.barplot(data=counts, x='Category', y='Count', palette='Set2')
```



```
/tmp/ipython-input-34-3545338463.py:49: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

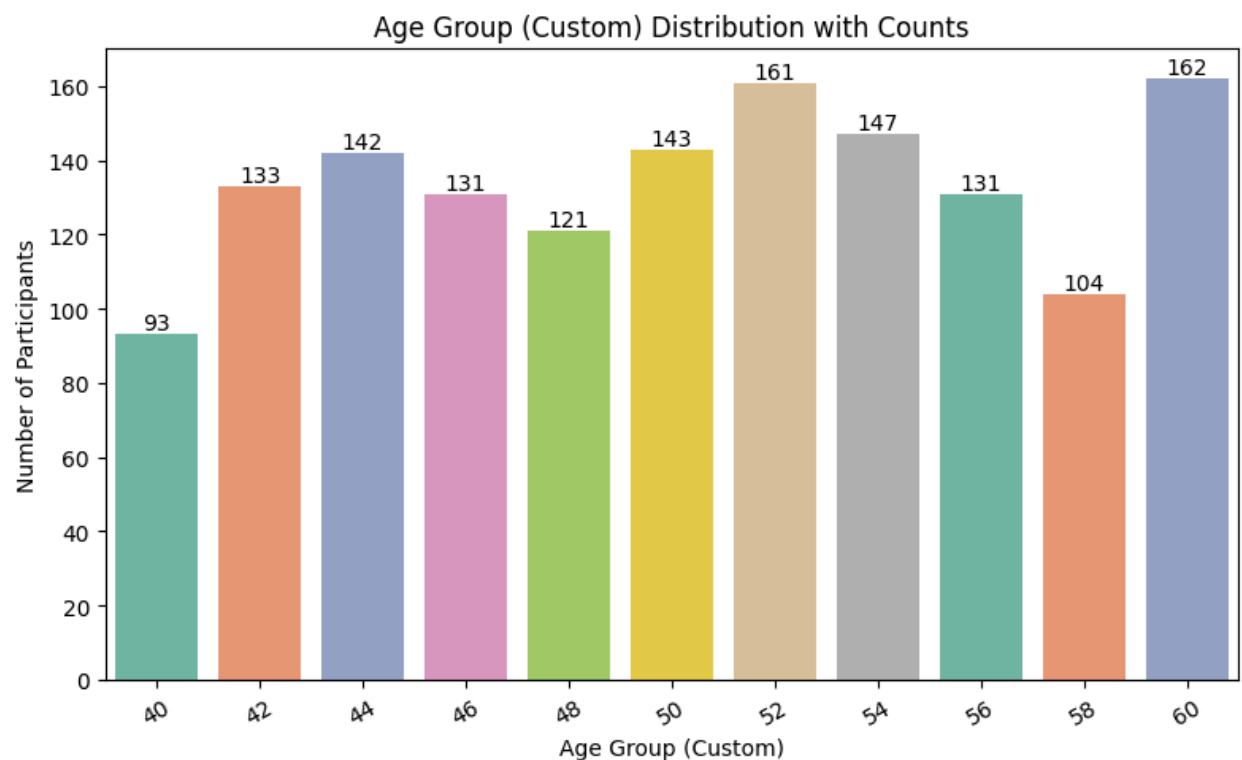
```
ax = sns.barplot(data=counts, x='Category', y='Count', palette='Set2')
```



/tmp/ipython-input-34-3545338463.py:49: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
ax = sns.barplot(data=counts, x='Category', y='Count', palette='Set2')
```



Grouped bar chart plotting for visualisation of mean liver enzyme levels for each categorical variable

The code below generates grouped bar charts to visualise the mean liver enzyme levels (ALT, AST, GGT) for each categorical variable.

Grouped Bar Charts –

1. Mean Liver Enzyme Levels by Categorical Variable

This chart shows the average ALT/AST/GGT per category (e.g., per age group or gender).

```
In [ ]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Ensure column names are uppercase
df.columns = df.columns.str.upper()

# Define dependent variables (enzymes)
enzymes = ['LBXSATSI', 'LBXSASSI', 'LBXSGTSI']
enzyme_labels = ['ALT (U/L)', 'AST (U/L)', 'GGT (U/L)']

# Independent categorical variables
group_vars = {
    'GENDER_LABEL': 'Gender',
    'ETHNICITY_LABEL': 'Ethnicity',
    'AGE_GROUP_CUSTOM': 'Age Group (Custom)'
}

# Loop through grouping variables
for group_var, group_label in group_vars.items():
    if group_var in df.columns:
        # Drop missing data
        grouped = df[[group_var] + enzymes].dropna()

        # Melt for seaborn
        melted = pd.melt(grouped, id_vars=group_var, value_vars=enzymes,
                        var_name='Enzyme', value_name='Value')

        # Rename enzyme codes to readable labels
        enzyme_map = dict(zip(enzymes, enzyme_labels))
        melted['Enzyme'] = melted['Enzyme'].map(enzyme_map)

        # Create plot
        plt.figure(figsize=(10, 6))
        ax = sns.barplot(
            data=melted,
            x=group_var,
            y='Value',
            hue='Enzyme',
            ci='sd',
            palette='Set2'
        )

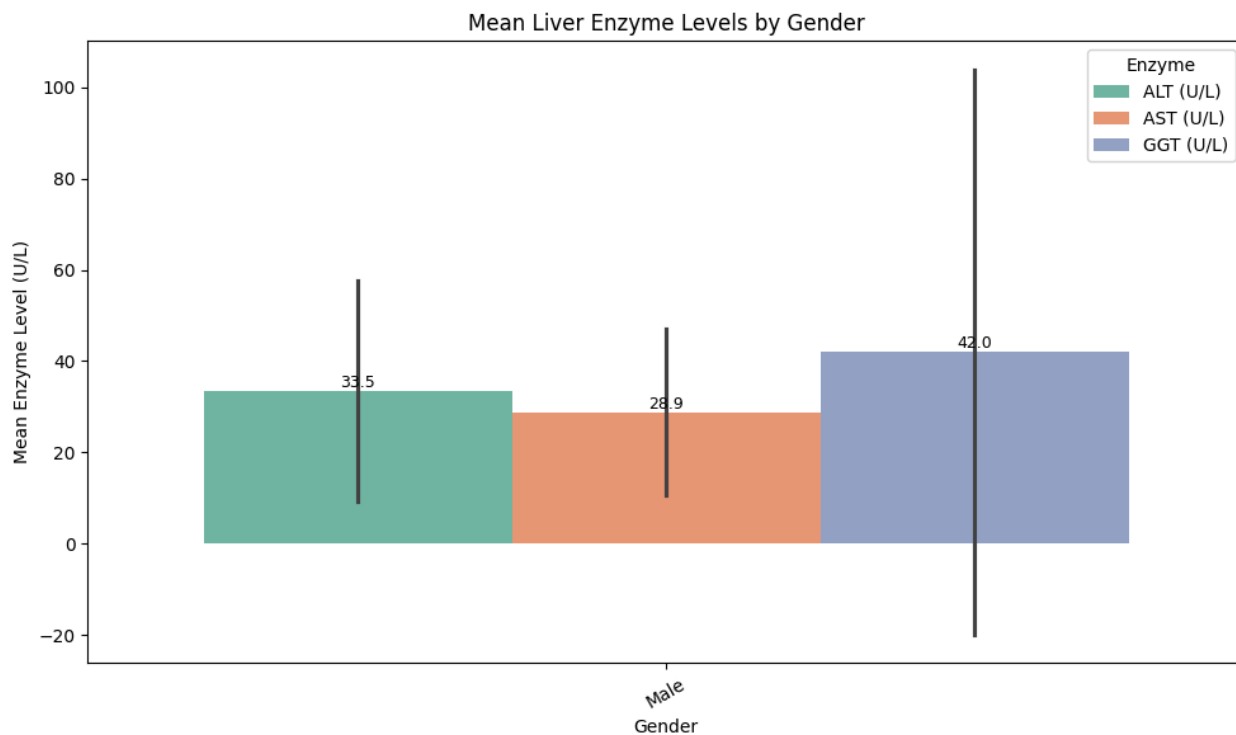
        # Add value labels on top of bars
        for container in ax.containers:
            ax.bar_label(container, fmt='%.1f', label_type='edge', fontsize=9)

        # Titles and labels
        plt.title(f'Mean Liver Enzyme Levels by {group_label}')
        plt.ylabel('Mean Enzyme Level (U/L)')
        plt.xlabel(group_label)
        plt.xticks(rotation=30)
        plt.legend(title='Enzyme')
        plt.tight_layout()
        plt.show()
```

/tmp/ipython-input-35-1722630987.py:35: FutureWarning:

The `ci` parameter is deprecated. Use `errorbar='sd'` for the same effect.

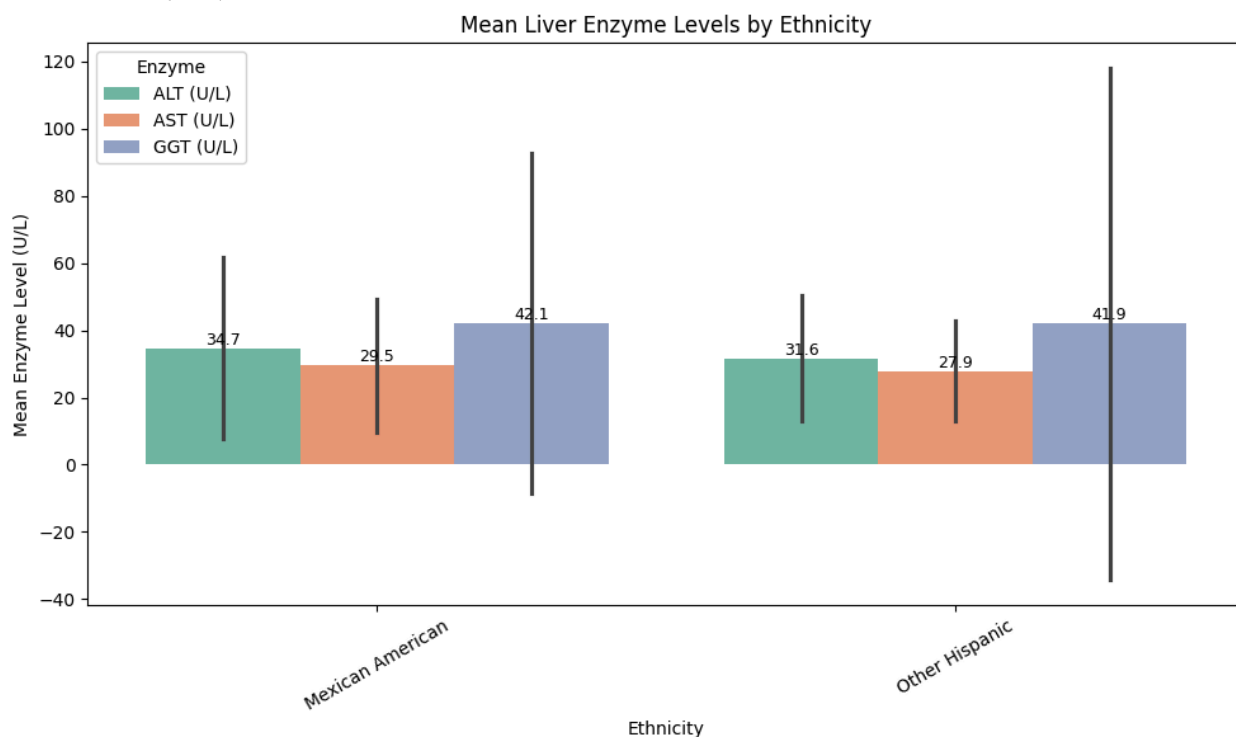
```
ax = sns.barplot(
```



/tmp/ipython-input-35-1722630987.py:35: FutureWarning:

The `ci` parameter is deprecated. Use `errorbar='sd'` for the same effect.

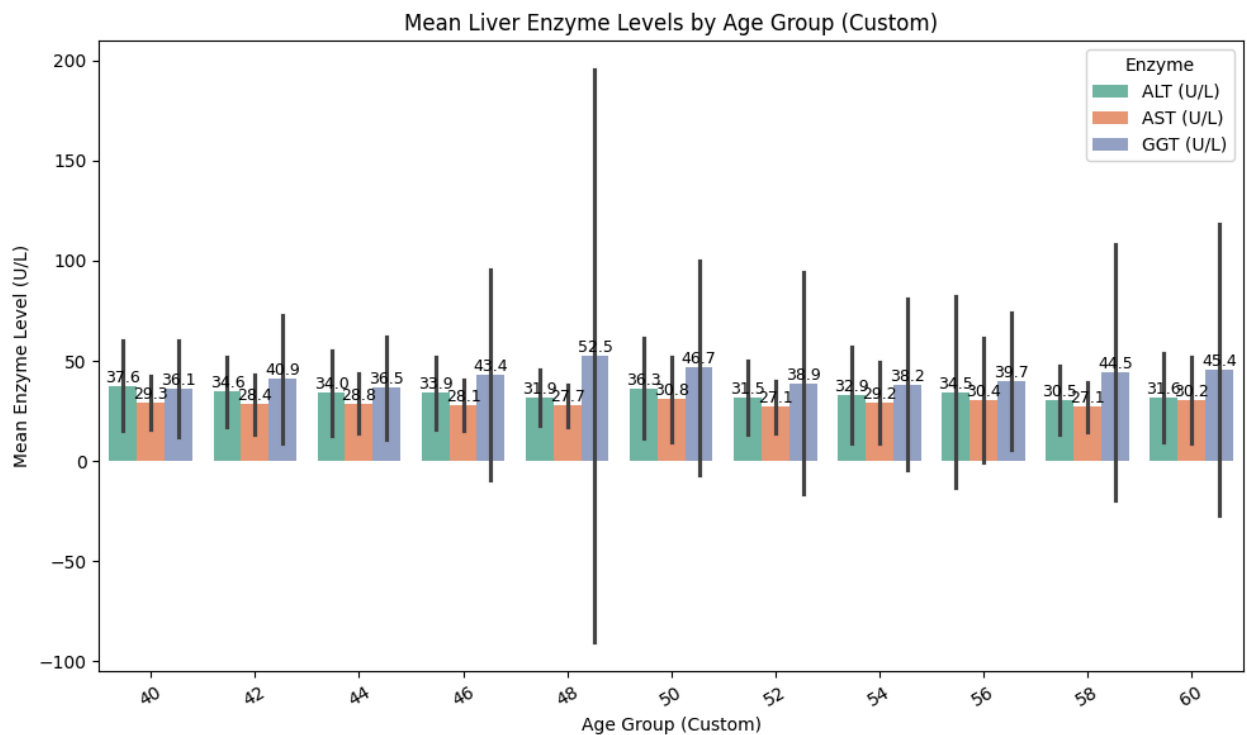
```
ax = sns.barplot(
```



/tmp/ipython-input-35-1722630987.py:35: FutureWarning:

The `ci` parameter is deprecated. Use `errorbar='sd'` for the same effect.

```
ax = sns.barplot(
```



Analysis of Bar Charts

According to the bar charts, GGT is the most inconsistent in gender, ethnicity and age as the values vary significantly and the error bars are large. Conversely, AST is the most consistent with the smallest variation in values and the smallest error bars. This follows the descriptive statistics that was calculated above.

Heatmap plotting for median levels of each liver enzyme by age group and gender

The code below generates a heatmap for the median levels of each liver enzyme (ALT, AST, GGT) by gender.

```
In [ ]: #heatmap!
# Create a pivot table for the heatmap
pivot_table = df.pivot_table(
    values=['LBXSATSI', 'LBXSASSI', 'LBXSGTSI'],
    index='AGE_GROUP_CUSTOM',
    columns='GENDER_LABEL',
    aggfunc='median'
)

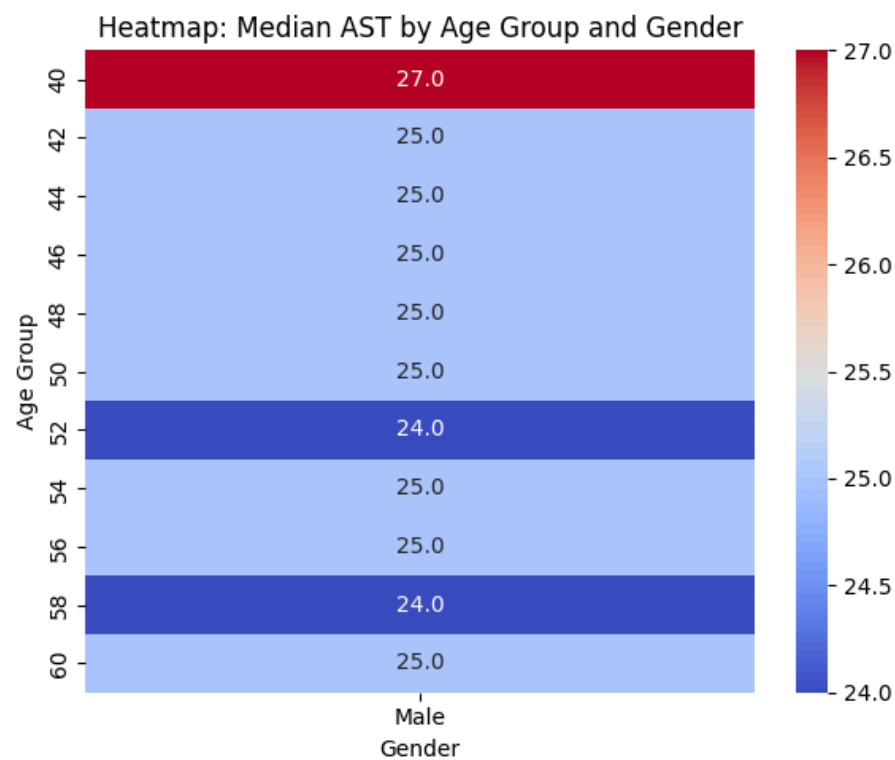
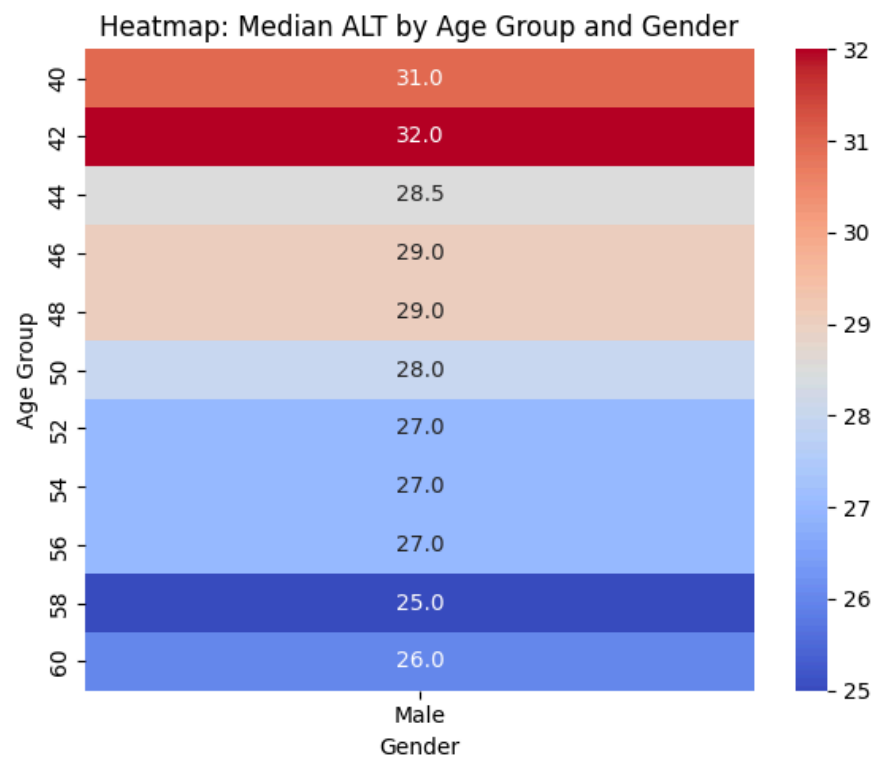
# Rename for clarity (optional but good practice)
pivot_table.columns.names = ['Enzyme', 'Gender']

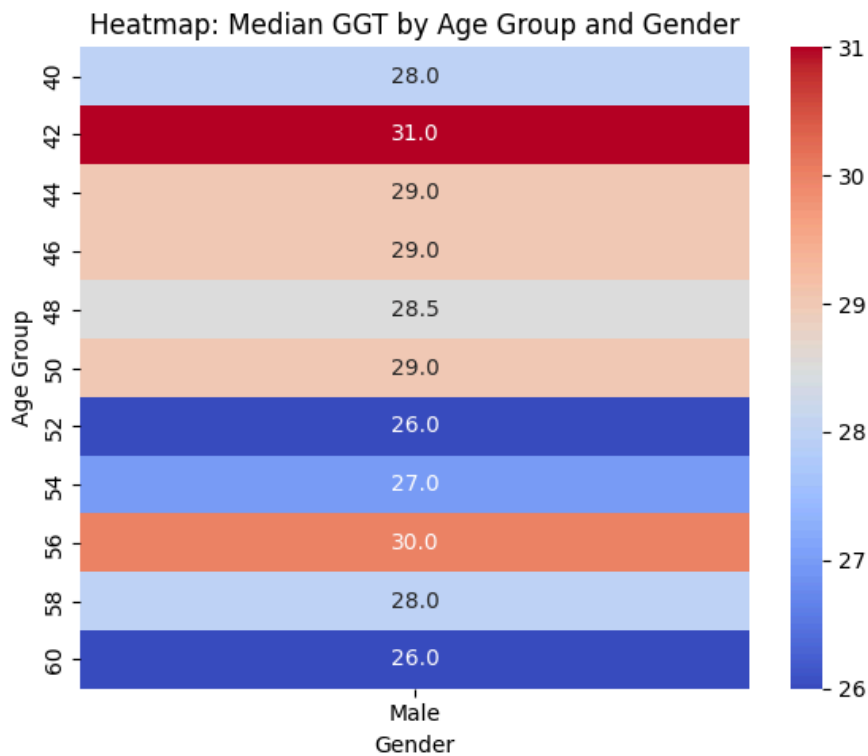
# Define enzymes and their labels for titles
enzymes = ['LBXSATSI', 'LBXSASSI', 'LBXSGTSI']
enzyme_titles = {'LBXSATSI': 'ALT', 'LBXSASSI': 'AST', 'LBXSGTSI': 'GGT'}

# Plot heatmap for each enzyme separately
for enzyme in enzymes:
    # Access the data for the current enzyme from the pivot table columns
    if (enzyme, 'Male') in pivot_table.columns or (enzyme, 'Female') in pivot_table.columns:
        plt.figure(figsize=(6, 5))
        sns.heatmap(pivot_table[enzyme], annot=True, fmt=".1f", cmap="coolwarm")
        plt.title(f'Heatmap: Median {enzyme_titles[enzyme]} by Age Group and Gender')
        plt.ylabel('Age Group')
        plt.xlabel('Gender')
        plt.tight_layout()
        plt.show()
    else:
        print(f"⚠️ Data for enzyme '{enzyme_titles[enzyme]}' not found in the pivot table.")
```

/tmp/ipython-input-36-3162931704.py:3: FutureWarning: The default value of observed=False is deprecated and will change to observed=True in a future version of pandas. Specify observed=False to silence this warning and retain the current behavior

```
    pivot_table = df.pivot_table(
```





Analysis of Heatmaps

For the ALT heatmap, a general decreasing trend on ALT levels can be observed where the younger age groups have higher ALT levels, closer to the maximum level of 40 U/L, which may put them at higher risk. (Assessing liver function and interpreting liver blood tests, 2024)

For the AST heatmap, most age groups are fairly consistent at the 24.0 U/L to 25.0 U/L range, with the exception of the 40-42 age group with a higher level of 27.0 U/L. The median levels shown are well within the normal range of AST levels of 0-40 U/L.

For the GGT heatmap, no trend is observed and levels range from 26.0 U/L to 31.0 U/L which are well within the normal range of GGT levels of 0-50 U/L. The age group 42 has the highest levels of GGT at 31.0 U/L while the age groups 52 and 60 has the lowest levels of GGT at 26.0 U/L. (Assessing liver function and interpreting liver blood tests, 2024)

Box plot of biomarkers against age, gender and ethnicity.

```
In [ ]: import seaborn as sns
import matplotlib.pyplot as plt

df.columns = df.columns.str.upper()

enzyme_vars = {
    'LBXSATSI': 'ALT (U/L)',
    'LBXSASSI': 'AST (U/L)',
    'LBXSGTSI': 'GGT (U/L)'
}

grouping_vars = {
    'AGE_GROUP_CUSTOM': 'Age Group',
    'ETHNICITY_LABEL': 'Ethnicity',
    'GENDER_LABEL': 'Gender'
}

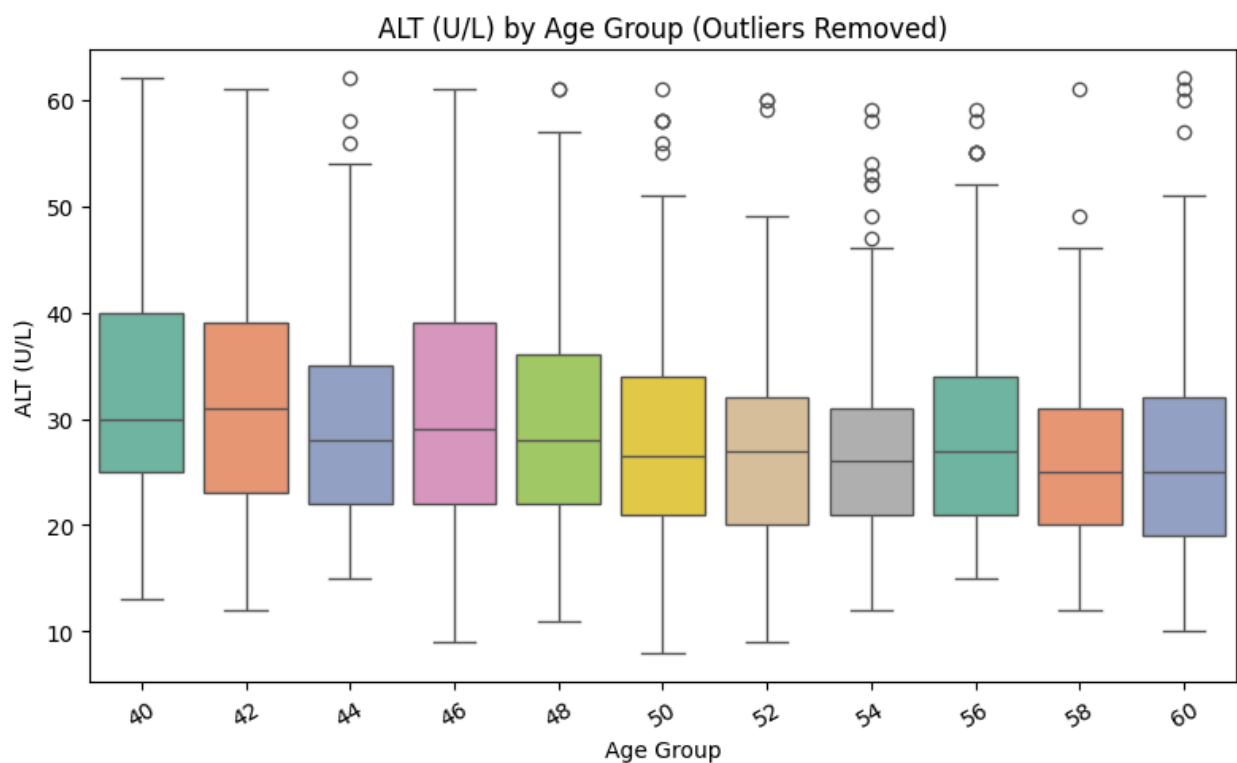
for group_var, group_label in grouping_vars.items():
    if group_var in df.columns:
        for biomarker, biomarker_label in enzyme_vars.items():
            if biomarker in df.columns:
                # Calculate IQR and remove outliers
                Q1 = df[biomarker].quantile(0.25)
                Q3 = df[biomarker].quantile(0.75)
                IQR = Q3 - Q1
                filtered_df = df[(df[biomarker] >= Q1 - 1.5 * IQR) & (df[biomarker] <= Q3 + 1.5 * IQR)]

                plt.figure(figsize=(8, 5))
                sns.boxplot(data=filtered_df, x=group_var, y=biomarker, palette='Set2')
                plt.title(f'{biomarker_label} by {group_label} (Outliers Removed)')
                plt.xlabel(group_label)
                plt.ylabel(biomarker_label)
                plt.xticks(rotation=30)
                plt.tight_layout()
                plt.show()
```

/tmp/ipython-input-37-183678998.py:29: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

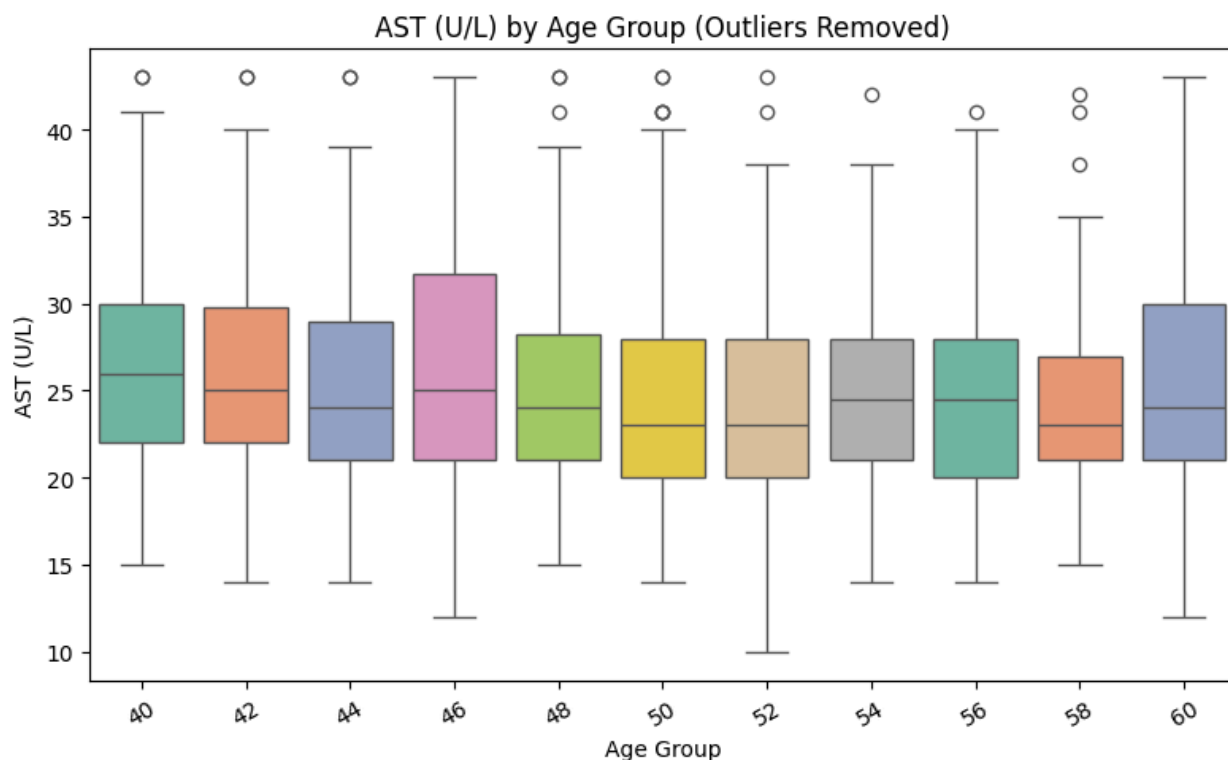
```
sns.boxplot(data=filtered_df, x=group_var, y=biomarker, palette='Set2')
```



```
/tmp/ipython-input-37-183678998.py:29: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

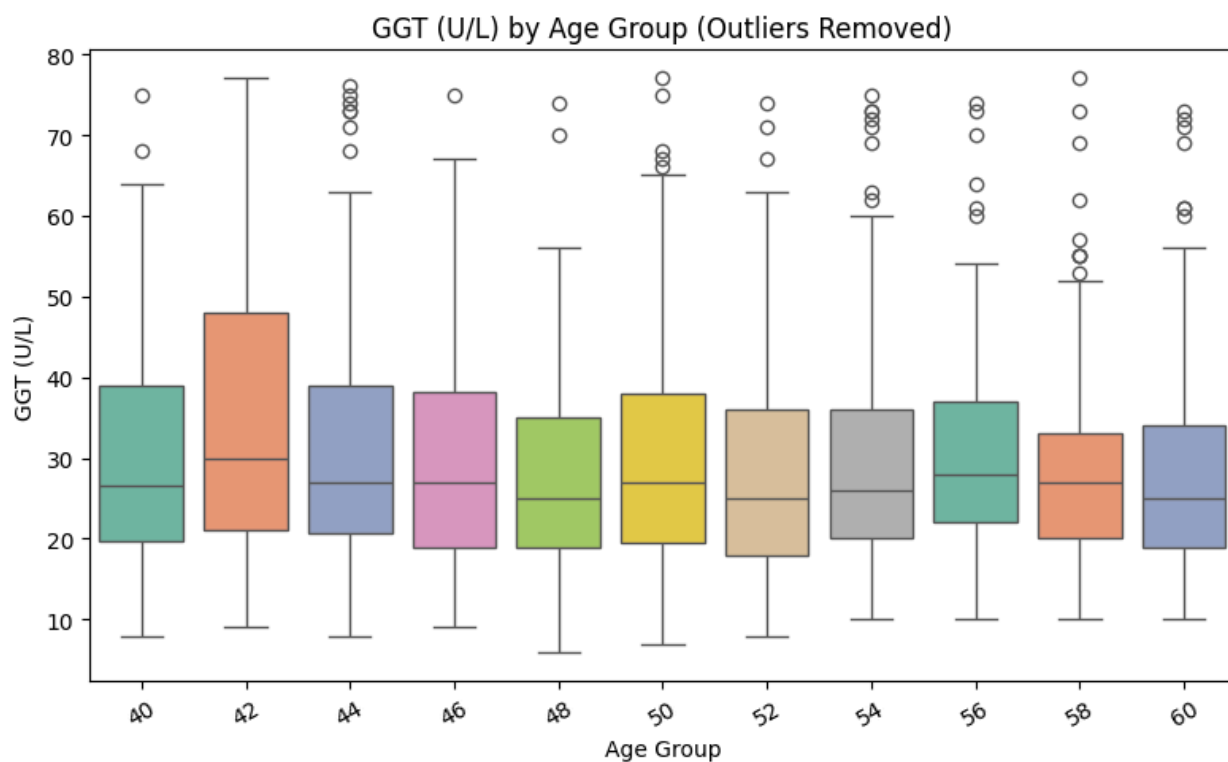
```
sns.boxplot(data=filtered_df, x=group_var, y=biomarker, palette='Set2')
```



```
/tmp/ipython-input-37-183678998.py:29: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

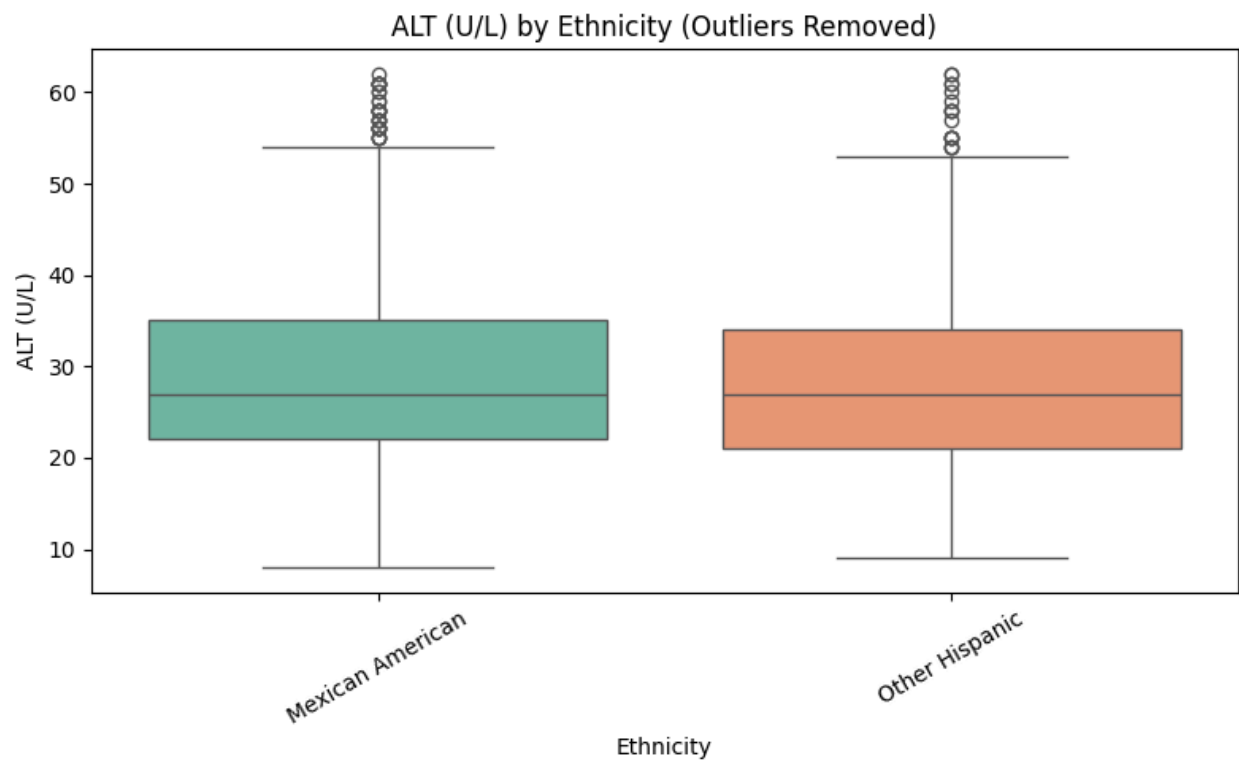
```
sns.boxplot(data=filtered_df, x=group_var, y=biomarker, palette='Set2')
```



```
/tmp/ipython-input-37-183678998.py:29: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

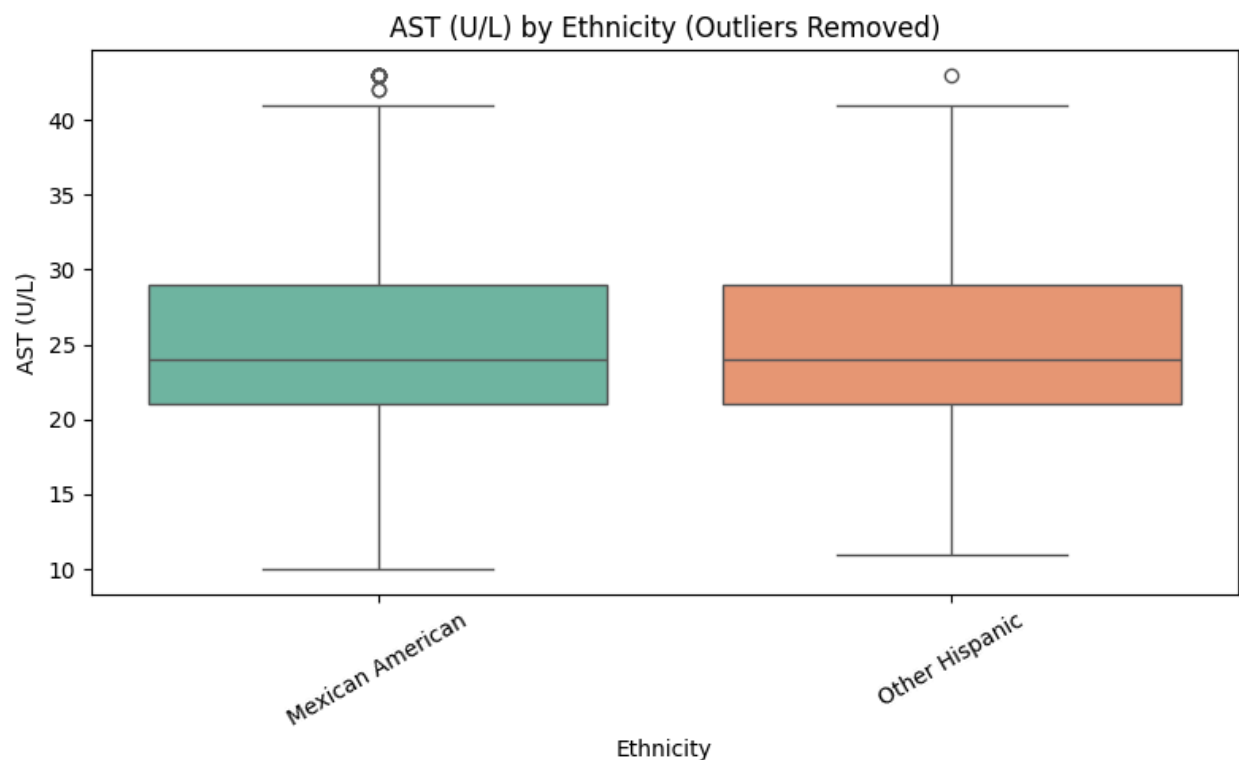
```
sns.boxplot(data=filtered_df, x=group_var, y=biomarker, palette='Set2')
```



/tmp/ipython-input-37-183678998.py:29: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

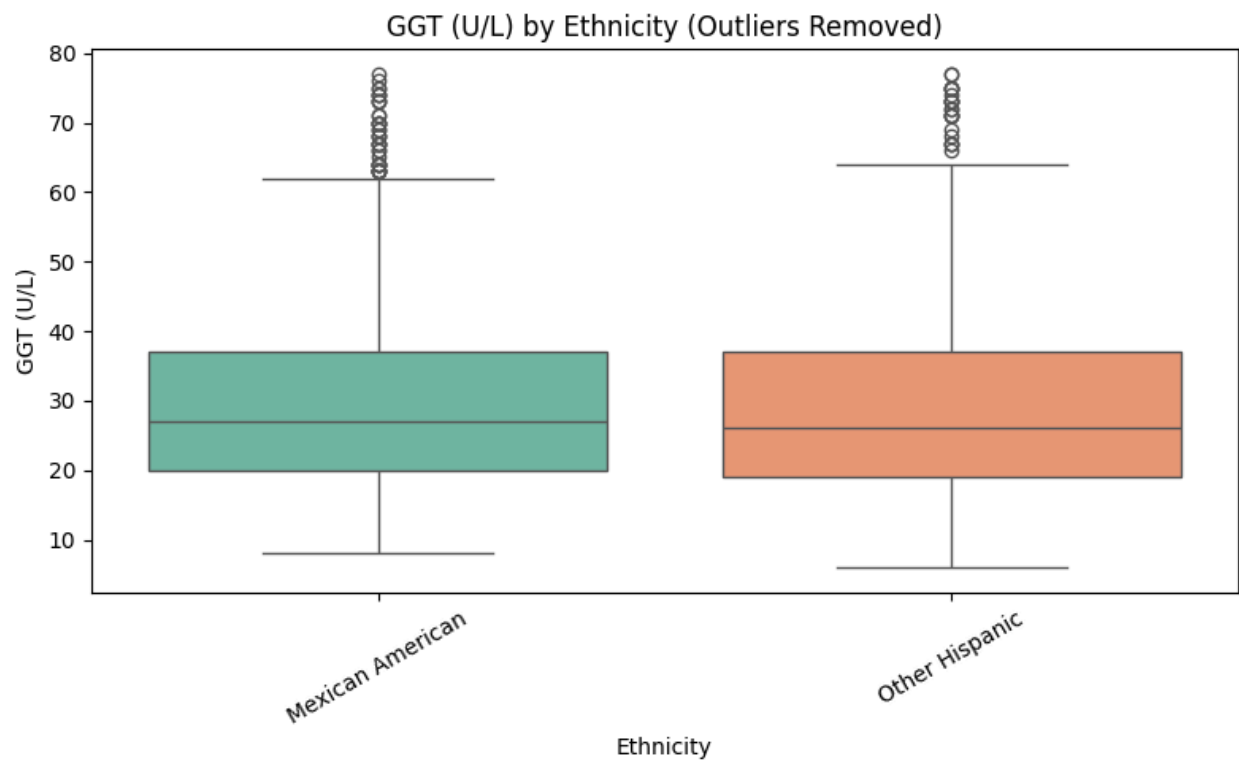
```
sns.boxplot(data=filtered_df, x=group_var, y=biomarker, palette='Set2')
```



/tmp/ipython-input-37-183678998.py:29: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

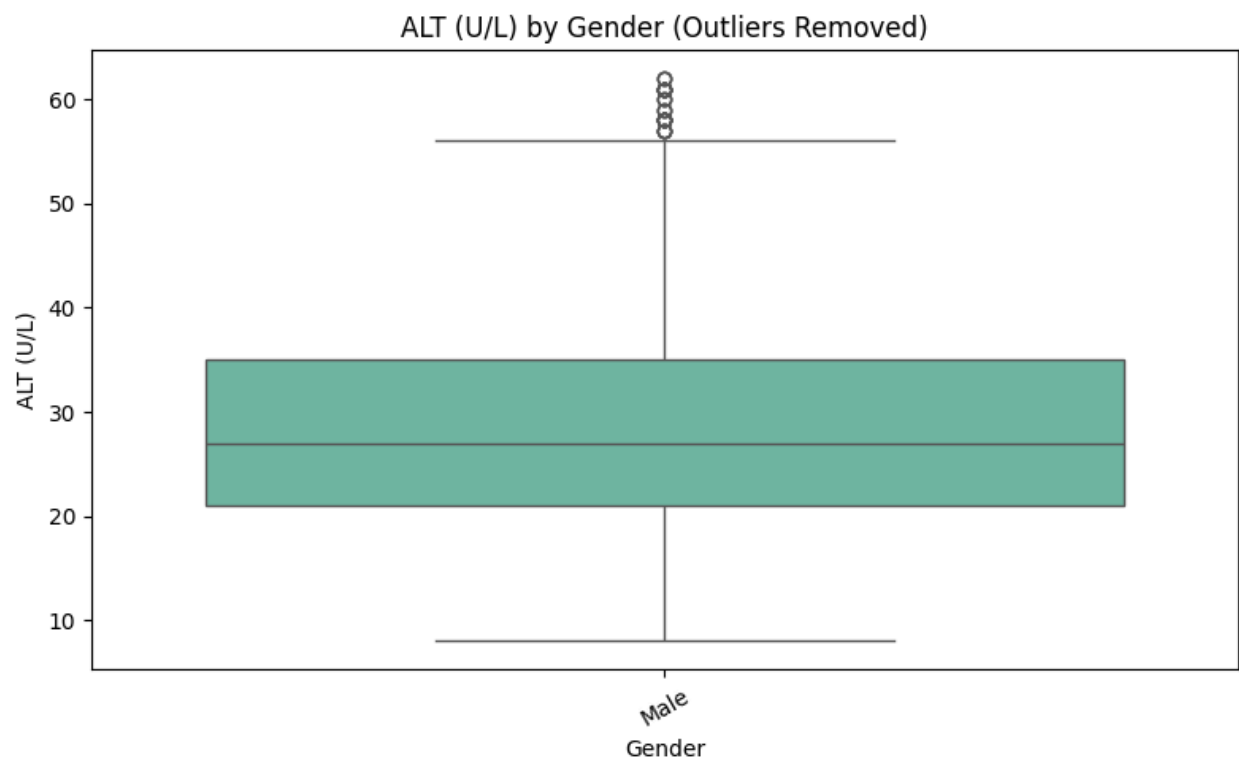
```
sns.boxplot(data=filtered_df, x=group_var, y=biomarker, palette='Set2')
```



/tmp/ipython-input-37-183678998.py:29: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

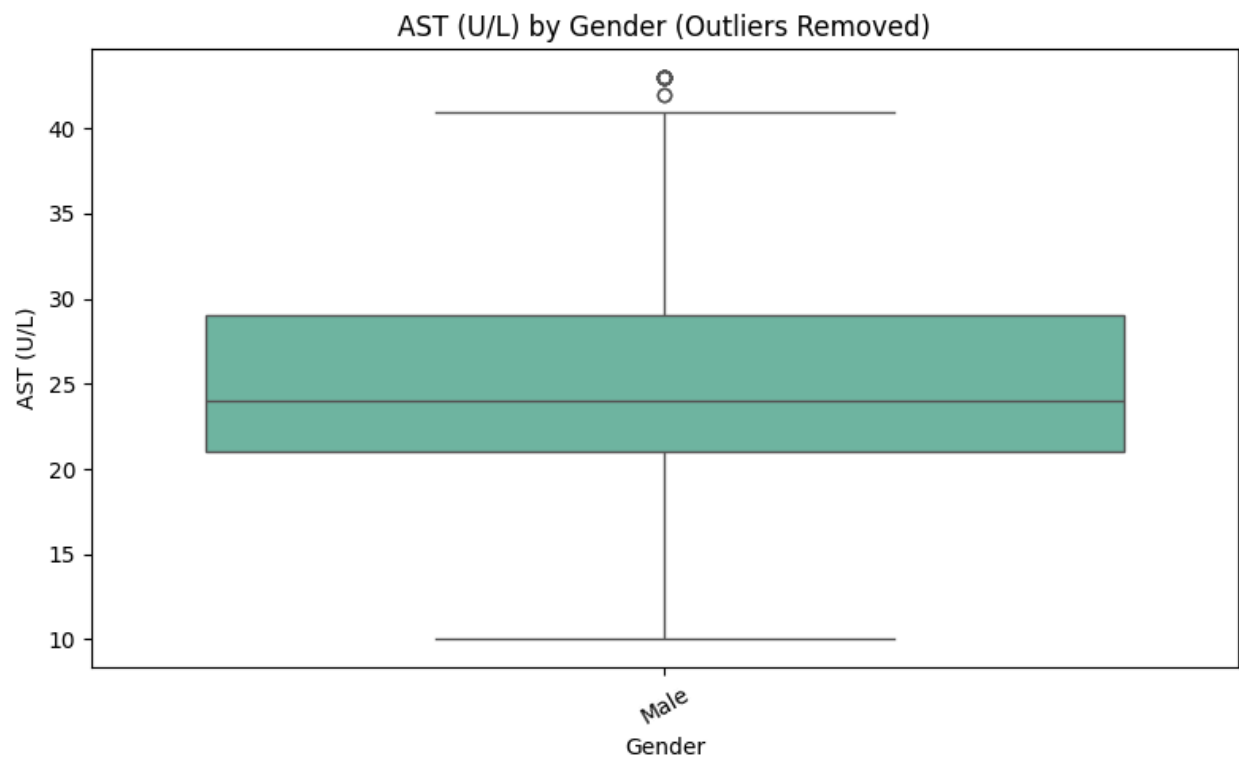
```
sns.boxplot(data=filtered_df, x=group_var, y=biomarker, palette='Set2')
```



/tmp/ipython-input-37-183678998.py:29: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(data=filtered_df, x=group_var, y=biomarker, palette='Set2')
```



/tmp/ipython-input-37-183678998.py:29: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(data=filtered_df, x=group_var, y=biomarker, palette='Set2')
```



Analysis of boxplots

ALT

In the ALT(U/L) by age group graph. The median and interquartile ranges remain the similar throughout the age groups, showing that there is consistent variability. In the ALT(U/L) by ethnicity graph, Mexican Americans have a similar median ALT level but both groups show similar variability in ALT levels within the IQR. In the ALT(U/L) by gender graph, the median ALT level for males appears to be moderate, approximately 26 U/L and the IQR suggests a moderate spread in ALT levels among males.

AST

In the AST(U/L) by age there is no clear upward or downward trend in AST levels across age groups and the median AST values appear relatively stable across the age range. Age groups are still not a strong predictor of AST levels. In the AST(U/L) by ethnicity graph, the median AST levels are identical between the two groups as the IQRs and whiskers are also very similar, indicating that the distribution and variability of AST levels are nearly the same for both ethnicities. In the AST (U/L) by gender graph, there is a moderate spread through the quartiles and the median AST levels are moderate, being approximately 25 U/L.

GGT

In the GGT (U/L) by age group graph, the median and interquartile ranges are also stable throughout the age groups. In the GGT(U/L) by ethnicity graph, the median GGT levels are identical between the two groups as the IQRs and whiskers are also very similar, indicating that the distribution and variability of AST levels are nearly the same for both ethnicities. In the AST (U/L) by gender graph, the IQR shows moderate variability. This shows that most males have GGT levels within a relatively tight range, but there is a long tail of higher values due to outliers.

Relationship of Sleep Duration and Liver Enzyme Levels

Box plots: ALT/AST/GGT distribution by sleep group

We first categorized sleep hours into groups of either missing data, short (<6), normal(6-8hrs), and long (>8h) sleeping hours.

```
In [ ]: import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Categorize sleep duration
def sleep_cat(hours):
    if pd.isna(hours):
        return 'Missing'
    elif hours < 6:
        return 'Short (<6h)'
    elif 6 <= hours <= 8:
        return 'Normal (6-8h)'
    elif hours > 8:
        return 'Long (>8h)'
    else:
        return 'Other'

df_cleaned['SLEEP_CAT'] = df_cleaned['SLD010H'].apply(sleep_cat)

# Define enzymes and their labels
enzymes = {
    'LBXSATSI': 'ALT (U/L)',
    'LBXSASSI': 'AST (U/L)',
    'LBXSGTSI': 'GGT (U/L)'
}

# Plot bar graphs (mean enzyme level by sleep category)
for enz, label in enzymes.items():
    if enz in df_cleaned.columns:
        summary = df_cleaned.groupby('SLEEP_CAT')[enz].mean().reset_index()

        plt.figure(figsize=(8, 5))
        sns.barplot(data=summary, x='SLEEP_CAT', y=enz, palette='Set2')

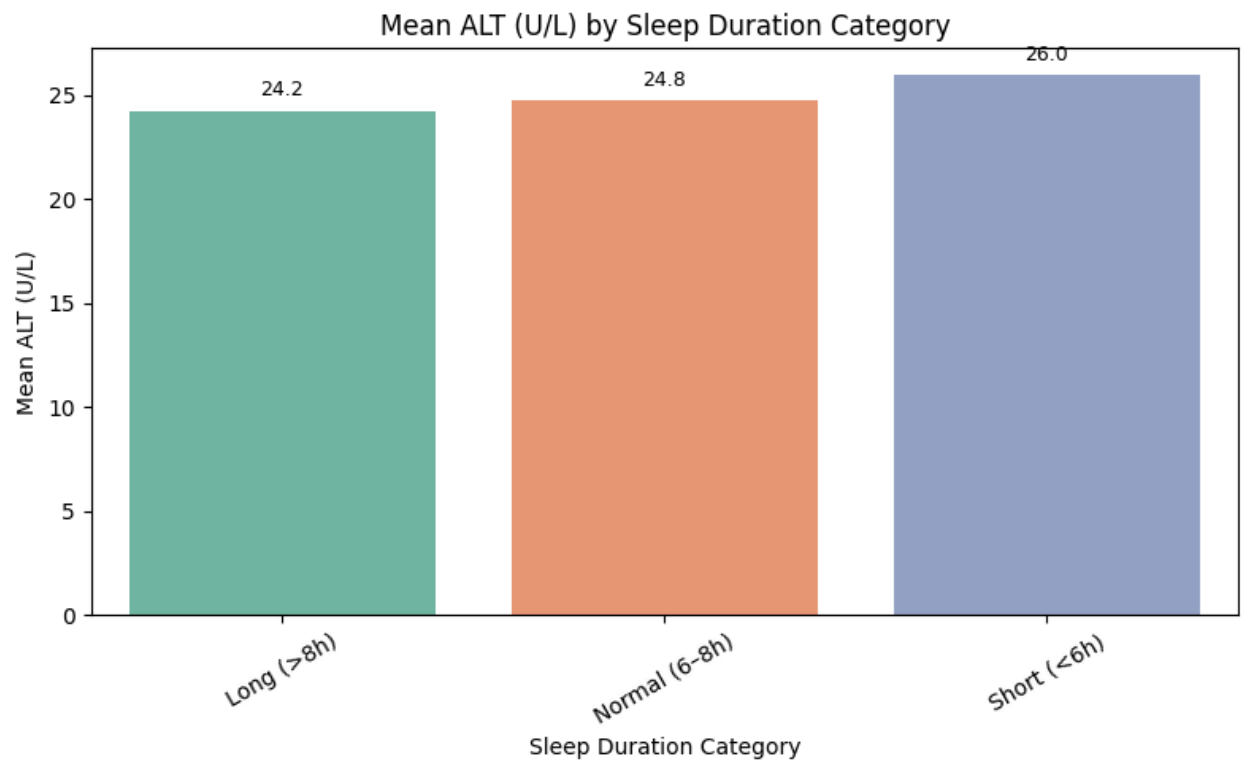
        # Annotate bars
        for index, row in summary.iterrows():
            plt.text(index, row[enz] + 0.5, f'{row[enz]:.1f}', ha='center', va='bottom', fontsize=9)

        plt.title(f'Mean {label} by Sleep Duration Category')
        plt.xlabel('Sleep Duration Category')
        plt.ylabel(f'Mean {label}')
        plt.xticks(rotation=30)
        plt.tight_layout()
        plt.show()
```

```
/tmp/ipython-input-48-3855873490.py:33: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

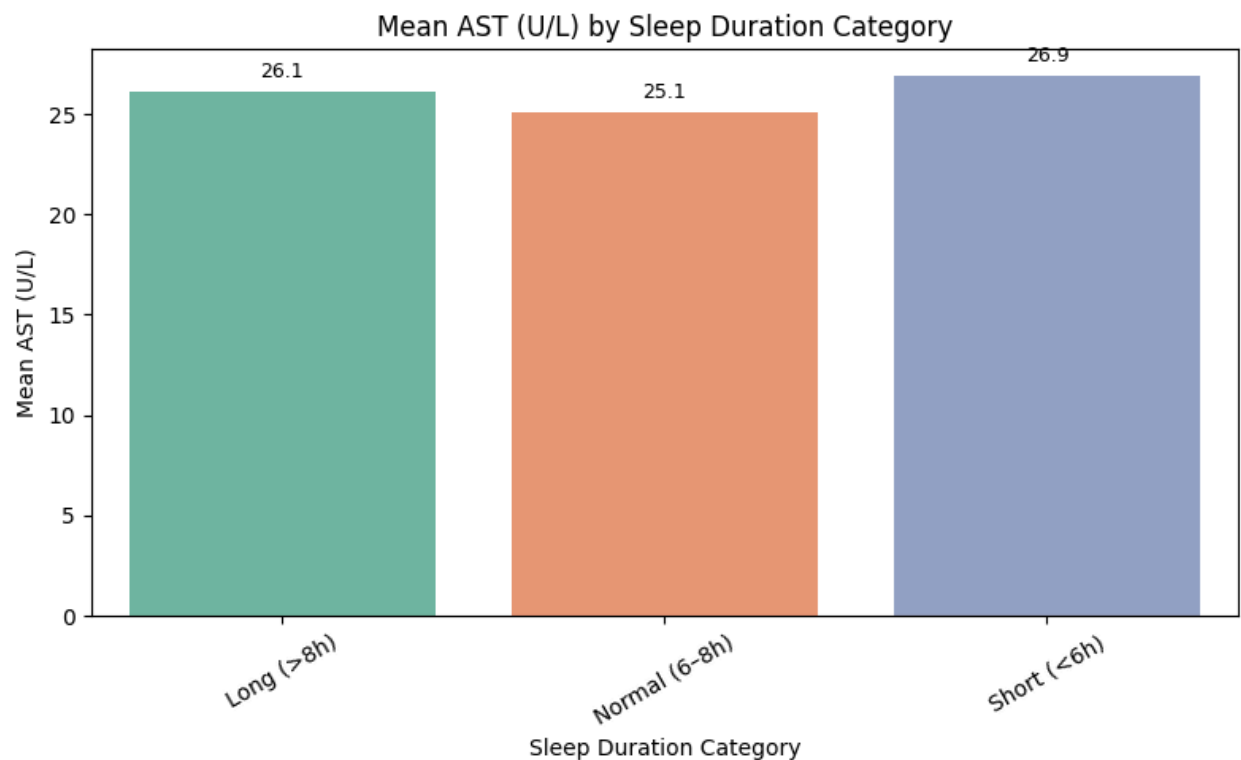
```
sns.barplot(data=summary, x='SLEEP_CAT', y=enx, palette='Set2')
```



```
/tmp/ipython-input-48-3855873490.py:33: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

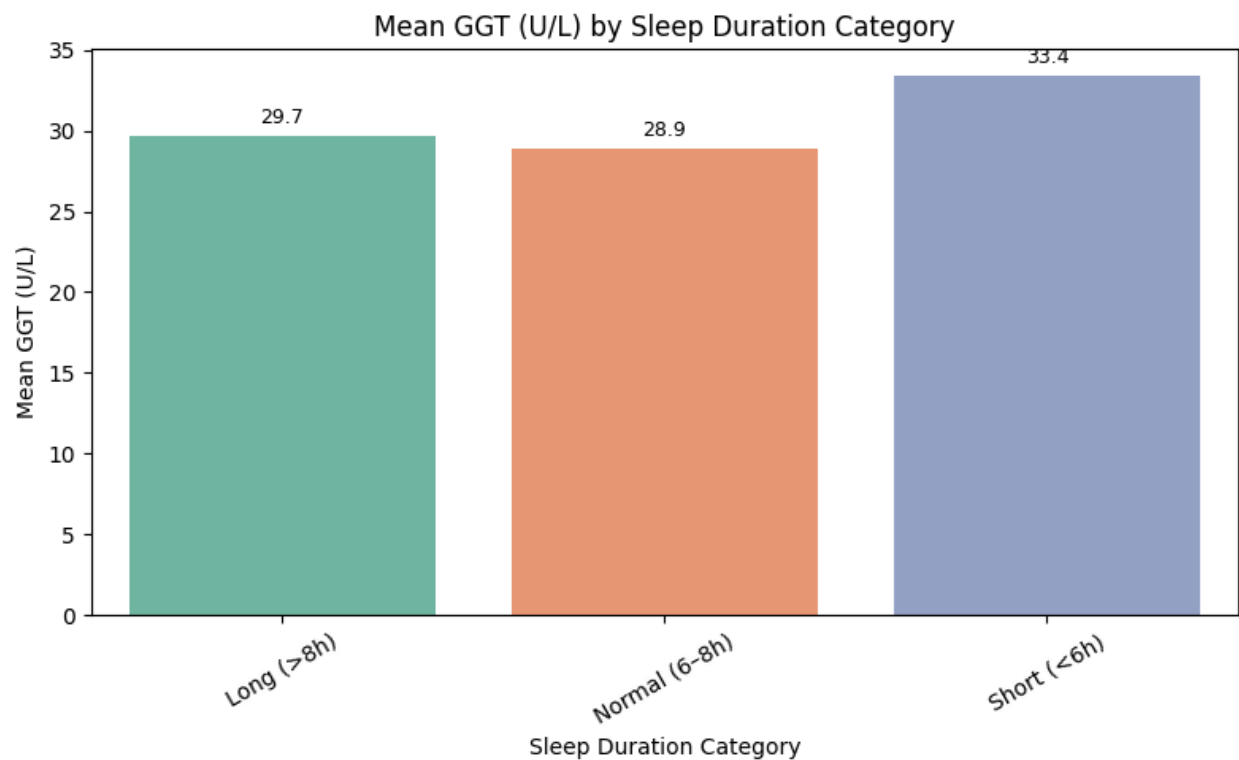
```
sns.barplot(data=summary, x='SLEEP_CAT', y=enx, palette='Set2')
```



```
/tmp/ipython-input-48-3855873490.py:33: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=summary, x='SLEEP_CAT', y=enx, palette='Set2')
```



Spearman correlation: explores whether liver enzymes increase/decrease with more sleep

```
In [ ]: from scipy.stats import spearmanr

# Define enzymes to compare
enzymes = {
    'LBXSATSI': 'ALT (U/L)',
    'LBXSASSI': 'AST (U/L)',
    'LBXSGTSI': 'GGT (U/L)'
}

# Interpret correlation strength
def interpret_correlation(rho):
    abs_rho = abs(rho)
    if abs_rho < 0.1:
        return "negligible"
    elif abs_rho < 0.3:
        return "weak"
    elif abs_rho < 0.5:
        return "moderate"
    else:
        return "strong"

# Run and interpret test
print("\n📊 Spearman Correlation between Sleep Duration and Liver Enzymes:\n")
for enz, label in enzymes.items():
    if enz in df_cleaned.columns:
        sleep = df_cleaned['SLD010H']
        biomarker = df_cleaned[enz]
        mask = ~sleep.isna() & ~biomarker.isna()

        rho, p = spearmanr(sleep[mask], biomarker[mask])

        # Direction
        direction = "positive" if rho > 0 else "negative"

        # Strength interpretation
        strength = interpret_correlation(rho)

        # Significance interpretation
        significance = "statistically significant" if p < 0.05 else "not statistically significant"

        # Final summary
        print(f" - {label}:")
        print(f"   Spearman ρ = {rho:.3f}, p = {p:.4f}")
        print(f"   ➤ There is a {strength}, {direction} correlation between sleep duration and {label}, which
```


 Spearman Correlation between Sleep Duration and Liver Enzymes:

- ALT (U/L):
Spearman $\rho = -0.031$, $p = 0.0000$
➤ There is a negligible, negative correlation between sleep duration and ALT (U/L), which is statistically significant.
- AST (U/L):
Spearman $\rho = -0.008$, $p = 0.1290$
➤ There is a negligible, negative correlation between sleep duration and AST (U/L), which is not statistically significant.
- GGT (U/L):
Spearman $\rho = -0.039$, $p = 0.0000$
➤ There is a negligible, negative correlation between sleep duration and GGT (U/L), which is statistically significant.

8. Annual Trends

1. Annual Trend of Median Values (biomarkers + independents)

```
In [ ]: # Create AST/ALT ratio
df_cleaned['AST_ALT_RATIO'] = df_cleaned['LBXSASSI'] / df_cleaned['LBXSATSI']

# Variables to analyze
vars_to_track = ['LBXSATSI', 'LBXSASSI', 'LBXSGTSI', 'AST_ALT_RATIO',
                 'ALQ130', 'SLD010H', 'PAD615', 'BMXBMI']

# Compute medians by cycle
median_trend = df_cleaned.groupby('CYCLE')[vars_to_track].median().reset_index()

# Plot + show median values
import seaborn as sns
import matplotlib.pyplot as plt
from IPython.display import display # Only works in notebooks like Colab

for var in vars_to_track:
    plt.figure(figsize=(8, 4))
    sns.lineplot(data=median_trend, x='CYCLE', y=var, marker='o')
    plt.title(f'Median {var} by NHANES Cycle')
    plt.xticks(rotation=45)
    plt.ylabel('Median Value')
    plt.xlabel('NHANES Cycle')
    plt.tight_layout()
    plt.show()

# Show table of median values
print(f"\n Table of Median Values for {var}:")
display(median_trend[['CYCLE', var]])
```

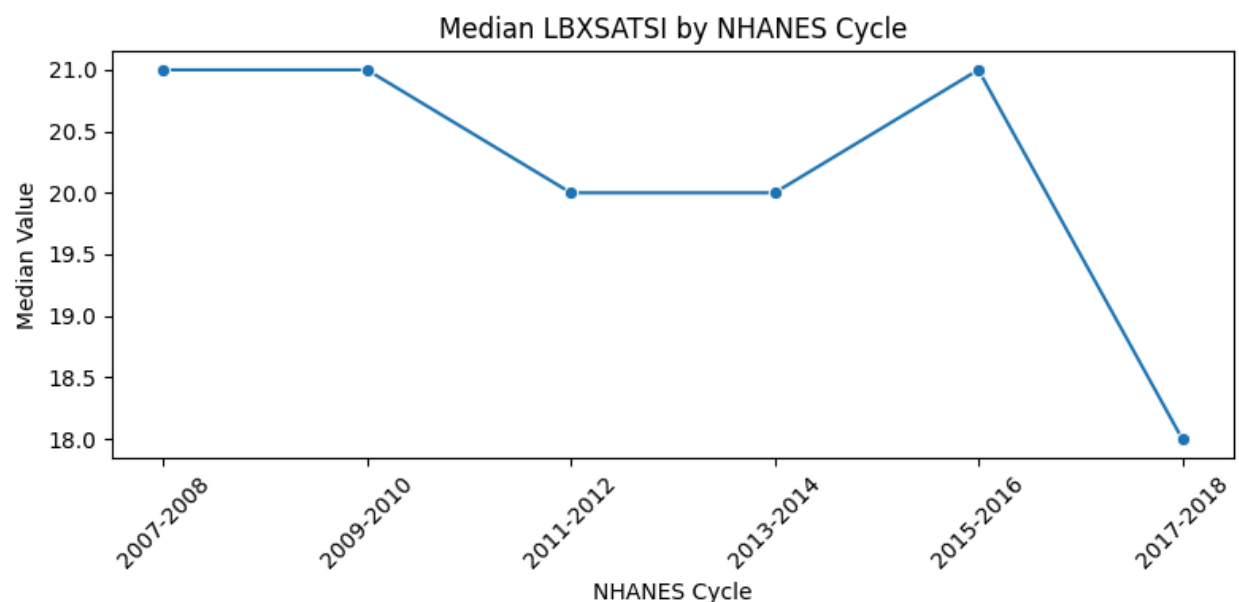


 Table of Median Values for LBXSATSI:

	CYCLE	LBXSATSI
0	2007-2008	21.0
1	2009-2010	21.0
2	2011-2012	20.0
3	2013-2014	20.0
4	2015-2016	21.0
5	2017-2018	18.0

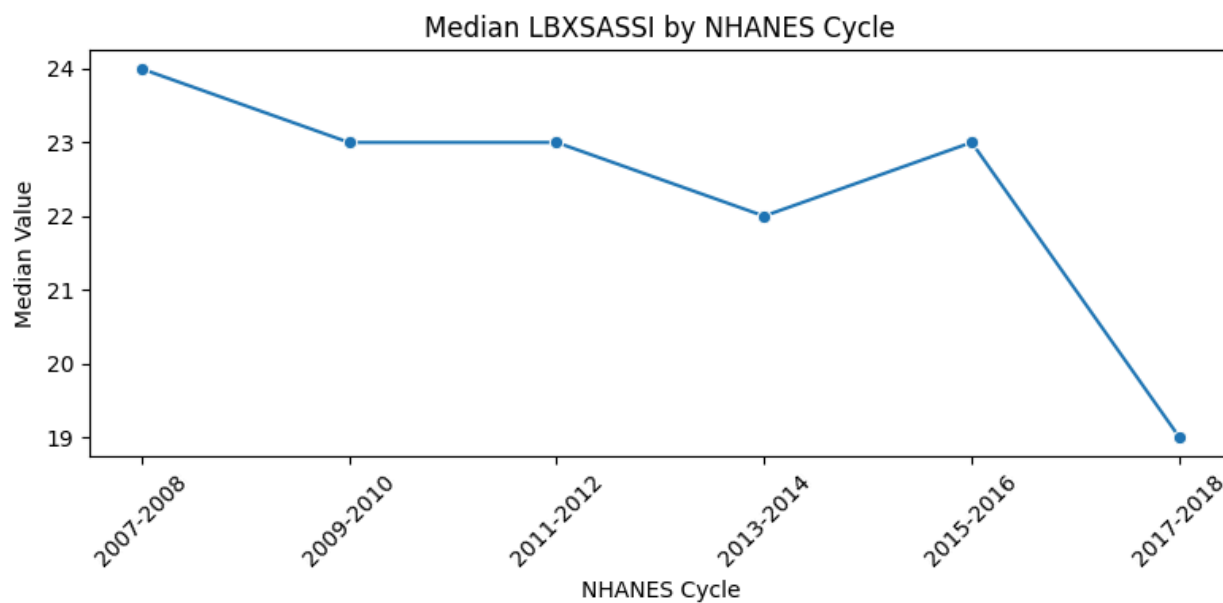


Table of Median Values for LBXSASSI:

	CYCLE	LBXSASSI
0	2007-2008	24.0
1	2009-2010	23.0
2	2011-2012	23.0
3	2013-2014	22.0
4	2015-2016	23.0
5	2017-2018	19.0

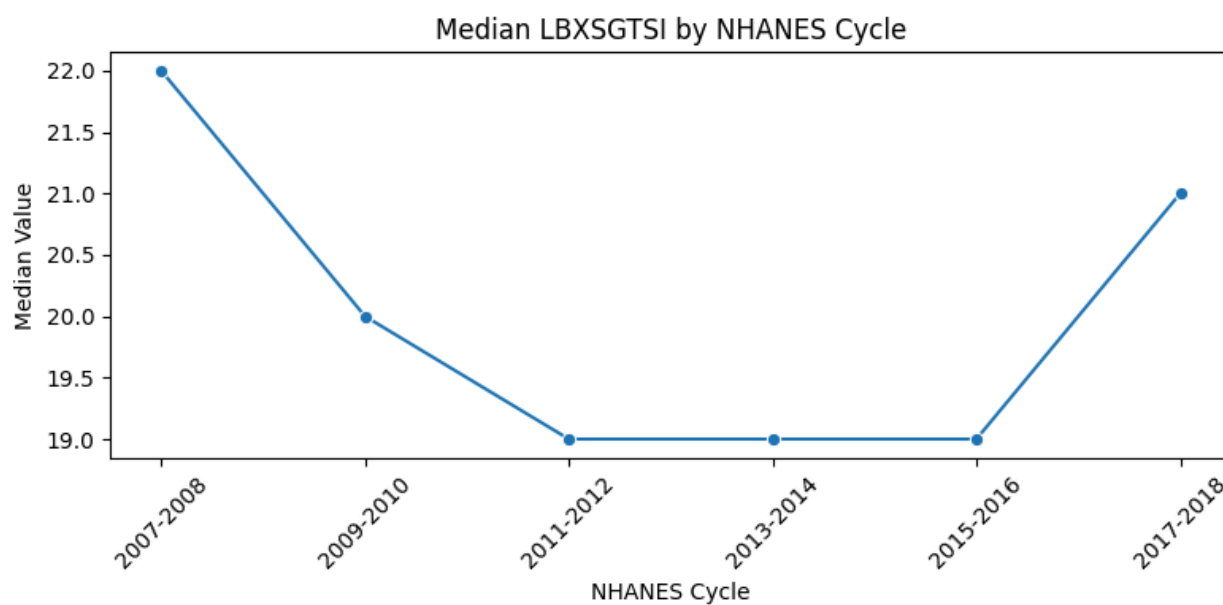


Table of Median Values for LBXSGTSI:

	CYCLE	LBXSGTSI
0	2007-2008	22.0
1	2009-2010	20.0
2	2011-2012	19.0
3	2013-2014	19.0
4	2015-2016	19.0
5	2017-2018	21.0

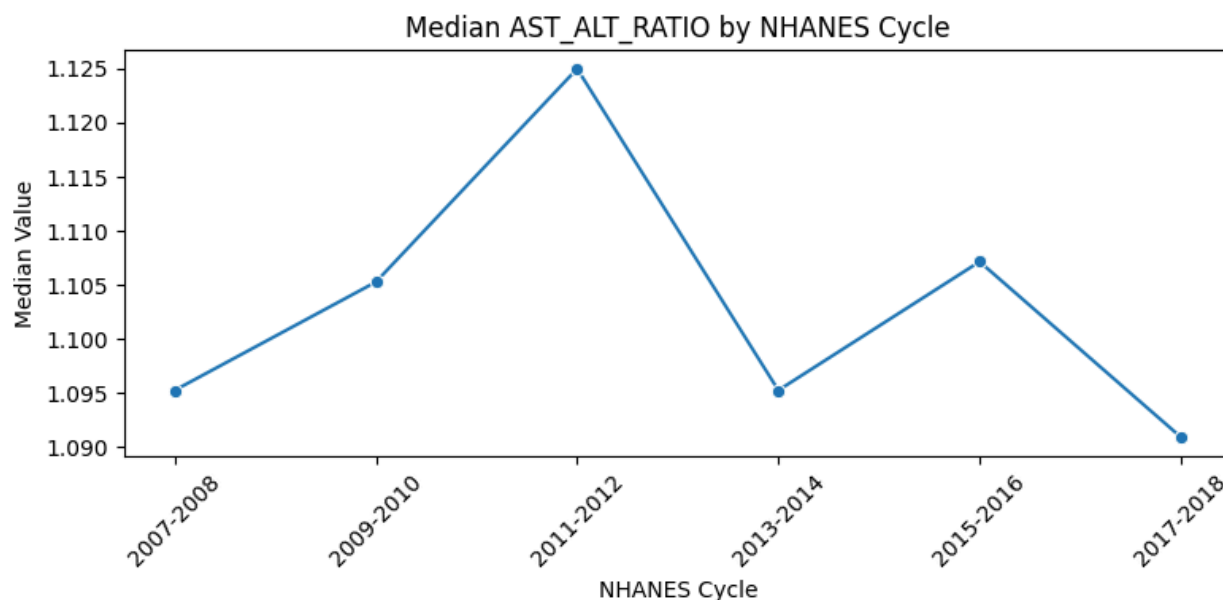


Table of Median Values for AST_ALT_RATIO:

	CYCLE	AST_ALT_RATIO
0	2007-2008	1.095238
1	2009-2010	1.105263
2	2011-2012	1.125000
3	2013-2014	1.095238
4	2015-2016	1.107143
5	2017-2018	1.090909

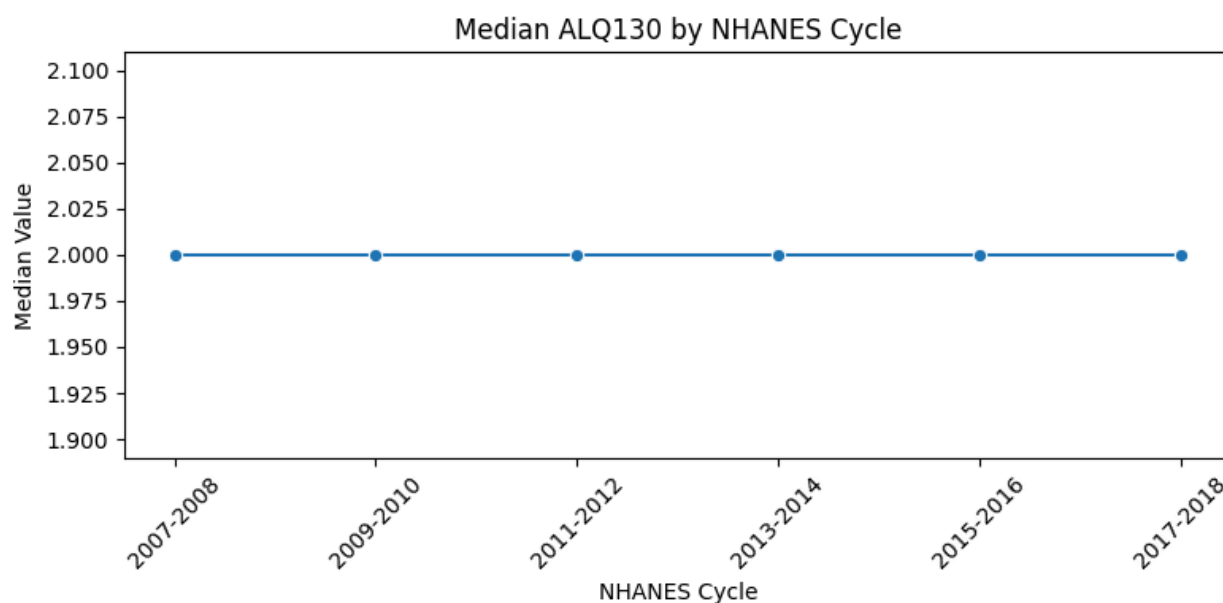


Table of Median Values for ALQ130:

	CYCLE	ALQ130
0	2007-2008	2.0
1	2009-2010	2.0
2	2011-2012	2.0
3	2013-2014	2.0
4	2015-2016	2.0
5	2017-2018	2.0

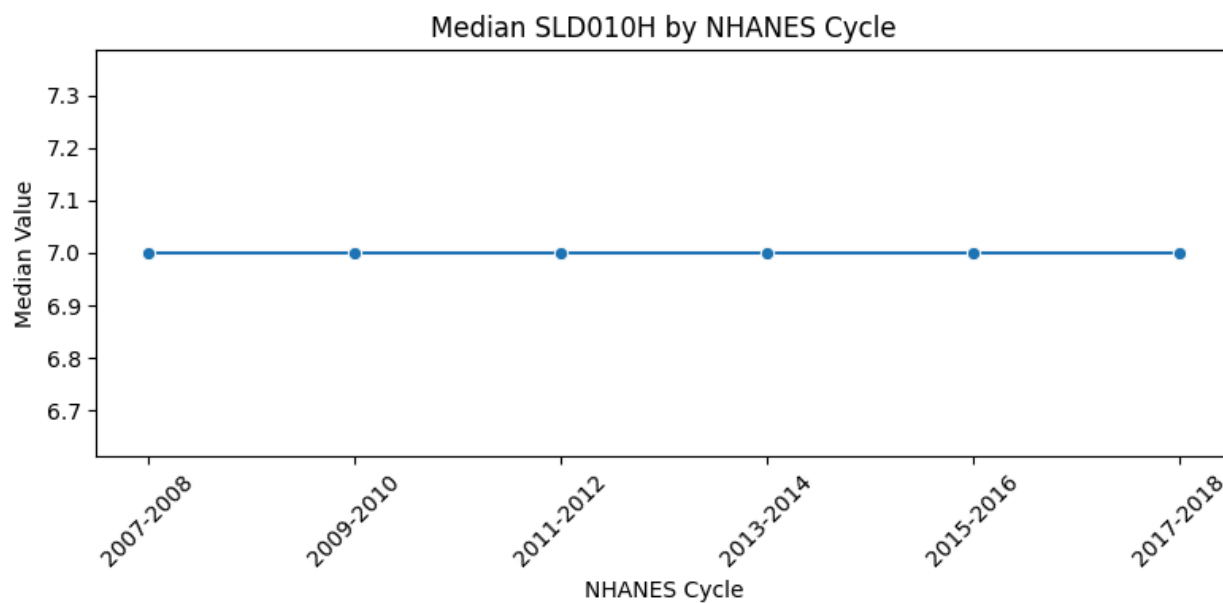


Table of Median Values for SLD010H:

	CYCLE	SLD010H
0	2007-2008	7.0
1	2009-2010	7.0
2	2011-2012	7.0
3	2013-2014	7.0
4	2015-2016	7.0
5	2017-2018	7.0

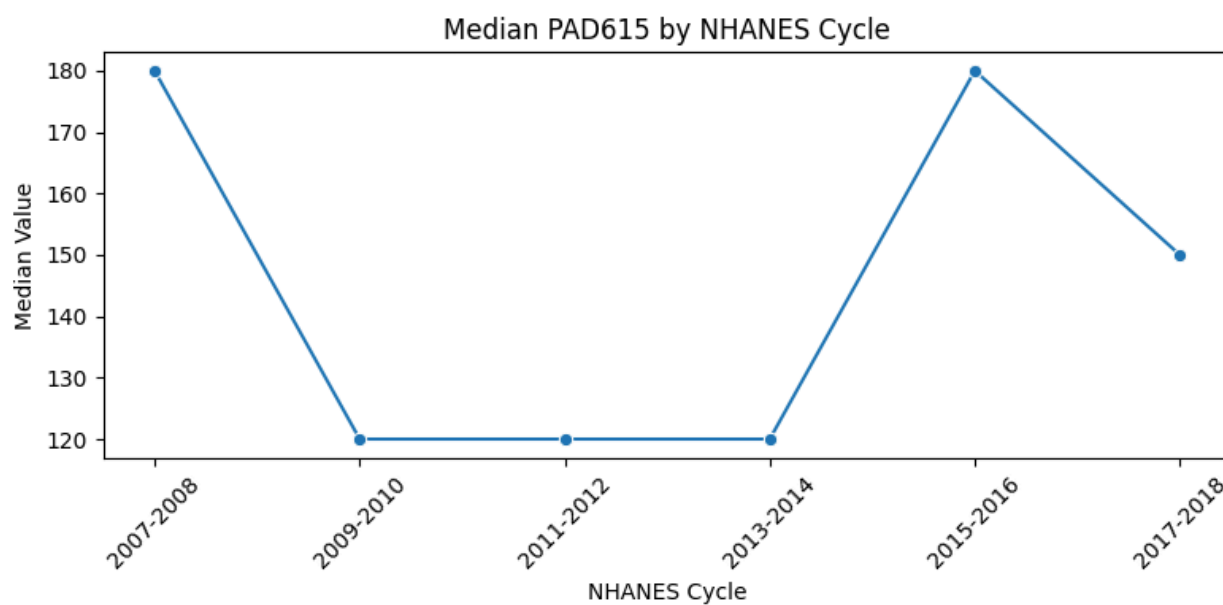


Table of Median Values for PAD615:

	CYCLE	PAD615
0	2007-2008	180.0
1	2009-2010	120.0
2	2011-2012	120.0
3	2013-2014	120.0
4	2015-2016	180.0
5	2017-2018	150.0

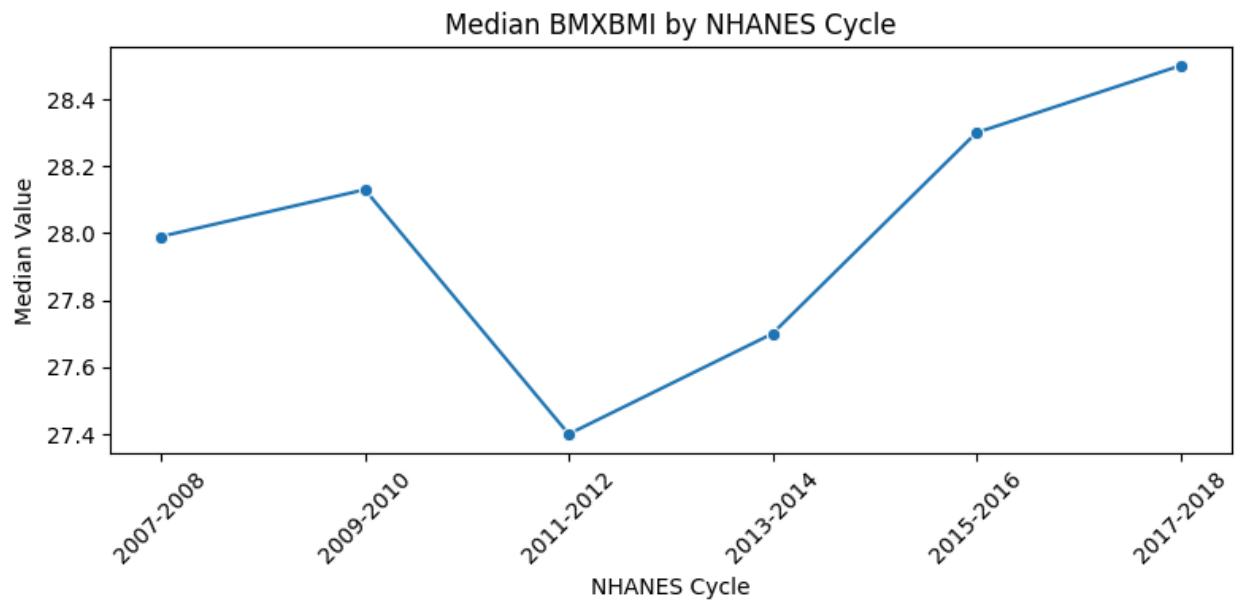


Table of Median Values for BMXBMI:

	CYCLE	BMXBMI
0	2007-2008	27.99
1	2009-2010	28.13
2	2011-2012	27.40
3	2013-2014	27.70
4	2015-2016	28.30
5	2017-2018	28.50

Analysis

For median ALT levels, the trend is generally decreasing throughout the years from 21.0 U/L in 2007-2008 to 18.0 U/L in 2017-2018.

For median AST levels, the trend is generally decreasing throughout the years from 24.0 U/L in 2007-2008 to 19.0 U/L in 2017-2018.

For median GGT levels, the trend is generally decreasing throughout the years from 22.0 U/L in 2007-2008 to 19.0 U/L in 2015-2016. The exception is the final cycle, 2017-2018, where the median GGT levels increased from 19.0 U/L to 21.0 U/L.

For median AST/ALT ratio, the trend increases from 1.095 in 2007-2008 to 1.125 from 2011-2012. The trend then decreases to 1.090 U/L in 2017-2018 with an increase in the 2015-2016 cycle to 1.105.

For median alcohol consumption, it remains consistent at 2 drinks per day throughout the years from 2007-2018.

For median sleep duration, it remains consistent at 7 hours of sleep every night throughout the years from 2007-2018.

For median physical activity, the trend decreases from 180 minutes in 2007-2008 to 120 in 2009-2010. The trend then remains constant at 120 minutes from 2009-2014, where it increases again to 180 minutes in 2015-2016, then decreases again to 150 minutes in 2017-2018.

For median BMI, the trend is generally increasing with the exception of 2011-2012 where the median BMI dropped to 27.4.

2. Percentage Sample with Suspected Alcohol-Related Liver Dysfunction

Criteria:

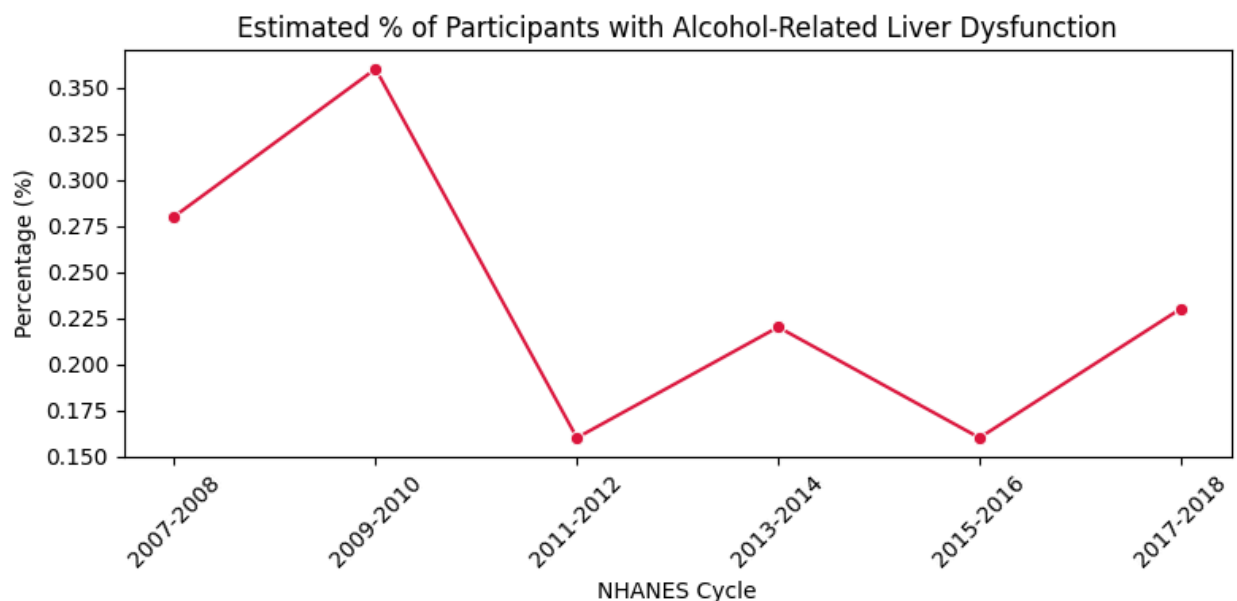
AST/ALT ratio > 2

GGT > 50 U/L (upper limit) (Assessing liver function and interpreting liver blood tests, 2024)

```
In [ ]: # Define ALD-related thresholds
df_cleaned['ALD_FLAG'] = ((df_cleaned['AST_ALT_RATIO'] > 2) & (df_cleaned['LBXSGTSI'] > 50))

# Calculate % ALD-flagged per cycle
ald_rate = df_cleaned.groupby('CYCLE')['ALD_FLAG'].mean().reset_index()
ald_rate['Percent_ALD'] = round(ald_rate['ALD_FLAG'] * 100, 2)

# Plot
plt.figure(figsize=(8, 4))
sns.lineplot(data=ald_rate, x='CYCLE', y='Percent_ALD', marker='o', color='crimson')
plt.title('Estimated % of Participants with Alcohol-Related Liver Dysfunction')
plt.ylabel('Percentage (%)')
plt.xlabel('NHANES Cycle')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



Analysis of percentage sample with suspected ALD graph

From 2007-2008 to 2009-2010, there is a significant and sharp increase, increasing from 0.280% to 0.363%. Afterwards, the percentage drops even more significantly than before from 2009-2010 to 2011-2012, where the percentage goes from 0.363% to approximately 0.16%. From 2011-2012 onwards, there is only a gradual increase to 2013-2014, percentage increasing from 0.16% to 0.219%. Next, there is a drop from 0.219% to 0.157% from 2013-2014. Finally, there is an increase from 2015-2016 where 0.11% increases to 0.235%.

3. Percentage sample with High-Risk Ranges

(e.g., ALT > 40, AST > 40, GGT > 60, AST/ALT > 2)

```

In [ ]: # Map cycle to short labels
cycle_map = {
    '2007-2008': 'A',
    '2009-2010': 'B',
    '2011-2012': 'C',
    '2013-2014': 'D',
    '2015-2016': 'E',
    '2017-2018': 'F'
}

df_cleaned['CYCLE_SHORT'] = df_cleaned['CYCLE'].map(cycle_map)
df_cleaned['ALT_HIGH'] = df_cleaned['LBXSATSI'] > 40
df_cleaned['AST_HIGH'] = df_cleaned['LBXSASSI'] > 40
df_cleaned['GGT_HIGH'] = df_cleaned['LBXSGTSI'] > 60
df_cleaned['AST_ALT_RATIO'] = df_cleaned['LBXSASSI'] / df_cleaned['LBXSATSI']
df_cleaned['AST_ALT_HIGH'] = df_cleaned['AST_ALT_RATIO'] > 2
#define risk flags
risk_flags = ['ALT_HIGH', 'AST_HIGH', 'GGT_HIGH', 'AST_ALT_HIGH']

# Group by short cycle label
risk_trends = df_cleaned.groupby('CYCLE_SHORT')[risk_flags].mean().reset_index() * 100
risk_trends = risk_trends.round(2)

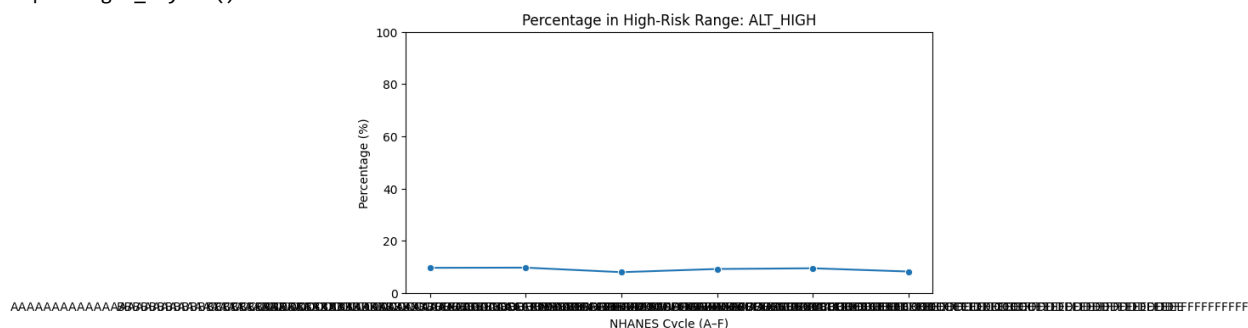
# Plot
import seaborn as sns
import matplotlib.pyplot as plt

for flag in risk_flags:
    plt.figure(figsize=(8, 4))
    sns.lineplot(data=risk_trends, x='CYCLE_SHORT', y=flag, marker='o')
    plt.title(f'Percentage in High-Risk Range: {flag}')
    plt.ylabel('Percentage (%)')
    plt.xlabel('NHANES Cycle (A-F)')
    plt.ylim(0, 100)
    plt.tight_layout()
    plt.show()

```

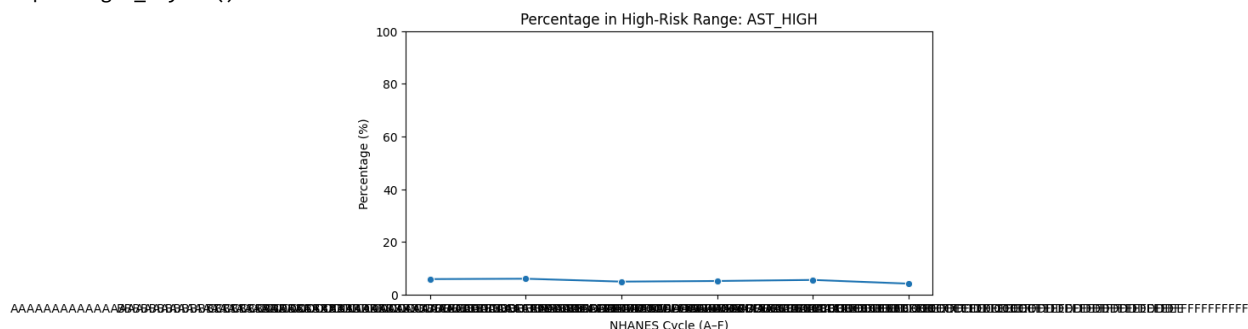
/tmp/ipython-input-55-212703745.py:35: UserWarning: Tight layout not applied. The left and right margins cannot be made large enough to accommodate all Axes decorations.

```
plt.tight_layout()
```



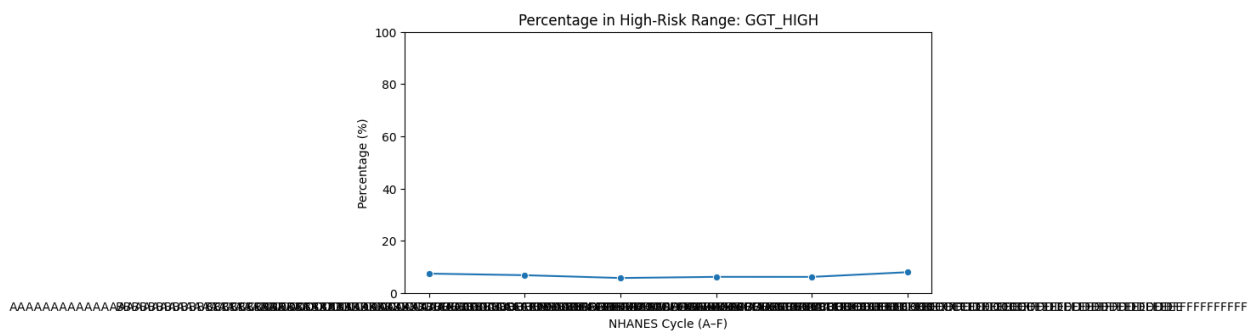
/tmp/ipython-input-55-212703745.py:35: UserWarning: Tight layout not applied. The left and right margins cannot be made large enough to accommodate all Axes decorations.

```
plt.tight_layout()
```



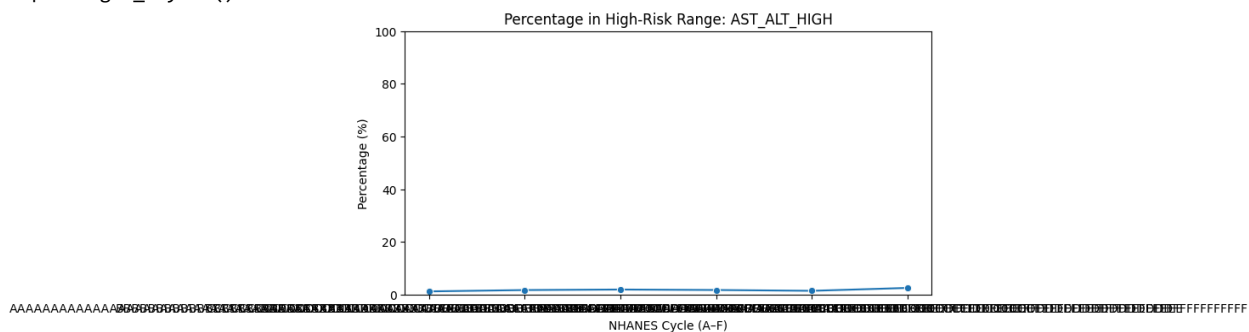
/tmp/ipython-input-55-212703745.py:35: UserWarning: Tight layout not applied. The left and right margins cannot be made large enough to accommodate all Axes decorations.

```
plt.tight_layout()
```



/tmp/ipython-input-55-212703745.py:35: UserWarning: Tight layout not applied. The left and right margins cannot be made large enough to accommodate all Axes decorations.

plt.tight_layout()



Analysis of results

In general, the percentage sample with high-risk ranges amongst ALT, AST, and GGT as well as AST/ALT are all relatively low throughout the 6 cycles. ALT and AST levels indicating risk have a slightly decreasing trend over time, but GGT and AST/ALT levels indicating risk have a slightly increasing trend over time. This shows that throughout the years the percentage of people with risk of disease did increase, as AST/ALT values and GGT values are the main indicators of ALD.

Relationship between sleep duration and risk of ALD

```
In [ ]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.stats import chi2_contingency

# 1. Categorize sleep duration
def sleep_cat(hours):
    if pd.isna(hours):
        return 'Missing'
    elif hours < 6:
        return 'Short (<6h)'
    elif 6 <= hours <= 8:
        return 'Normal (6-8h)'
    elif hours > 8:
        return 'Long (>8h)'
    else:
        return 'Other'

df_cleaned['SLEEP_CAT'] = df_cleaned['SLD010H'].apply(sleep_cat)

# 2. Flag suspected ALD cases (AST/ALT > 2 and GGT > 60)
df_cleaned['ALD_FLAG'] = ((df_cleaned['LBXSASSI'] / df_cleaned['LBXSATSI']) > 2) & (df_cleaned['LBXSGTSI'] > 5)

# 3. Summary Table: % ALD risk in each sleep group
summary = df_cleaned.groupby('SLEEP_CAT')['ALD_FLAG'].agg(['sum', 'count'])
summary['Percent_ALD'] = round((summary['sum'] / summary['count']) * 100, 2)
print("Percentage of Suspected ALD Cases by Sleep Category:")
display(summary)

# 4. Plot bar chart
plt.figure(figsize=(7, 5))
sns.barplot(data=summary.reset_index(), x='SLEEP_CAT', y='Percent_ALD', palette='Set2')
plt.title('Estimated % with Alcohol-Related Liver Dysfunction by Sleep Duration')
plt.xlabel('Sleep Duration Category')
plt.ylabel('% Suspected ALD')
plt.tight_layout()
plt.show()

# 5. Chi-square test for association
cross_tab = pd.crosstab(df_cleaned['SLEEP_CAT'], df_cleaned['ALD_FLAG'])
chi2, p, dof, expected = chi2_contingency(cross_tab)

# 6. Interpretation
print("Chi-Square Test of ALD Risk vs Sleep Duration:")
print(f"Chi² = {chi2:.2f}, df = {dof}, p = {p:.4f}")
if p < 0.05:
    print("➤ There is a statistically significant association between sleep duration and ALD risk.")
else:
    print("➤ There is no statistically significant association between sleep duration and ALD risk.")
```

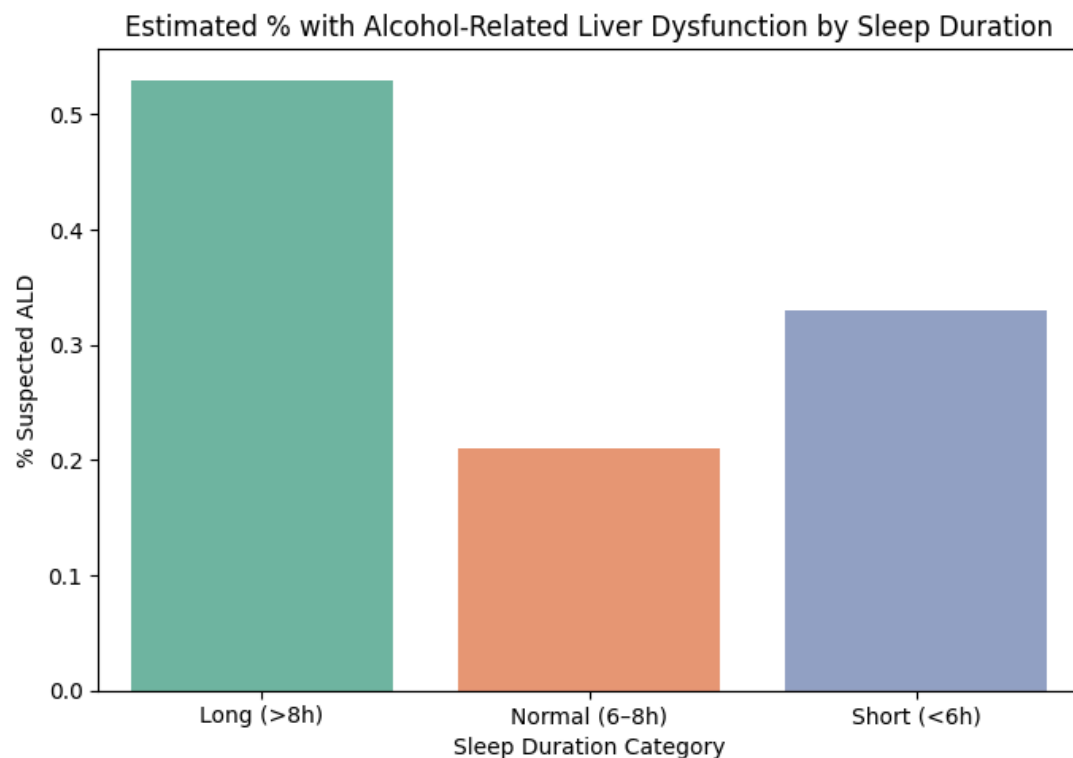
Percentage of Suspected ALD Cases by Sleep Category:

	sum	count	Percent_ALD
SLEEP_CAT			
Long (>8h)	10	1871	0.53
Normal (6-8h)	60	29097	0.21
Short (<6h)	12	3605	0.33

/tmp/ipython-input-69-3449873078.py:32: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=summary.reset_index(), x='SLEEP_CAT', y='Percent_ALD', palette='Set2')
```



Chi-Square Test of ALD Risk vs Sleep Duration:

$\chi^2 = 9.56$, $df = 2$, $p = 0.0084$

► There is a statistically significant association between sleep duration and ALD risk.

From the graph, the estimated percentage of people with ALD is the highest in those who sleep for long hours, followed by those who sleep short hours, and lastly the sleeping normal hours. This can suggest that sleeping hours too long and short increases the risk of a person having ALD. The Chi-Square test also shows that there is a statistically significant association between sleep duration and ALD risk.

Heatmap of sleep duration against ALD

```
In [ ]: import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# 1. Recalculate ALD flag
df_cleaned['ALD_FLAG'] = ((df_cleaned['LBXSASSI'] / df_cleaned['LBXSATSI']) > 2) & (df_cleaned['LBXSGTSI'] > 5)

# 2. Bin sleep duration
df_cleaned['SLEEP_BIN'] = pd.cut(df_cleaned['SLD010H'], bins=range(0, 13), right=False)

# 3. Create ALD summary per bin
bin_summary = df_cleaned.groupby('SLEEP_BIN')['ALD_FLAG'].agg(['sum', 'count']).reset_index()
bin_summary['Percent_ALD'] = round((bin_summary['sum'] / bin_summary['count']) * 100, 2)

# 4. Filter out bins with too few people (<10)
filtered = bin_summary[bin_summary['count'] >= 10]

# 5. Pivot for heatmap
heat_table = filtered.pivot_table(index='SLEEP_BIN', values='Percent_ALD')

# 6. Plot heatmap
plt.figure(figsize=(6, 6))
sns.heatmap(heat_table, annot=True, fmt=".1f", cmap="YlOrRd", cbar_kws={'label': '% Suspected ALD'})
plt.title('% Suspected Alcohol-Related Liver Dysfunction by Sleep Duration')
plt.xlabel('ALD Risk')
plt.ylabel('Sleep Duration (Hours)')
plt.tight_layout()
plt.show()
```

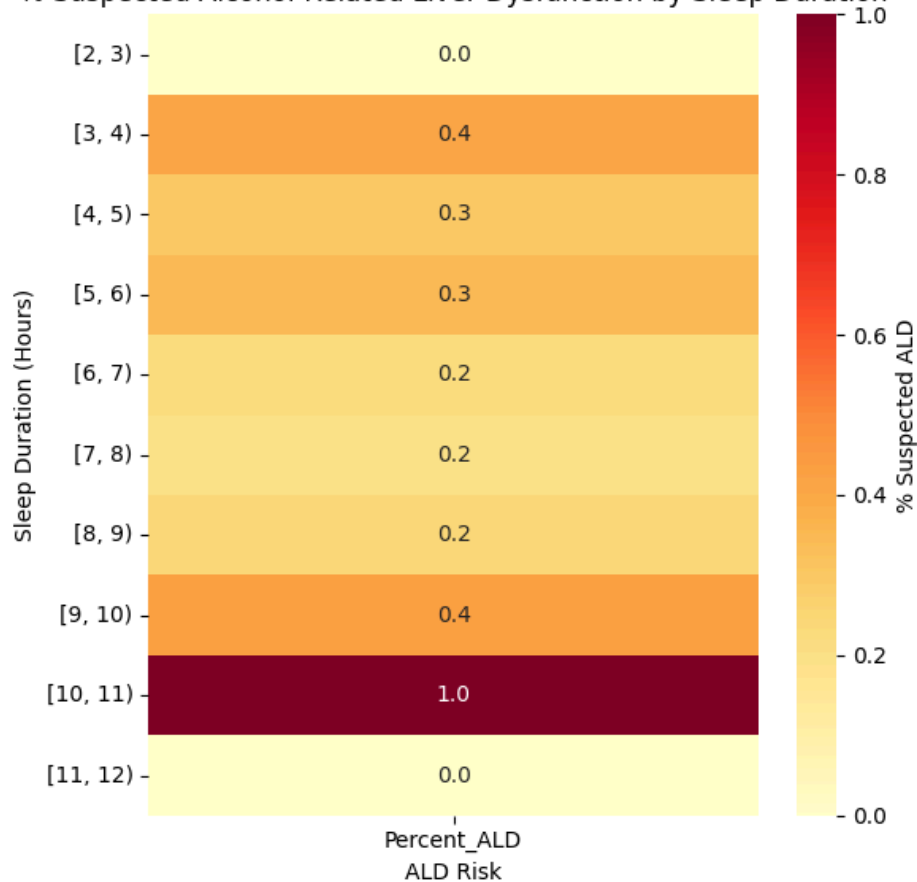
/tmp/ipython-input-72-858213281.py:12: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
bin_summary = df_cleaned.groupby('SLEEP_BIN')['ALD_FLAG'].agg(['sum', 'count']).reset_index()
```

/tmp/ipython-input-72-858213281.py:19: FutureWarning: The default value of observed=False is deprecated and will change to observed=True in a future version of pandas. Specify observed=False to silence this warning and retain the current behavior

```
heat_table = filtered.pivot_table(index='SLEEP_BIN', values='Percent_ALD')
```

% Suspected Alcohol-Related Liver Dysfunction by Sleep Duration



```
In [ ]: # Show counts of people in each sleep bin
df_cleaned['SLEEP_BIN'] = pd.cut(df_cleaned['SLD010H'], bins=range(0, 13), right=False)
bin_summary = df_cleaned.groupby('SLEEP_BIN')['ALD_FLAG'].agg(['sum', 'count'])
bin_summary['Percent_ALD'] = round((bin_summary['sum'] / bin_summary['count']) * 100, 2)

print(bin_summary)
```

SLEEP_BIN	sum	count	Percent_ALD
[0, 1)	0	0	NaN
[1, 2)	0	9	0.00
[2, 3)	0	63	0.00
[3, 4)	1	244	0.41
[4, 5)	3	1010	0.30
[5, 6)	8	2279	0.35
[6, 7)	12	5488	0.22
[7, 8)	33	17380	0.19
[8, 9)	15	6229	0.24
[9, 10)	5	1161	0.43
[10, 11)	5	499	1.00
[11, 12)	0	54	0.00

/tmp/ipython-input-58-2155967804.py:3: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
bin_summary = df_cleaned.groupby('SLEEP_BIN')['ALD_FLAG'].agg(['sum', 'count'])
```

The estimated percentage of people with suspected ALD is highest in those sleeping 10-11 hours and is generally higher below 6 hours and above 9 hours.

The reason for 0.0 values for 2-3 hrs and 11-12 hrs of sleep might be that very few people report sleeping that little, and those that do may not match the liver dysfunction profile, compared to the number of people sleeping 6-9 hours, hence the data would be more accurate for those timeframes.

Analysis of suspected ALD by sleep duration

For both very Short (2–3 hrs) and very Long (11–12 hrs) sleep durations it showed that 0.0% suspected ALD, suggesting no observed cases in these ranges. Whereas, for moderate Sleep Durations (3–4 hrs and 9–10 hrs), shows moderate ALD percentages (around 0.2 to 0.4), indicating some risk. Lastly, the peak risk zone (10–11 hrs), shows the highest percentage (1.0) of suspected ALD. This suggests a extended sleep duration has increased ALD risk in this range.

Heatmap plotting for other independent variables on ALD risk

Age range

```
In [ ]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

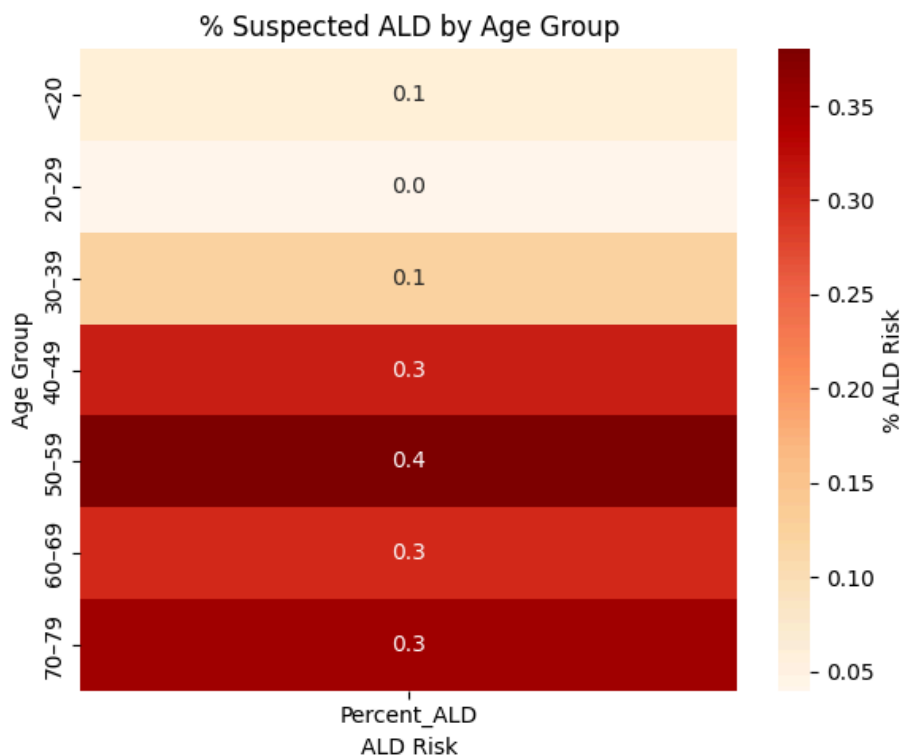
# Step 1: Bin age into groups
df_cleaned['AGE_GROUP'] = pd.cut(df_cleaned['RIDAGEYR'],
                                bins=[0, 20, 30, 40, 50, 60, 70, 80, 100],
                                labels=['<20', '20-29', '30-39', '40-49', '50-59', '60-69', '70-79', '80+'])

# Step 2: Calculate % ALD in each age group
age_summary = df_cleaned.groupby('AGE_GROUP')['ALD_FLAG'].agg(['sum', 'count']).reset_index()
age_summary['Percent_ALD'] = round((age_summary['sum'] / age_summary['count']) * 100, 2)

# Step 3: Pivot to make it 2D (age groups as rows)
age_matrix = age_summary.pivot_table(index='AGE_GROUP', values='Percent_ALD')

# Step 4: Plot the heatmap
plt.figure(figsize=(6, 5))
sns.heatmap(age_matrix, annot=True, fmt=".1f", cmap="OrRd", cbar_kws={'label': '% ALD Risk'})
plt.title('% Suspected ALD by Age Group')
plt.xlabel('ALD Risk')
plt.ylabel('Age Group')
plt.tight_layout()
plt.show()

/tmp/ipython-input-60-2937521781.py:11: FutureWarning: The default of observed=False is deprecated and will be
changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True
to adopt the future default and silence this warning.
  age_summary = df_cleaned.groupby('AGE_GROUP')['ALD_FLAG'].agg(['sum', 'count']).reset_index()
/tmp/ipython-input-60-2937521781.py:15: FutureWarning: The default value of observed=False is deprecated and
will change to observed=True in a future version of pandas. Specify observed=False to silence this warning and
retain the current behavior
  age_matrix = age_summary.pivot_table(index='AGE_GROUP', values='Percent_ALD')
```



Analysis of suspected ALD by age group

There is no ALD risk is observed in the <20 and 20–29 age groups, risk only begins to appear in the 30–39 age group (0.1%). After the increase to 0.3% in 40–49 age group, ALD risk is the highest at 0.4% in the 50–59 age group.

From 60 years onward, the risk stabilizes at 0.3%, indicating a plateau in suspected ALD risk among older adults. This suggests that ALD risk increases with age but only starting in the 30s. The highest and stable risk is observed from age 40 to 79, possibly reflecting the cumulative exposure to alcohol or age-related vulnerability.

BMI

```
In [ ]: # Bin BMI
df_cleaned['BMI_BIN'] = pd.cut(df_cleaned['BMXBMI'], bins=[0, 18.5, 24.9, 29.9, 34.9, 40, 60],
                               labels=['Underweight', 'Normal', 'Overweight', 'Obese I', 'Obese II', 'Obese I

# Summary table
bmi_summary = df_cleaned.groupby('BMI_BIN')['ALD_FLAG'].agg(['sum', 'count']).reset_index()
bmi_summary['Percent_ALD'] = round((bmi_summary['sum'] / bmi_summary['count']) * 100, 2)

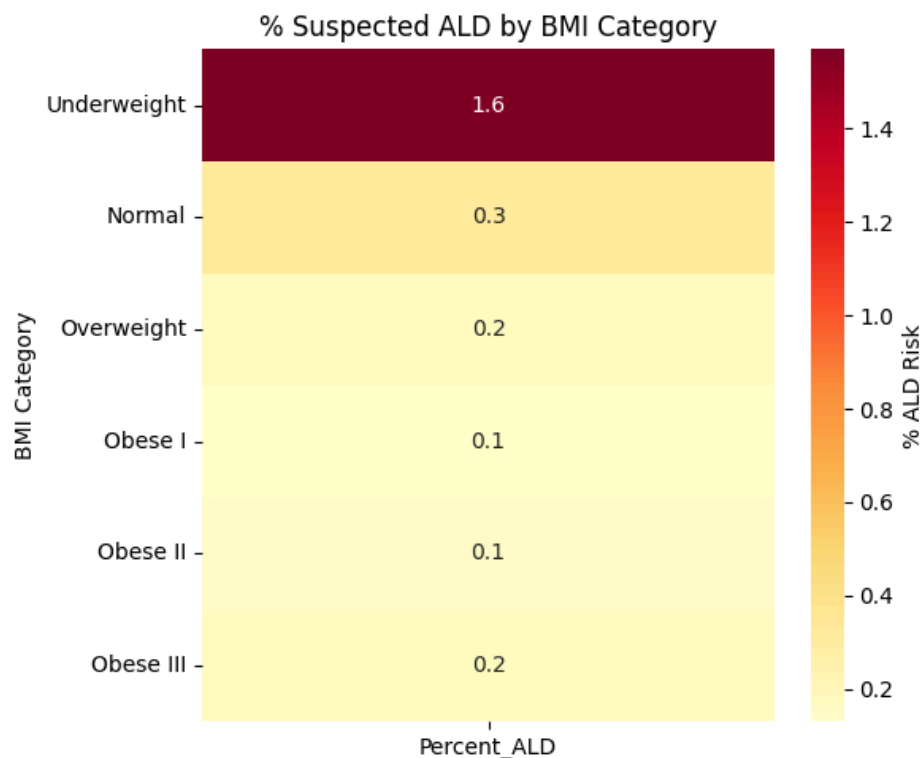
# Pivot and plot
bmi_heat = bmi_summary.pivot_table(index='BMI_BIN', values='Percent_ALD')

plt.figure(figsize=(6, 5))
sns.heatmap(bmi_heat, annot=True, fmt=".1f", cmap="YlOrRd", cbar_kws={'label': '% ALD Risk'})
plt.title('% Suspected ALD by BMI Category')
plt.ylabel('BMI Category')
plt.tight_layout()
plt.show()
```

/tmp/ipython-input-61-1841312448.py:6: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
bmi_summary = df_cleaned.groupby('BMI_BIN')['ALD_FLAG'].agg(['sum', 'count']).reset_index()
/tmp/ipython-input-61-1841312448.py:10: FutureWarning: The default value of observed=False is deprecated and will change to observed=True in a future version of pandas. Specify observed=False to silence this warning and retain the current behavior
```

```
bmi_heat = bmi_summary.pivot_table(index='BMI_BIN', values='Percent_ALD')
```



Analysis of BMI heatmap

Overall, this heatmap, shows a decline in ALD risk % from underweight to obese II. Underweight individuals show the highest ALD risk at 1.6%, significantly higher all other categories. Normal to Obese II categories show a declining trend in ALD risk, bottoming out at 0.1%. Obese III shows a slight uptick to 0.2%, but still much lower than underweight.

We can conclude that underweight status may be associated with higher vulnerability to liver dysfunction, possibly due to malnutrition or underlying health conditions. Higher BMI categories show lower ALD risk, which may reflect protective metabolic factors.

Physical Activity (Minutes/Week) vs ALD Risk

```
In [ ]: # Bin physical activity
df_cleaned['ACTIVITY_BIN'] = pd.cut(df_cleaned['PAD615'], bins=[-1, 0, 149, 299, 600, 2000],
                                   labels=['None', 'Low (<150)', 'Moderate (150-299)', 'High (300-599)', 'Very High (>599)'])

# Summary table
activity_summary = df_cleaned.groupby('ACTIVITY_BIN')['ALD_FLAG'].agg(['sum', 'count']).reset_index()
activity_summary['Percent_ALD'] = round((activity_summary['sum'] / activity_summary['count']) * 100, 2)

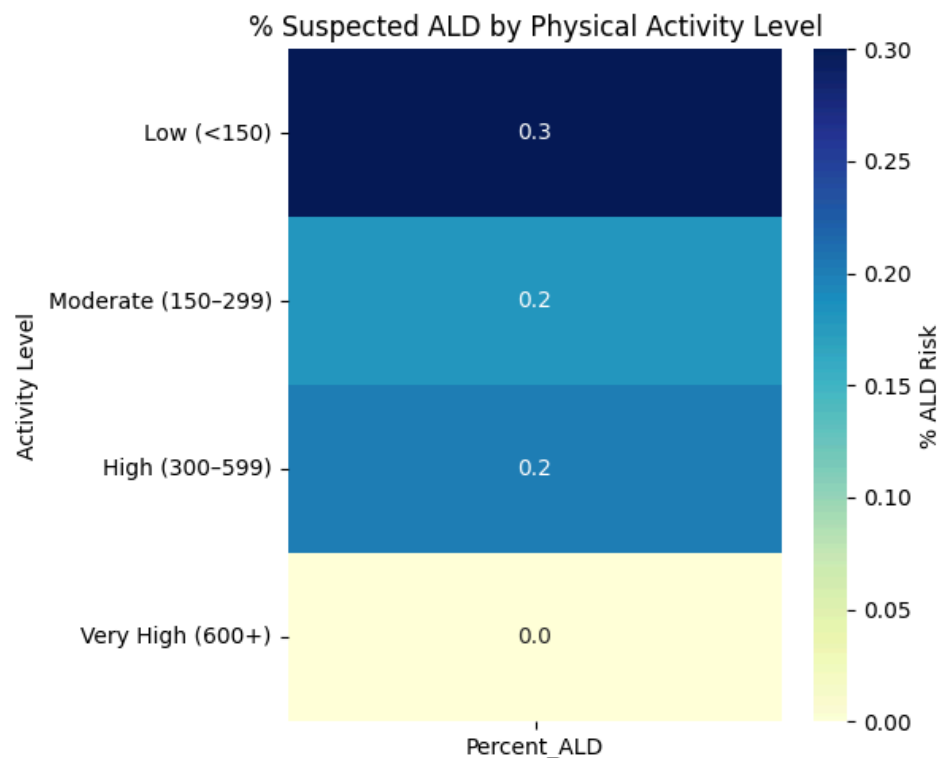
# Pivot and plot
act_heat = activity_summary.pivot_table(index='ACTIVITY_BIN', values='Percent_ALD')

plt.figure(figsize=(6, 5))
sns.heatmap(act_heat, annot=True, fmt=".1f", cmap="YlGnBu", cbar_kws={'label': '% ALD Risk'})
plt.title('% Suspected ALD by Physical Activity Level')
plt.ylabel('Activity Level')
plt.tight_layout()
plt.show()
```

/tmp/ipython-input-64-1415974976.py:6: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

activity_summary = df_cleaned.groupby('ACTIVITY_BIN')['ALD_FLAG'].agg(['sum', 'count']).reset_index()
/tmp/ipython-input-64-1415974976.py:10: FutureWarning: The default value of observed=False is deprecated and will change to observed=True in a future version of pandas. Specify observed=False to silence this warning and retain the current behavior

```
act_heat = activity_summary.pivot_table(index='ACTIVITY_BIN', values='Percent_ALD')
```



The percentage of people with suspected ALD is highest in those with low physical activity at 0.3% . Those with very high physical activity have a percentage of 0.0%, may be due to a sample size that is too little such that they do not match the liver dysfunction profile.

Alcohol Consumption (Drinks/Day) vs ALD Risk

```
In [ ]: # Filter out invalid ALQ130 values (Refused/Don't Know)
valid_alq = df_cleaned[~df_cleaned['ALQ130'].isin([777, 999])]

# Bin alcohol intake
valid_alq['ALCOHOL_BIN'] = pd.cut(valid_alq['ALQ130'], bins=[-0.1, 0, 2, 4, 6, 10, 20],
                                  labels=['None', 'Light (1-2)', 'Moderate (3-4)', 'High (5-6)', 'Heavy (7-10)'])

# Summary table
alcohol_summary = valid_alq.groupby('ALCOHOL_BIN')['ALD_FLAG'].agg(['sum', 'count']).reset_index()
alcohol_summary['Percent_ALD'] = round((alcohol_summary['sum'] / alcohol_summary['count']) * 100, 2)

# Pivot and plot
alc_heat = alcohol_summary.pivot_table(index='ALCOHOL_BIN', values='Percent_ALD')

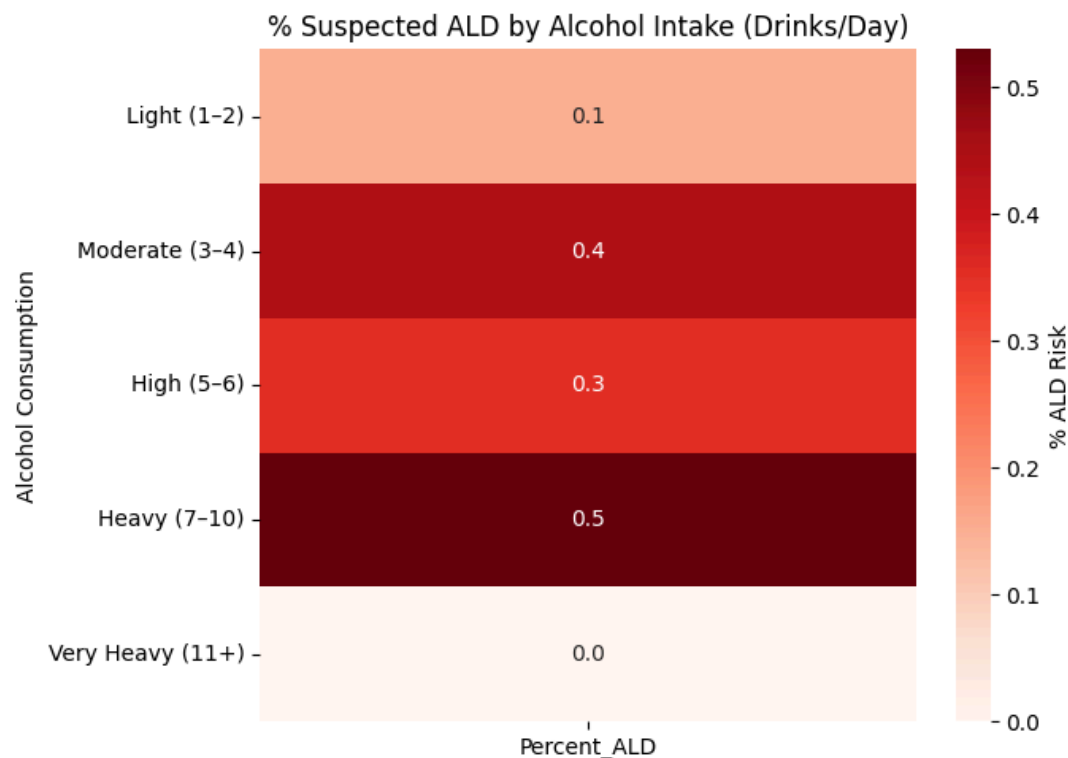
plt.figure(figsize=(7, 5))
sns.heatmap(alc_heat, annot=True, fmt=".1f", cmap="Reds", cbar_kws={'label': '% ALD Risk'})
plt.title('% Suspected ALD by Alcohol Intake (Drinks/Day)')
plt.ylabel('Alcohol Consumption')
plt.tight_layout()
plt.show()
```

/tmp/ipython-input-63-2633048176.py:5: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
valid_alq['ALCOHOL_BIN'] = pd.cut(valid_alq['ALQ130'], bins=[-0.1, 0, 2, 4, 6, 10, 20],
/tmp/ipython-input-63-2633048176.py:9: FutureWarning: The default of observed=False is deprecated and will be
changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True
e to adopt the future default and silence this warning.
```

```
alcohol_summary = valid_alq.groupby('ALCOHOL_BIN')['ALD_FLAG'].agg(['sum', 'count']).reset_index()
/tmp/ipython-input-63-2633048176.py:13: FutureWarning: The default value of observed=False is deprecated and
will change to observed=True in a future version of pandas. Specify observed=False to silence this warning an
d retain the current behavior
alc_heat = alcohol_summary.pivot_table(index='ALCOHOL_BIN', values='Percent_ALD')
```



Analysis of % suspected ALD by alcohol intake

From light to heavy drinking, the risk of suspected ALD steadily increases, peaking at 0.6 for 7–10 drinks/day. However, there is an unexpected drop in the "Very Heavy" category (11+ drinks/day) that shows 0.0% risk, which is unexpected. This may suggest that only a small sample size are "Very Heavy" drinkers. Other than the anomaly, the heat map suggests that the percentage of suspected ALD increases as the alcohol intake increases.

Sleep duration vs alcohol consumption in ALD risk

```
In [ ]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# 1. Filter valid alcohol responses (exclude refused/don't know)
df_valid = df_cleaned[~df_cleaned['ALQ130'].isin([777, 999])]

# 2. Bin alcohol intake
df_valid['ALCOHOL_BIN'] = pd.cut(df_valid['ALQ130'],
                                bins=[-0.1, 0, 2, 4, 6, 10, 20],
                                labels=['None', 'Light (1-2)', 'Moderate (3-4)', 'High (5-6)', 'Heavy (7-10)'])

# 3. Bin sleep duration (custom)
df_valid['SLEEP_BIN'] = pd.cut(df_valid['SLD010H'],
                                bins=[0, 4, 6, 8, 10, 12],
                                labels=['<4h', '4-6h', '6-8h', '8-10h', '10-12h'])

# 4. Cross-tabulate ALD risk by both alcohol & sleep bins
heatmap_data = df_valid.groupby(['SLEEP_BIN', 'ALCOHOL_BIN'])['ALD_FLAG'].agg(['sum', 'count']).reset_index()
heatmap_data['Percent_ALD'] = round((heatmap_data['sum'] / heatmap_data['count']) * 100, 2)

# 5. Pivot to create 2D matrix for heatmap
heat_matrix = heatmap_data.pivot(index='SLEEP_BIN', columns='ALCOHOL_BIN', values='Percent_ALD')

# 6. Plot heatmap
plt.figure(figsize=(10, 6))
sns.heatmap(heat_matrix, annot=True, fmt=".1f", cmap="YlOrBr", cbar_kws={'label': '% ALD Risk'})
plt.title('% Suspected ALD by Sleep Duration & Alcohol Consumption')
plt.xlabel('Alcohol Intake (Drinks/Day)')
plt.ylabel('Sleep Duration')
plt.tight_layout()
plt.show()
```



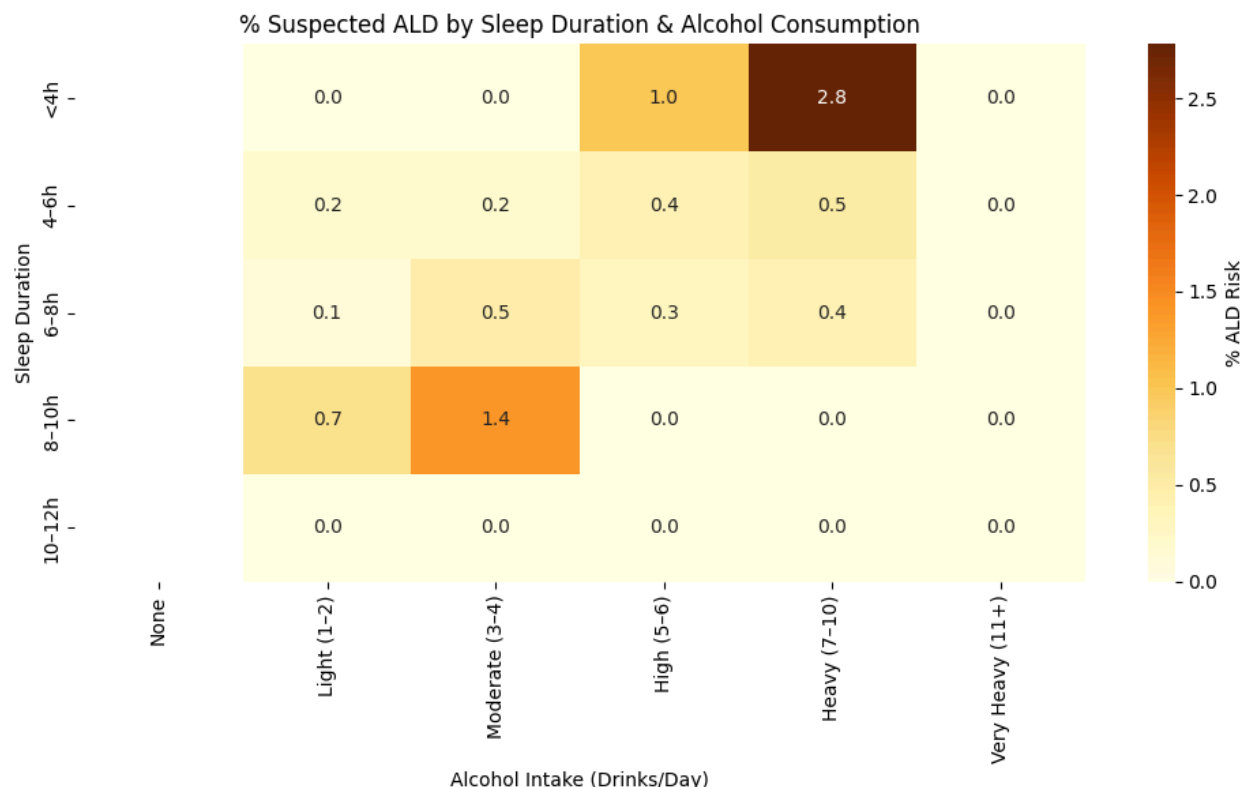
```
/tmp/ipython-input-68-3333230544.py:9: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df_valid['ALCOHOL_BIN'] = pd.cut(df_valid['ALQ130'],
/tmp/ipython-input-68-3333230544.py:14: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df_valid['SLEEP_BIN'] = pd.cut(df_valid['SLD010H'],
/tmp/ipython-input-68-3333230544.py:19: FutureWarning: The default of observed=False is deprecated and will b
e changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=T
rue to adopt the future default and silence this warning.
heatmap_data = df_valid.groupby(['SLEEP_BIN', 'ALCOHOL_BIN'])['ALD_FLAG'].agg(['sum', 'count']).reset_index
())
```



Analysis of sleep duration vs alcohol consumption in ALD risk heat map

Short sleep durations (<4h) combined with moderate to heavy alcohol intake are associated with the highest ALD risk, peaking at 2.8% when heavy drinking is paired with less than 4 hours of sleep.

Longer sleep durations (4-6h) and (6-8h) with no alcohol show no ALD risk, but as the alcohol intake increases, the percentage risk increases gradually.

However for (8-10h), the light and moderate alcohol intake has a high percentage of 0.7% and 1.4% respectively, but 0.0% for high intakes. This may be due to a small sample size, so there is not enough data to support that heavy drinking paired with longer durations of sleep has a high risk percentage.

Ignoring the anomaly, we can conclude that short sleep paired with high alcohol intake significantly increases ALD risk. Whereas, adequate sleep and no alcohol are associated with minimal or no risk. This highlights that both lifestyle factors, sleep and alcohol, jointly influence liver health.

Sleep duration vs BMI in ALD risk

```
In [ ]: # Bin BMI (body mass index)
df_cleaned['BMI_BIN'] = pd.cut(df_cleaned['BMXBMI'],
                               bins=[0, 18.5, 24.9, 29.9, 34.9, 40, 60],
                               labels=['Underweight', 'Normal', 'Overweight', 'Obese I', 'Obese II', 'Obese I

# Bin sleep duration
df_cleaned['SLEEP_BIN'] = pd.cut(df_cleaned['SLD010H'],
                                  bins=[0, 4, 6, 8, 10, 12],
                                  labels=['<4h', '4-6h', '6-8h', '8-10h', '10-12h'])

# Cross-tabulate ALD risk by BMI and sleep duration
heatmap_bmi = df_cleaned.groupby(['SLEEP_BIN', 'BMI_BIN'])['ALD_FLAG'].agg(['sum', 'count']).reset_index()
heatmap_bmi['Percent_ALD'] = round((heatmap_bmi['sum'] / heatmap_bmi['count']) * 100, 2)

# Pivot for heatmap
bmi_matrix = heatmap_bmi.pivot(index='SLEEP_BIN', columns='BMI_BIN', values='Percent_ALD')

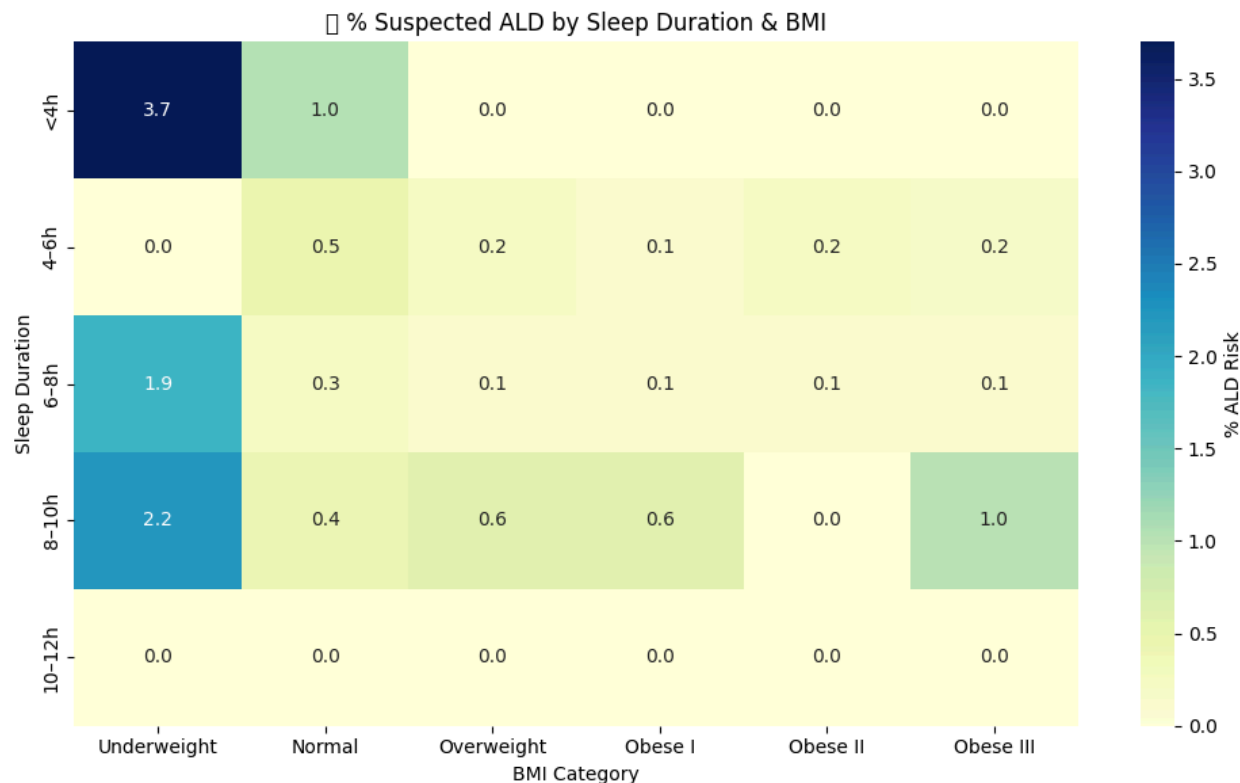
# Plot
plt.figure(figsize=(10, 6))
sns.heatmap(bmi_matrix, annot=True, fmt=".1f", cmap="YlGnBu", cbar_kws={'label': '% ALD Risk'})
plt.title('% Suspected ALD by Sleep Duration & BMI')
plt.xlabel('BMI Category')
plt.ylabel('Sleep Duration')
plt.tight_layout()
plt.show()
```

/tmp/ipython-input-67-3180202400.py:12: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

heatmap_bmi = df_cleaned.groupby(['SLEEP_BIN', 'BMI_BIN'])['ALD_FLAG'].agg(['sum', 'count']).reset_index()
/tmp/ipython-input-67-3180202400.py:24: UserWarning: Glyph 128202 (\N{BAR CHART}) missing from font(s) DejaVu Sans.

plt.tight_layout()
/usr/local/lib/python3.11/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 128202 (\N{BAR CHART}) missing from font(s) DejaVu Sans.

fig.canvas.print_figure(bytes_io, **kw)



From the heatmap, those sleeping 10-12 hours all have 0.0% of ALD risk regardless of BMI. This may be because there are too little people who reported having 10-12 hours of sleep, they do not match the liver dysfunction profile or both.

From the heatmap, those who are underweight appear to have the highest percentages of ALD risk, especially those who sleep less than 4 hours at 3.7%. Those sleeping 8-10 hours also have increased risk at 2.2%.

Those who have normal BMI and sleep less than 4 hours as well as those who have Obese III BMI and sleep 8-10 hours also have an increased risk at 1.0%.

Sleep duration vs Physical activity in ALD risk

```
In [ ]: # Bin physical activity (minutes/week)
df_cleaned['ACTIVITY_BIN'] = pd.cut(df_cleaned['PAD615'],
                                   bins=[-1, 0, 149, 299, 600, 2000],
                                   labels=['None', 'Low (<150)', 'Moderate (150-299)', 'High (300-599)', 'Very High (600+)'])

# Sleep bins already defined as SLEEP_BIN

# Cross-tabulate ALD risk by Activity and Sleep Duration
heatmap_act = df_cleaned.groupby(['SLEEP_BIN', 'ACTIVITY_BIN'])['ALD_FLAG'].agg(['sum', 'count']).reset_index()
heatmap_act['Percent_ALD'] = round((heatmap_act['sum'] / heatmap_act['count']) * 100, 2)

# Pivot for heatmap
act_matrix = heatmap_act.pivot(index='SLEEP_BIN', columns='ACTIVITY_BIN', values='Percent_ALD')

# Plot
plt.figure(figsize=(10, 6))
sns.heatmap(act_matrix, annot=True, fmt=".1f", cmap="BuPu", cbar_kws={'label': '% ALD Risk'})
plt.title('% Suspected ALD by Sleep Duration & Physical Activity')
plt.xlabel('Physical Activity Level')
plt.ylabel('Sleep Duration')
plt.tight_layout()
plt.show()
```

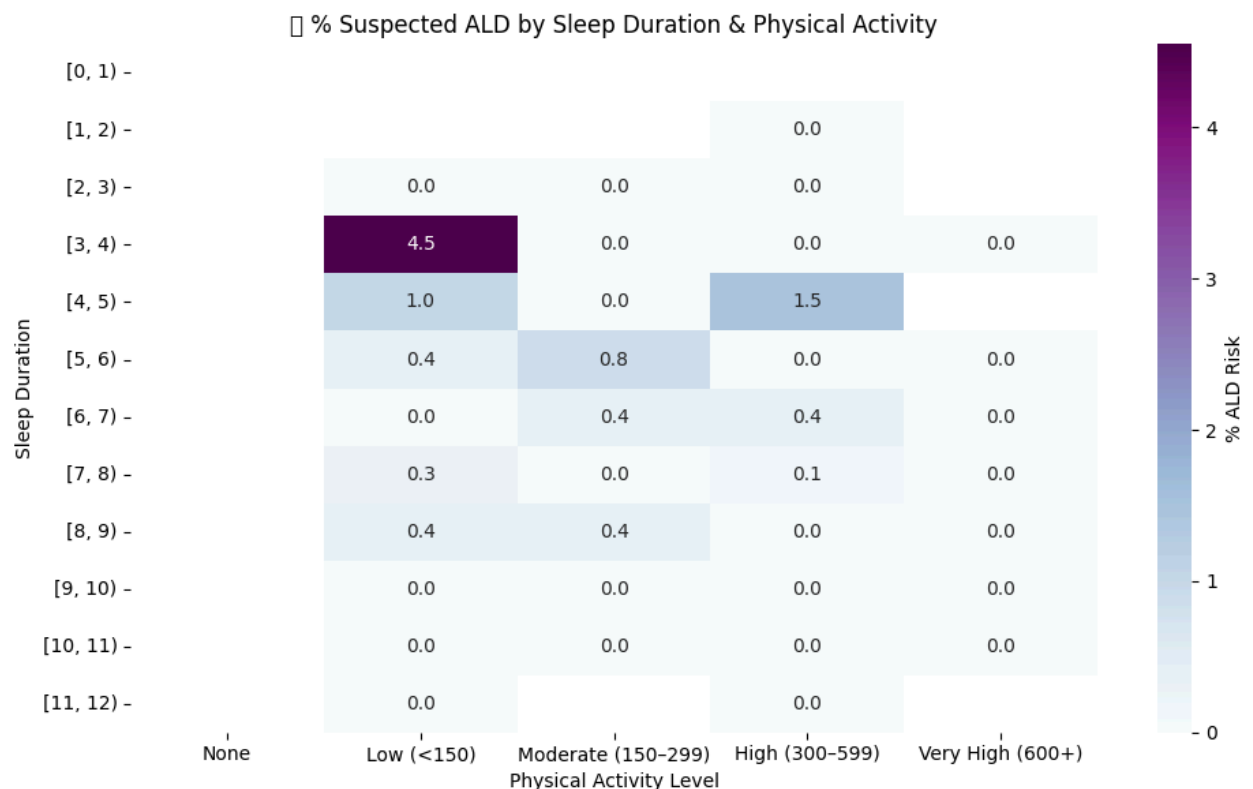
/tmp/ipython-input-66-1351512994.py:9: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
heatmap_act = df_cleaned.groupby(['SLEEP_BIN', 'ACTIVITY_BIN'])['ALD_FLAG'].agg(['sum', 'count']).reset_index()
```

/tmp/ipython-input-66-1351512994.py:21: UserWarning: Glyph 128202 (\N{BAR CHART}) missing from font(s) DejaVu Sans.

```
plt.tight_layout()
/usr/local/lib/python3.11/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 128202 (\N{BAR CHART}) missing from font(s) DejaVu Sans.
```

```
fig.canvas.print_figure(bytes_io, **kw)
```



Analysis of sleep duration vs physical activity heatmap

Short sleep durations (<4h) paired with low physical activity shows the highest suspected ALD percentage of 4.5%. If the physical activity increases, the suspected ALD drops to 0.0%. For sleep duration (4-5h), the ALD % is relatively lower 1.0% physical activity, seems to have no affect the percentage risks as the ALD% is 1.5% at high physical activity. For sleep duration (5-6h), the ALD % is 0.4% at low physical activity and 0.8% at moderate physical activity. For sleep duration (6-7h), the ALD % is 0.4% for both moderate and high activity. For sleep duration (7-8h), the ALD % is 0.3% for low physical activity but lowers to 0.1% for moderate activity For sleep duration (8-9h) the percentage is 0.4% for both low and moderate physical activity.

From the heatmap we can conclude that insufficient sleep combined with low or excessive physical activity may be linked to higher ALD risk. Whereas, moderate sleep and activity seem to correlate with lower risk.

Conclusion

To conclude:

1. The AST/ALT ratio > 2 and GGT > 50 U/L method proved useful in flagging participants with a liver profile indicative of ALD.
2. There is no strong evidence of a direct association between sleep duration and liver enzyme levels individually.
3. However, short sleep duration, when combined with higher alcohol intake and low BMI, appears to correlate with an increased risk of suspected ALD.
4. This suggests that multifactorial models (sleep + alcohol + BMI) may offer better predictive value than looking at any one factor alone.

9. References

1. Assessing liver function and interpreting liver blood tests (2024). <https://www.sps.nhs.uk/articles/assessing-liver-function-and-interpreting-liver-blood-tests/> (<https://www.sps.nhs.uk/articles/assessing-liver-function-and-interpreting-liver-blood-tests/>).
2. Medical complications: Common Alcohol-Related Concerns | National Institute on Alcohol Abuse and Alcoholism (NIAAA) (2025). <https://www.niaaa.nih.gov/health-professionals-communities/core-resource-on-alcohol/medical-complications-common-alcohol-related-concerns> (<https://www.niaaa.nih.gov/health-professionals-communities/core-resource-on-alcohol/medical-complications-common-alcohol-related-concerns>).
3. Carrion, A.F. et al. (2011) 'Chronic liver disease in the Hispanic population of the United States,' *Clinical Gastroenterology and Hepatology*, 9(10), pp. 834–841. <https://doi.org/10.1016/j.cgh.2011.04.027> (<https://doi.org/10.1016/j.cgh.2011.04.027>).
4. Tapper, E.B. and Parikh, N.D., 2018. Mortality due to cirrhosis and liver cancer in the United States, 1999–2016: observational study. *BMJ*, 362, p.k2817. <https://doi.org/10.1136/bmj.k2817> (<https://doi.org/10.1136/bmj.k2817>).
5. Yao, L. and Chen, T. (2024) 'A combined association of alanine aminotransferase, aspartate transaminase and bilirubin with sleep duration in aged 16–85 years (2005–2010),' *Medicine*, 103(49), p. e40915. <https://doi.org/10.1097/md.00000000000040915> (<https://doi.org/10.1097/md.00000000000040915>).
6. Yeung, C.H.C., Bauer, C. and Xiao, Q. (2023) 'Associations between Rest–Activity Rhythms and Liver Function Tests: The US National Health and Nutrition Examination Survey, 2011–2014,' *Clocks & Sleep*, 5(4), pp. 667–685. <https://doi.org/10.3390/clocksleep5040045> (<https://doi.org/10.3390/clocksleep5040045>).
7. Younossi, Z.M., Koenig, A.B., Abdelatif, D., Fazel, Y., Henry, L. and Wymer, M., 2016. Global epidemiology of nonalcoholic fatty liver disease—Meta-analytic assessment of prevalence, incidence, and outcomes. *Hepatology*, 64(1), pp.73–84. <https://doi.org/10.1002/hep.28431> (<https://doi.org/10.1002/hep.28431>).
8. Zong, G. et al. (2024) 'Association of sleep patterns and disorders with metabolic dysfunction-associated steatotic liver disease and liver fibrosis in contemporary American adults,' *Annals of Hepatology*, p. 101583. <https://doi.org/10.1016/j.aohep.2024.101583> (<https://doi.org/10.1016/j.aohep.2024.101583>).

